

Causal Inference Basics

05/29/2026 11:02:17



Chapter 1 — What Causal Inference Is

1. The core problem

Causal inference is about answering questions of the form:

What would happen if we changed something?

Examples:

Does studying more cause higher exam scores?

Does smoking cause lung cancer?

Does a new medicine reduce symptoms?

Does showing users a recommendation banner increase purchases?

Does attending university increase income?

These are **causal questions**, not just predictive questions.

A predictive question asks:

Given what I observe, what is likely to happen?

A causal question asks:

What would happen if I intervened and changed something?

That distinction is the entire subject.

2. Association is not causation

Suppose we observe:

Students who drink more coffee get higher exam scores.

This is an **association**.

But it does not automatically mean:

Coffee causes better exam scores.

There are many other explanations.

Maybe students who drink coffee also study longer.

Maybe older students drink more coffee and also perform better.

Maybe students who are already serious about exams drink more coffee.

So we need to distinguish:

X is associated with Y

from:

X causes Y

In causal inference, we usually write:

Treatment → Outcome

For example:

Coffee → Exam score

where:

Treatment = coffee consumption

Outcome = exam score

But the arrow is exactly what we are trying to justify.

3. The basic causal question

The ideal causal question is:

For the same person, what would their outcome be if they received the treatment versus if they did not?

Example:

For one student, we want to know:

Exam score if they drink coffee
Exam score if they do not drink coffee

The causal effect for that student would be:

score with coffee - score without coffee

Simple.

But there is a problem.

4. The fundamental problem of causal inference

For the same person, at the same time, we cannot observe both worlds.

A student either drinks coffee or does not drink coffee before the exam.

We observe only one outcome:

Student drinks coffee → observed score

But we do not observe the counterfactual:

What would their score have been if they had not drunk coffee?

This missing outcome is called the **counterfactual**.

So causal inference is difficult because we are always missing one of the two possible outcomes.

This is called:

The fundamental problem of causal inference.

5. Potential outcomes

To formalize this, causal inference often uses the **potential outcomes framework**.

Let:

$Y(1)$ = outcome if treated
 $Y(0)$ = outcome if untreated

For example:

$Y(1)$ = exam score if the student drinks coffee
 $Y(0)$ = exam score if the student does not drink coffee

The individual causal effect is:

$Y(1) - Y(0)$

But for each person, we only observe one of them.

If the student drinks coffee, we observe:

$Y(1)$

but not:

$Y(0)$

If the student does not drink coffee, we observe:

$Y(0)$

but not:

$Y(1)$

So the table looks like this:

Student	Treatment	Observed outcome	Missing counterfactual
A	Coffee	$Y(1)$	$Y(0)$
B	No coffee	$Y(0)$	$Y(1)$

Causal inference is basically about estimating the missing counterfactuals well enough to make valid conclusions.

6. Average Treatment Effect

Because we usually cannot know the causal effect for every individual, we often estimate the **average causal effect**.

This is called the **Average Treatment Effect**, or **ATE**.

$$\text{ATE} = E[Y(1) - Y(0)]$$

Meaning:

On average, how much does the treatment change the outcome?

Example:

$$\text{ATE} = \text{average exam score with coffee} - \text{average exam score without coffee}$$

If:

$$\begin{aligned} E[Y(1)] &= 78 \\ E[Y(0)] &= 72 \end{aligned}$$

then:

$$\text{ATE} = 78 - 72 = 6$$

Interpretation:

Coffee increases exam score by 6 points on average.

But only if the causal assumptions are valid.

7. Why simple group comparison can fail

Suppose we compare:

Average score of coffee drinkers - average score of non-coffee drinkers

This gives an observed difference:

$$E[Y \mid T = 1] - E[Y \mid T = 0]$$

where:

$T = 1$ means treated

$T = 0$ means untreated

But this is not automatically equal to:

$$E[Y(1)] - E[Y(0)]$$

The observed comparison is:

People who chose coffee vs people who did not choose coffee

The causal comparison is:

The same kind of people under coffee vs no coffee

Those are different.

Why?

Because people who choose treatment may already be different from people who do not.

This is where **confounding** enters.

8. Confounding

A **confounder** is a variable that affects both the treatment and the outcome.

Example:

Study time → Coffee consumption

Study time → Exam score

Students who study more may drink more coffee.

Students who study more may also score higher.

So coffee and exam score become associated, even if coffee itself has no effect.

The causal diagram is:

```
graph TD
    A[Study time] --> B[Coffee]
    A --> C[Exam score]
    B --> C
```

Here, study time is a confounder.

It creates a non-causal path between coffee and exam score:

```
graph LR
    A[Coffee] --> B[Study time]
    B --> C[Exam score]
```

This makes coffee look more beneficial than it really is.

9. Randomized experiments

The cleanest way to estimate causality is through a randomized experiment.

Example:

Randomly assign students into two groups:

```
Group 1: drink coffee
Group 2: do not drink coffee
```

Because assignment is random, the two groups should be similar on average.

So study time, motivation, sleep, intelligence, and other factors should balance out across groups.

Then:

```
difference in average scores ≈ causal effect of coffee
```

This is why randomized controlled trials are considered strong evidence.

Randomization breaks the link between confounders and treatment.

Instead of:

Study time → Coffee

we get:

Random assignment → Coffee

So the treatment is no longer chosen based on background characteristics.

10. Observational studies

In many cases, we cannot randomize.

We cannot randomly assign people to smoke for 20 years.

We cannot randomly assign students to be poor or rich.

We cannot randomly assign countries into wars.

So we use **observational data**.

Observational data means treatment was not randomly assigned.

Examples:

Some people smoke, some do not.
Some people attend university, some do not.
Some users click ads, some do not.
Some patients receive a medicine, some do not.

The problem is that treatment choice may depend on other variables.

So the main challenge becomes:

How do we make observational data behave as much like a randomized experiment as possible?

That is a major part of causal inference.

11. The four major threats

When estimating causal effects, you constantly worry about these problems.

1. Confounding

A third variable affects both treatment and outcome.

Example:

Study time affects both coffee consumption and exam score.

2. Selection bias

People select into treatment in a non-random way.

Example:

Sicker patients are more likely to receive medicine.

If treated patients have worse outcomes, it may not mean the medicine is bad. It may mean they were already sicker.

3. Reverse causality

The supposed outcome actually affects the supposed cause.

Example:

Does exercise reduce stress?

Maybe:

Stress reduces exercise.

The direction may be reversed.

4. Missing variables

Important causes are not measured.

Example:

You measure:

Coffee, exam score

but not:

study time, sleep, prior ability, motivation

Then your estimate can be badly biased.

12. Prediction vs causation

Machine learning is usually very good at prediction.

Example:

Predict exam score from coffee, study time, sleep, attendance, past grades.

A model might learn:

Coffee is correlated with high scores.

But that does not mean coffee causes high scores.

Prediction asks:

What features help forecast Y?

Causation asks:

What happens to Y if I intervene on X?

This distinction is critical.

A predictive model can use non-causal signals.

Example:

Ice cream sales predict drowning deaths.

But banning ice cream would not prevent drowning.

Why?

Because hot weather causes both:

Hot weather → Ice cream sales

Hot weather → Swimming → Drowning risk

The causal variable is weather/swimming exposure, not ice cream.

13. Causal diagrams

Causal inference often uses diagrams called **Directed Acyclic Graphs**, or **DAGs**.

A DAG contains variables and arrows.

Example:

```
Study time → Exam score
Study time → Coffee
Coffee → Exam score
```

This says:

```
Study time affects coffee.
Study time affects exam score.
Coffee may affect exam score.
```

DAGs help us reason about what variables to control for.

For example, to estimate the effect of coffee on exam score, we probably need to adjust for study time.

That means comparing coffee drinkers and non-coffee drinkers with similar study time.

14. Adjustment intuition

Suppose study time is the confounder.

A naive comparison is:

```
Coffee drinkers vs non-coffee drinkers
```

A better comparison is:

```
Coffee drinkers who studied 5 hours
vs
Non-coffee drinkers who studied 5 hours
```

Then repeat for different study levels and average the results.

This is called **adjustment** or **controlling for confounders**.

The basic idea:

Compare treated and untreated units that are similar in relevant background variables.

This can be done using regression, matching, stratification, inverse probability weighting, or other methods.

15. A tiny numerical example

Suppose we observe this:

Student	Coffee	Study hours	Score
A	Yes	8	90
B	Yes	7	85
C	No	2	60
D	No	3	65

Naive comparison:

Average coffee score = $(90 + 85) / 2 = 87.5$
Average no-coffee score = $(60 + 65) / 2 = 62.5$
Difference = 25

Naively, coffee appears to improve scores by 25 points.

But that is obviously suspicious.

Coffee drinkers studied much more.

So the observed difference combines:

effect of coffee
+ effect of study hours
+ other differences

This is confounding.

A better dataset would compare students with similar study hours.

Example:

Student	Coffee	Study hours	Score
A	Yes	5	78
B	No	5	75
C	Yes	6	82
D	No	6	79

Now the comparison is more meaningful.

For 5 hours:

$$78 - 75 = 3$$

For 6 hours:

$$82 - 79 = 3$$

Estimated causal effect:

3 points

Much smaller than 25.

16. The central logic of causal inference

Most causal inference methods follow this pattern:

1. Define the causal question.
2. Define treatment and outcome.
3. Identify possible confounders.
4. Decide what assumptions are needed.
5. Use a method to estimate the causal effect.
6. Check whether the assumptions are plausible.

The method is not the most important part.

The assumptions are.

A regression can look mathematically clean and still be causally wrong.

17. The key phrase: “all else equal”

When people say:

X causes Y

they usually mean:

Changing X changes Y, all else equal.

Example:

Coffee causes higher exam scores, all else equal.

That means comparing students who are similar in sleep, study time, prior ability, motivation, and so on, but differ in coffee consumption.

Causal inference is the formal study of how to make “all else equal” comparisons.

18. Chapter 1 summary

Causal inference studies cause-and-effect relationships.

The basic causal question is:

What would happen if we intervened?

The central problem is that we cannot observe both potential outcomes for the same unit:

$Y(1)$: outcome if treated

$Y(0)$: outcome if untreated

The individual causal effect is:

$Y(1) - Y(0)$

The average treatment effect is:

$ATE = E[Y(1) - Y(0)]$

Naive comparison can fail because treated and untreated groups may differ systematically.

The biggest early concept is **confounding**:

A confounder affects both treatment and outcome.

Randomized experiments solve this by making treatment independent of confounders.

Observational studies are harder because treatment is not randomly assigned.

The goal is to make observational comparisons approximate the logic of a randomized experiment.

19. Minimal vocabulary for Chapter 1

Term	Meaning
Treatment	The variable/intervention whose effect we care about
Outcome	The result we want to measure
Causal effect	Difference between outcome with treatment and without treatment
Counterfactual	The unobserved “what would have happened otherwise”
Potential outcomes	$Y(1)$ and $Y(0)$, the possible outcomes under treatment/no treatment
ATE	Average Treatment Effect
Confounder	A variable that affects both treatment and outcome
Randomization	Assigning treatment randomly to remove confounding
Observational data	Data where treatment was not randomly assigned
DAG	A diagram showing assumed causal relationships

20. Practice questions

Try answering these before moving to Chapter 2.

Q1

A study finds that people who own expensive laptops get better grades.

Can we conclude that expensive laptops cause better grades?

Q2

In this example, identify the treatment and outcome:

Does attending extra tutorial sessions improve final exam performance?

Q3

In this causal question, name one possible confounder:

Does playing video games reduce academic performance?

Q4

Why is a randomized experiment usually better than an observational comparison?

Q5

Suppose:

$$Y(1) = 90$$

$$Y(0) = 80$$

What is the individual causal effect?

Q6

Why can we usually not directly observe the individual causal effect?

21. Answers

A1

No.

Students with expensive laptops may differ in income, school quality, parental support, prior preparation, or study habits. These factors may affect grades independently of laptop

quality.

A2

Treatment = attending extra tutorial sessions

Outcome = final exam performance

A3

Possible confounders:

study habits

sleep

prior academic ability

time management

course difficulty

mental health

For example, students with poor time management may both play more games and score lower.

A4

Because randomization makes the treated and untreated groups similar on average. This reduces confounding and makes the difference in outcomes more plausibly caused by the treatment.

A5

$$Y(1) - Y(0) = 90 - 80 = 10$$

The treatment effect is 10 points.

A6

Because the same person cannot both receive and not receive the treatment at the same time. One potential outcome is observed; the other is counterfactual.

5/27/2026, 11:57:12 AM



Chapter 2 — Potential Outcomes and Treatment Effects

Chapter 1 gave the intuition:

Causal inference asks what would happen if we changed something.

Chapter 2 makes that precise.

The core framework here is the **potential outcomes framework**, also called the **Rubin causal model**.

1. Units, treatment, and outcome

Every causal question needs three basic pieces.

Unit

The object/person/entity being studied.

Examples:

Question	Unit
Does coffee improve exam scores?	Student
Does medicine reduce fever?	Patient
Does an ad increase purchases?	User
Does fertilizer increase crop yield?	Farm plot
Does a policy reduce unemployment?	Region/country

Treatment

The intervention or exposure whose effect we care about.

Usually denoted:

T

or sometimes:

D

For binary treatment:

$T = 1$ means treated

$T = 0$ means untreated

Example:

$T = 1$: student drinks coffee

$T = 0$: student does not drink coffee

Outcome

The result we measure.

Usually denoted:

Y

Example:

$Y = \text{exam score}$

So a causal question has the structure:

Treatment $\rightarrow$ Outcome

Example:

Coffee $\rightarrow$ Exam score

2. Potential outcomes

For each unit, there are multiple possible outcomes depending on treatment.

For binary treatment:

$Y(1)$ = outcome if treated

$Y(0)$ = outcome if untreated

Example:

For a student:

$Y(1)$ = score if they drink coffee

$Y(0)$ = score if they do not drink coffee

The individual causal effect is:

$Y(1) - Y(0)$

Example:

$Y(1) = 85$

$Y(0) = 80$

Causal effect = $85 - 80 = 5$

Meaning:

Coffee increased this student's score by 5 points.

But in reality, we cannot observe both values for the same student at the same time.

3. The observed outcome

We only observe one outcome depending on the treatment actually received.

If:

$T = 1$

then we observe:

$Y(1)$

If:

$$T = 0$$

then we observe:

$$Y(0)$$

This can be written as:

$$Y = T \cdot Y(1) + (1 - T) \cdot Y(0)$$

This formula just means:

If treated:

$$T = 1$$

$$Y = 1 \cdot Y(1) + 0 \cdot Y(0)$$

$$Y = Y(1)$$

If untreated:

$$T = 0$$

$$Y = 0 \cdot Y(1) + 1 \cdot Y(0)$$

$$Y = Y(0)$$

So the observed outcome is only one of the two potential outcomes.

4. The missing counterfactual

Suppose this is the real causal table:

Student	Y(1): score with coffee	Y(0): score without coffee	Effect
A	85	80	5
B	75	78	-3
C	90	84	6
D	70	70	0

If we had this full table, causal inference would be easy.

But in real life, we observe only this:

Student	Treatment	Observed score	Missing outcome
A	Coffee	85	$Y(0)$
B	No coffee	78	$Y(1)$
C	Coffee	90	$Y(0)$
D	No coffee	70	$Y(1)$

The unobserved value is the **counterfactual**.

For A, we know:

A drank coffee and scored 85.

But we do not know:

What would A have scored without coffee?

That is the missing counterfactual.

Causal inference is the art of estimating these missing counterfactuals under defensible assumptions.

5. Individual Treatment Effect

The **Individual Treatment Effect**, or **ITE**, is:

$$\text{ITE}_i = Y_i(1) - Y_i(0)$$

where **i** refers to one unit.

Example:

$$\text{ITE}_A = Y_A(1) - Y_A(0)$$

If:

$$Y_A(1) = 85$$

$$Y_A(0) = 80$$

then:

$$ITE_A = 5$$

Meaning:

Treatment helped unit A by 5 points.

In practice, ITE is usually impossible to know exactly because one of the two outcomes is missing.

6. Average Treatment Effect

The **Average Treatment Effect**, or **ATE**, is the average effect across the whole population.

$$ATE = E[Y(1) - Y(0)]$$

This is equivalent to:

$$ATE = E[Y(1)] - E[Y(0)]$$

Meaning:

Average outcome if everyone were treated minus average outcome if everyone were untreated.

Example:

Student	Y(1)	Y(0)	Effect
A	85	80	5
B	75	78	-3
C	90	84	6
D	70	70	0

ATE:

$$\text{ATE} = (5 + (-3) + 6 + 0) / 4$$

$$\text{ATE} = 8 / 4$$

$$\text{ATE} = 2$$

So the average effect is:

2 points

Alternatively:

$$\text{Average } Y(1) = (85 + 75 + 90 + 70) / 4 = 80$$

$$\text{Average } Y(0) = (80 + 78 + 84 + 70) / 4 = 78$$

$$\text{ATE} = 80 - 78 = 2$$

Same result.

7. Average Treatment Effect on the Treated

Sometimes we do not care about everyone.

We care about the effect on the people who actually received treatment.

This is called the **Average Treatment Effect on the Treated**, or **ATT**.

$$\text{ATT} = E[Y(1) - Y(0) \mid T = 1]$$

Meaning:

Among the treated group, how much did treatment help or hurt?

Example:

Suppose students A and C drank coffee.

Student	Treatment	Y(1)	Y(0)	Effect
A	Coffee	85	80	5

Student	Treatment	Y(1)	Y(0)	Effect
B	No coffee	75	78	-3
C	Coffee	90	84	6
D	No coffee	70	70	0

ATT only averages over A and C:

$$\text{ATT} = (5 + 6) / 2$$

$$\text{ATT} = 5.5$$

So among coffee drinkers, coffee increased scores by 5.5 points on average.

8. Average Treatment Effect on the Untreated

The **Average Treatment Effect on the Untreated**, or **ATU**, asks:

$$\text{ATU} = E[Y(1) - Y(0) \mid T = 0]$$

Meaning:

Among the untreated group, what would the effect have been if they had received treatment?

Example:

Students B and D did not drink coffee.

$$\text{ATU} = (-3 + 0) / 2$$

$$\text{ATU} = -1.5$$

So among non-coffee drinkers, coffee would have decreased scores by 1.5 points on average.

This is useful because treatment may not affect everyone the same way.

9. Treatment effects can be heterogeneous

A treatment does not need to have the same effect on everyone.

Example:

Coffee may help tired students but hurt anxious students.

Student type	Effect of coffee
Sleepy student	+8
Alert student	+1
Anxious student	-5

This is called **treatment effect heterogeneity**.

Meaning:

The causal effect varies across units.

This is why ATE can hide important differences.

Example:

Group 1 effect = +10
Group 2 effect = -10
Average effect = 0

The ATE says no average effect, but that is misleading.

The treatment helps one group and harms another.

10. Conditional Average Treatment Effect

The **Conditional Average Treatment Effect**, or **CATE**, is the average effect for a subgroup.

$$\text{CATE}(x) = E[Y(1) - Y(0) \mid X = x]$$

where x is some characteristic.

Example:

x = sleep level

Then:

CATE(sleep deprived) = effect of coffee among sleep-deprived students
CATE(well rested) = effect of coffee among well-rested students

Example table:

Group	Average Y(1)	Average Y(0)	CATE
Sleep-deprived	78	70	+8
Well-rested	85	84	+1
Anxious	72	77	-5

CATE is extremely important in medicine, policy, and recommender systems because the average effect may not be the effect for a specific subgroup.

11. Observed difference vs causal effect

Now the most important distinction.

The observed difference in outcomes is:

$$E[Y \mid T = 1] - E[Y \mid T = 0]$$

The causal effect is:

$$E[Y(1)] - E[Y(0)]$$

These are not the same thing.

The observed difference compares:

People who actually got treated
vs
People who actually did not get treated

The causal effect compares:

The same population if treated
vs

The same population if untreated

That is a huge difference.

12. Why the observed difference is biased

Suppose we observe:

Student	Coffee?	Score
A	Yes	90
B	Yes	85
C	No	70
D	No	65

Naive difference:

Coffee average = 87.5
No coffee average = 67.5
Difference = 20

But maybe the real potential outcomes are:

Student	Coffee?	Y(1)	Y(0)	Effect
A	Yes	90	88	2
B	Yes	85	83	2
C	No	72	70	2
D	No	67	65	2

The real ATE is:

2

But the observed difference is:

20

Why?

Because the coffee group was already stronger.

Even without coffee:

A would score 88
B would score 83

while the no-coffee group would score:

C would score 70
D would score 65

So the observed difference includes both:

causal effect of coffee
+ pre-existing differences between groups

This is selection bias/confounding.

13. Bias decomposition

The naive observed difference is:

$$E[Y \mid T = 1] - E[Y \mid T = 0]$$

Because observed Y depends on treatment:

$$E[Y \mid T = 1] = E[Y(1) \mid T = 1]$$

and:

$$E[Y \mid T = 0] = E[Y(0) \mid T = 0]$$

So the naive difference is:

$$E[Y(1) \mid T = 1] - E[Y(0) \mid T = 0]$$

But ATT is:

$$E[Y(1) \mid T = 1] - E[Y(0) \mid T = 1]$$

Notice the difference.

The first term is the same:

$$E[Y(1) \mid T = 1]$$

But the second term differs:

Naïve comparison uses: $E[Y(0) \mid T = 0]$

ATT needs: $E[Y(0) \mid T = 1]$

The missing counterfactual for ATT is:

What would treated people have experienced without treatment?

Bias appears when:

$$E[Y(0) \mid T = 1] \neq E[Y(0) \mid T = 0]$$

Meaning:

The treated and untreated groups would have had different outcomes even without treatment.

That is the core mathematical definition of selection bias.

14. Simple numeric bias example

Suppose:

$$E[Y(1) \mid T = 1] = 90$$

$$E[Y(0) \mid T = 1] = 80$$

$$E[Y(0) \mid T = 0] = 70$$

ATT:

```
ATT = E[Y(1) | T = 1] - E[Y(0) | T = 1]
ATT = 90 - 80
ATT = 10
```

Naive difference:

```
Naive = E[Y(1) | T = 1] - E[Y(0) | T = 0]
Naive = 90 - 70
Naive = 20
```

Bias:

```
Naive - ATT = 20 - 10 = 10
```

Why biased upward?

Because treated people would already have scored higher even without treatment:

```
E[Y(0) | T = 1] = 80
E[Y(0) | T = 0] = 70
```

So the treated group had a 10-point baseline advantage.

15. The assignment mechanism

The **assignment mechanism** is the process that determines who gets treated.

Example:

```
T = 1 if student drinks coffee
T = 0 otherwise
```

But why does a student drink coffee?

Possible assignment mechanisms:

```
Random assignment
Self-selection
Doctor decision
Algorithm decision
Teacher recommendation
```

Income/access constraint
Government policy rule

The assignment mechanism matters because it determines whether treated and untreated units are comparable.

Random assignment

Random number → Coffee

Treatment does not depend on student characteristics.

This makes causal estimation easier.

Non-random assignment

Study time → Coffee
Motivation → Coffee
Sleep deprivation → Coffee

Treatment depends on characteristics that may also affect the outcome.

This creates confounding.

16. Independence assumption

The clean causal condition is:

$$(Y(1), Y(0)) \perp\!\!\!\perp T$$

Read as:

The potential outcomes are independent of treatment assignment.

Meaning:

Who gets treated is unrelated to what their outcomes would have been under either condition.

This is true in a randomized experiment, at least approximately.

If:

$$(Y(1), Y(0)) \perp\!\!\!\perp T$$

then:

$$E[Y(1) \mid T = 1] = E[Y(1) \mid T = 0] = E[Y(1)]$$

and:

$$E[Y(0) \mid T = 1] = E[Y(0) \mid T = 0] = E[Y(0)]$$

So the treated and untreated groups are comparable.

Then:

$$E[Y \mid T = 1] - E[Y \mid T = 0]$$

can estimate:

$$E[Y(1)] - E[Y(0)]$$

That is the ATE.

17. Conditional independence

In observational studies, full independence usually fails.

But sometimes we can get conditional independence after adjusting for confounders.

The condition is:

$$(Y(1), Y(0)) \perp\!\!\!\perp T \mid X$$

Read as:

Potential outcomes are independent of treatment assignment conditional on X.

Meaning:

Within groups with the same X values, treatment is as-if random.

Example:

X = study time, sleep, prior grades

Then we assume:

Among students with same study time, same sleep, and same prior grades, coffee use is as good as random.

This is a strong assumption.

It says we have measured all relevant confounders.

This assumption is also called:

unconfoundedness
selection on observables
ignorability
conditional exchangeability

Different fields use different names.

18. Exchangeability

Exchangeability means treated and untreated units can stand in for each other.

In a randomized experiment:

The treated group represents what would happen if the untreated group were treated.

The untreated group represents what would happen if the treated group were untreated.

So the groups are exchangeable.

In observational data, treated and untreated units are often not exchangeable.

Example:

Patients who receive a strong medicine may be sicker.
Patients who do not receive it may be healthier.

Their outcomes are not directly comparable.

To estimate causality, we need exchangeability either through:

- randomization
- adjustment
- matching
- instrumental variables
- natural experiments
- difference-in-differences
- regression discontinuity

We will study those later.

19. Positivity / overlap

Another important assumption is **positivity**, also called **overlap**.

It means every type of unit has some chance of receiving each treatment.

Formally:

$$0 < P(T = 1 \mid X = x) < 1$$

for every relevant value of x .

Meaning:

For each kind of person, there must be both treated and untreated examples.

Example:

Suppose all students who study 10 hours drink coffee, and none avoid coffee.

Then for 10-hour students:

$$P(\text{Coffee} = 1 \mid \text{Study hours} = 10) = 1$$

There is no no-coffee comparison group for them.

So we cannot estimate what would happen to 10-hour students without coffee from the data.

Another example:

Study hours	Coffee students	No-coffee students	Can compare?
2	Yes	Yes	Yes
5	Yes	Yes	Yes
10	Yes	No	No

For study hours = 10, there is no overlap.

This creates extrapolation problems.

20. SUTVA

SUTVA stands for:

Stable Unit Treatment Value Assumption

It has two main parts.

1. No interference

One unit's treatment should not affect another unit's outcome.

Example where this holds reasonably:

One student drinking coffee probably does not affect another student's exam score.

Example where it fails:

Vaccination

If one person gets vaccinated, it may reduce transmission to others.

So one person's treatment affects another person's outcome.

That violates no interference.

2. No hidden versions of treatment

The treatment should be well-defined.

Example:

$T = 1$ means "drank coffee"

But what kind of coffee?

small coffee
large coffee
espresso
latte
with sugar
without sugar
1 hour before exam
5 minutes before exam

These may have different effects.

If they are all collapsed into one treatment, the causal question becomes blurry.

A good causal question defines treatment precisely.

Better:

$T = 1$: drinks 200 mg caffeine 60 minutes before exam
 $T = 0$: drinks placebo 60 minutes before exam

21. Consistency

Consistency means:

The observed outcome equals the potential outcome corresponding to the treatment actually received.

Formally:

If $T = 1$, then $Y = Y(1)$
If $T = 0$, then $Y = Y(0)$

This sounds obvious, but it requires treatment to be well-defined.

Example:

If $T = 1$ means “uses an online learning platform,” but students use very different platforms in very different ways, then $Y(1)$ is not a clean concept.

Consistency asks:

Did the treatment we observed actually correspond to the treatment we defined?

22. The three big assumptions for basic causal identification

To estimate causal effects from data, the standard assumptions are:

1. Exchangeability

$$(Y(1), Y(0)) \perp\!\!\!\perp T \mid X$$

After controlling for X , treatment is as-if random.

2. Positivity

$$0 < P(T = 1 \mid X = x) < 1$$

For every relevant type of unit, both treatment options are possible.

3. Consistency / SUTVA

The treatment is well-defined, and one unit's treatment does not improperly affect another unit's outcome.

Together, these allow us to connect observed data to causal effects.

23. Identification vs estimation

This distinction matters.

Identification

Identification asks:

Can the causal effect be expressed using observable quantities?

Example:

If assumptions hold, then:

$$ATE = E_X[E[Y \mid T = 1, X] - E[Y \mid T = 0, X]]$$

This expresses the causal effect using observed data.

That is identification.

Estimation

Estimation asks:

Given finite data, how do we numerically estimate that quantity?

Example methods:

- regression
- matching
- stratification
- inverse probability weighting
- causal forests
- double machine learning

Identification is conceptual.

Estimation is statistical/computational.

A causal effect can be statistically estimated very precisely but still be wrongly identified if the assumptions are false.

24. Identification using adjustment

Suppose conditional exchangeability holds:

$$(Y(1), Y(0)) \perp\!\!\!\perp T \mid X$$

Then:

$$E[Y(1) \mid X] = E[Y \mid T = 1, X]$$

and:

$$E[Y(0) \mid X] = E[Y \mid T = 0, X]$$

So:

$$ATE = E[Y(1) - Y(0)]$$

becomes:

$$ATE = E_X[E[Y \mid T = 1, X] - E[Y \mid T = 0, X]]$$

Meaning:

1. Compare treated vs untreated within the same value of X.
2. Average those comparisons across the population.

Example:

If $X = \text{study hours}$, compare coffee and no-coffee students within the same study-hour level.

25. Simple adjustment example

Suppose:

Study hours	Coffee avg score	No-coffee avg score	Difference	Number of students
Low	65	63	2	50
Medium	75	72	3	100
High	90	87	3	50

The adjusted ATE is the weighted average of within-group differences:

$$\begin{aligned} ATE &= (50/200)(2) + (100/200)(3) + (50/200)(3) \\ ATE &= 0.25(2) + 0.5(3) + 0.25(3) \end{aligned}$$

$$\text{ATE} = 0.5 + 1.5 + 0.75$$

$$\text{ATE} = 2.75$$

So after adjusting for study hours:

$$\text{Estimated effect} = 2.75 \text{ points}$$

This may be very different from the naive comparison.

26. Why you should not blindly control for everything

A common mistake:

Just control for every variable.

This is wrong.

Some variables should be controlled for.

Some should not.

Control for confounders

Example:

Study time → Coffee

Study time → Exam score

Study time should be controlled for.

Do not control for mediators when estimating total effect

A **mediator** lies on the causal path.

Example:

Coffee → Alertness → Exam score

If you control for alertness, you block part of coffee's effect.

That estimates the direct effect of coffee not through alertness, not the total effect.

Do not control for colliders

A **collider** is a variable caused by both treatment and outcome, or by causes of treatment and outcome.

Example:

Coffee → Feeling energetic ← Good sleep

Conditioning on a collider can create fake associations.

DAGs help decide what to control for.

This becomes Chapter 3.

27. Chapter 2 summary

The potential outcomes framework defines causality using hypothetical outcomes:

$Y(1)$ = outcome if treated
 $Y(0)$ = outcome if untreated

The individual treatment effect is:

$ITE = Y(1) - Y(0)$

But we usually cannot observe both potential outcomes for the same unit.

The major estimands are:

Estimand	Meaning
ITE	Effect for one unit
ATE	Average effect in the whole population
ATT	Average effect among treated units
ATU	Average effect among untreated units
CATE	Average effect within a subgroup

The observed difference:

$$E[Y \mid T = 1] - E[Y \mid T = 0]$$

is not necessarily causal.

The causal effect:

$$E[Y(1)] - E[Y(0)]$$

requires assumptions.

The main assumptions are:

exchangeability
positivity
consistency / SUTVA

Identification asks whether the causal effect can be expressed using observed data.

Estimation asks how to compute it from finite samples.

28. Minimal vocabulary for Chapter 2

Term	Meaning
Unit	Individual/entity being studied
Treatment	Intervention/exposure
Outcome	Result being measured
Potential outcome	Outcome under a possible treatment condition
Counterfactual	Potential outcome not observed
ITE	Individual Treatment Effect
ATE	Average Treatment Effect
ATT	Average Treatment Effect on the Treated
ATU	Average Treatment Effect on the Untreated
CATE	Conditional Average Treatment Effect

Term	Meaning
Assignment mechanism	Process determining treatment status
Exchangeability	Treated and untreated are comparable
Positivity	Each subgroup has both treated and untreated units
SUTVA	No interference and no hidden treatment versions
Consistency	Observed outcome equals relevant potential outcome
Identification	Expressing causal effect using observed data
Estimation	Numerically estimating from sample data

29. Practice questions

Q1

Define:

$Y(1)$
 $Y(0)$

for this question:

Does attending extra tutorials improve exam scores?

Q2

A student has:

$Y(1) = 88$
 $Y(0) = 82$

What is their individual treatment effect?

Q3

Why is this not automatically causal?

$$E[Y \mid T = 1] - E[Y \mid T = 0]$$

Q4

What does ATT measure?

Q5

What is treatment effect heterogeneity?

Q6

What does positivity mean?

Q7

Why might SUTVA fail in a vaccine study?

Q8

Why should you not control for a mediator when estimating the total effect?

30. Answers

A1

$Y(1)$ = exam score if the student attends extra tutorials

$Y(0)$ = exam score if the student does not attend extra tutorials

A2

$ITE = Y(1) - Y(0)$

$ITE = 88 - 82$

$ITE = 6$

The individual treatment effect is 6 points.

A3

Because it compares people who actually selected into treatment with people who did not. These groups may differ in motivation, background, ability, health, income, or other factors. So the observed difference may include confounding, not just the causal effect.

A4

ATT measures:

$$E[Y(1) - Y(0) \mid T = 1]$$

Meaning:

The average causal effect among those who actually received treatment.

A5

Treatment effect heterogeneity means the treatment effect differs across people or subgroups.

Example:

Coffee may help sleepy students, barely affect rested students, and hurt anxious students.

A6

Positivity means every relevant type of unit has a nonzero chance of receiving each treatment.

Formally:

$$0 < P(T = 1 \mid X = x) < 1$$

There must be both treated and untreated examples within each subgroup we want to compare.

A7

Because one person's vaccination can affect another person's infection risk by reducing transmission.

So one unit's treatment affects another unit's outcome, violating the no-interference part of SUTVA.

A8

Because a mediator lies on the causal pathway.

Example:

Coffee → Alertness → Exam score

If you control for alertness, you block part of coffee's effect. Then you are no longer estimating the total effect of coffee.

5/27/2026, 12:31:37 PM



Chapter 3 — Causal Graphs, DAGs, and What to Control For

We should cover this before DoWhy.

DoWhy depends heavily on causal graphs. If you do not understand DAGs, backdoor paths, colliders, mediators, and adjustment sets, DoWhy will feel like magic syntax rather than causal reasoning.

1. Why causal graphs matter

In Chapter 2, we said causal inference needs assumptions like:

$$(Y(1), Y(0)) \perp\!\!\!\perp T \mid X$$

Meaning:

Treatment is as-good-as-random after controlling for X.

But the hard question is:

What exactly should X contain?

Should we control for age?

Should we control for income?

Should we control for post-treatment variables?

Should we control for everything?

No.

Causal graphs help answer:

Which variables should we adjust for?

Which variables should we not adjust for?

Which causal effect are we estimating?

A causal graph is not learned automatically from data in most practical cases. It encodes your causal assumptions.

2. Directed Acyclic Graphs

A **Directed Acyclic Graph**, or **DAG**, is a graph with:

```
variables = nodes  
causal relationships = arrows
```

Example:

```
Study Time → Exam Score
```

means:

Study time causally affects exam score.

A DAG is **directed** because arrows have direction.

A DAG is **acyclic** because you cannot follow arrows and return to the same node.

Allowed:

```
A → B → C
```

Not allowed in a DAG:

```
A → B → C → A
```

That would be a cycle.

3. Basic causal graph vocabulary

Suppose:

```
A → B → C
```

Then:

```
A is a parent of B  
B is a child of A
```

A is an ancestor of C
C is a descendant of A

In causal terms:

A causes B
B causes C
A indirectly causes C through B

Another graph:

$A \rightarrow C$
 $B \rightarrow C$

Here, both A and B are parents of C.

4. Treatment, outcome, and covariates

Usually we care about:

$T \rightarrow Y$

where:

T = treatment
Y = outcome

Other variables are often called:

X = covariates

But not all covariates play the same causal role.

A variable can be:

confounder
mediator
collider
instrument

proxy
effect modifier

The biggest beginner mistake is treating all covariates as “controls.”

They are not.

5. Confounders

A **confounder** affects both the treatment and the outcome.

Example:

Study Time → Coffee
Study Time → Exam Score
Coffee → Exam Score

Graph:

Study Time
↙ ↘
Coffee → Exam Score

Here:

T = Coffee
Y = Exam Score
C = Study Time

Study time is a confounder because:

Study Time → Coffee
Study Time → Exam Score

It creates a non-causal association between coffee and exam score.

The problematic path is:

Coffee ← Study Time → Exam Score

This path is not the causal effect of coffee. It is a backdoor path.

To estimate the effect of coffee on exam score, we should adjust for study time.

6. Backdoor paths

A **backdoor path** is a path from treatment to outcome that begins with an arrow into the treatment.

Example:

```
Coffee ← Study Time → Exam Score
```

This path starts with:

```
Coffee ← Study Time
```

So it is a backdoor path.

It creates confounding because coffee and exam score become associated through study time.

The causal path we care about is:

```
Coffee → Exam Score
```

The non-causal backdoor path is:

```
Coffee ← Study Time → Exam Score
```

To estimate the causal effect, we want to block backdoor paths while leaving the real causal path open.

7. Adjustment

To **adjust for** a variable means to compare treated and untreated units within the same level of that variable.

Example:

Instead of comparing:

```
coffee drinkers vs non-coffee drinkers
```

compare:

coffee drinkers who studied 5 hours
vs
non-coffee drinkers who studied 5 hours

Then repeat across study-hour groups and average.

In regression, adjustment looks like:

$$\text{ExamScore} = \beta_0 + \beta_1 \text{Coffee} + \beta_2 \text{StudyTime} + \text{error}$$

Here, β_1 is the estimated effect of coffee after holding study time fixed.

But this interpretation only works if the graph assumptions are correct.

8. The backdoor criterion

A set of variables Z satisfies the **backdoor criterion** for estimating the effect of T on Y if:

1. Z blocks every backdoor path from T to Y .
2. Z does not contain descendants of T .

Simplified intuition:

*Control for common causes of treatment and outcome.
Do not control for consequences of the treatment.*

Example:

Study Time $\rightarrow$ Coffee
Study Time $\rightarrow$ Exam Score
Coffee $\rightarrow$ Exam Score

The backdoor path is:

Coffee $\leftarrow$ Study Time $\rightarrow$ Exam Score

So:

$Z = \{\text{Study Time}\}$

is a valid adjustment set.

Then the causal effect can be identified by:

$$\text{ATE} = E_X[E[Y \mid T = 1, X] - E[Y \mid T = 0, X]]$$

where:

$X = \text{Study Time}$

9. Mediators

A **mediator** lies on the causal path from treatment to outcome.

Example:

Coffee → Alertness → Exam Score

Here:

Alertness

is a mediator.

Coffee affects alertness. Alertness affects exam score.

If you want the **total effect** of coffee on exam score, do not control for alertness.

Why?

Because the total effect includes:

Coffee → Alertness → Exam Score

If you control for alertness, you block this path.

Then you are estimating only the direct effect:

Coffee → Exam Score

not the full effect.

10. Total effect vs direct effect

Suppose:

```
Coffee → Alertness → Exam Score  
Coffee → Exam Score
```

There are two causal paths:

1. Coffee → Exam Score
2. Coffee → Alertness → Exam Score

The **total effect** includes both paths.

The **direct effect** tries to isolate only:

```
Coffee → Exam Score
```

while blocking:

```
Coffee → Alertness → Exam Score
```

So:

```
Do not control for mediators when estimating total effect.  
Control for mediators only if you explicitly want a direct effect.
```

Most beginner causal questions ask for the total effect.

Example:

Does coffee improve exam scores?

That usually means total effect. So do not control for alertness.

11. Colliders

A **collider** is a variable caused by two other variables.

Basic structure:

$$A \rightarrow C \leftarrow B$$

Here, **C** is a collider.

The important rule:

Do not control for colliders.

Why?

Because conditioning on a collider can create a fake association between its causes.

12. Collider example

Suppose:

$$\text{Talent} \rightarrow \text{Admission} \leftarrow \text{Wealth}$$

Universities admit students based on both talent and wealth.

In the general population, talent and wealth may be unrelated.

But among admitted students, if someone has less wealth, they likely needed more talent to get in.

So conditioning on admission creates a negative association between talent and wealth.

The graph:

$$\text{Talent} \rightarrow \text{Admission} \leftarrow \text{Wealth}$$

Normally, the path is blocked at the collider:

$$\text{Talent} \rightarrow \text{Admission} \leftarrow \text{Wealth}$$

But if you condition on admission, the path opens.

This is called **collider bias** or **selection bias**.

13. Collider bias in treatment-outcome analysis

Suppose we want the effect of coffee on exam score.

Imagine:

Coffee → Alertness ← Good Sleep
Good Sleep → Exam Score

Graph:

Coffee → Alertness ← Good Sleep → Exam Score

Here, alertness is a collider on the path:

Coffee → Alertness ← Good Sleep → Exam Score

If we control for alertness, we open a non-causal path between coffee and good sleep.

That can bias the coffee effect.

This is why “control for everything” is dangerous.

Some controls reduce bias.

Some controls create bias.

14. Forks, chains, and colliders

There are three basic path structures.

1. Fork

$A \leftarrow C \rightarrow B$

C is a common cause.

Example:

Coffee ← Study Time → Exam Score

This path is open by default.

Control for **C** to block it.

Control for confounders.

2. Chain

$A \rightarrow C \rightarrow B$

C is a mediator.

Example:

Coffee $\rightarrow$ Alertness $\rightarrow$ Exam Score

This path is open by default.

Control for **C** to block it.

Do not control for mediators if estimating total effect.

3. Collider

$A \rightarrow C \leftarrow B$

C is a common effect.

Example:

Talent $\rightarrow$ Admission $\leftarrow$ Wealth

This path is blocked by default.

Controlling for **C** opens it.

Do not control for colliders.

Summary:

Structure	Name	Open by default?	What conditioning does
$A \leftarrow C \rightarrow B$	Fork/confounder	Yes	Blocks path

Structure	Name	Open by default?	What conditioning does
$A \rightarrow C \rightarrow B$	Chain/mediator	Yes	Blocks path
$A \rightarrow C \leftarrow B$	Collider	No	Opens path

15. D-separation

D-separation is the graphical rule for determining whether paths are blocked or open.

You do not need the full formal definition yet. The practical version:

A path is blocked if:

1. It contains a non-collider that you condition on.
2. It contains a collider that you do not condition on.

A path is opened if:

1. It contains no conditioned non-collider.
2. Any collider on the path is conditioned on, or one of its descendants is conditioned on.

Beginner version:

Control for confounders.
Do not control for mediators unless estimating direct effects.
Do not control for colliders.
Be careful with descendants of treatment.

16. Descendants of treatment

A **descendant of treatment** is any variable caused by treatment.

Example:

Coffee $\rightarrow$ Alertness $\rightarrow$ Exam Score

Alertness is a descendant of coffee.

Another:

Coffee → Heart Rate
Coffee → Alertness
Alertness → Exam Score

Heart rate and alertness are descendants of coffee.

For total effect estimation, avoid controlling for descendants of treatment.

Why?

Because they may be part of the mechanism through which treatment affects outcome.

If you control for them, you may block part of the effect.

17. Good controls vs bad controls

Good controls

Variables measured before treatment that affect both treatment and outcome.

Example:

Prior Ability → Tutorial Attendance
Prior Ability → Exam Score

Prior ability is a good control.

Bad controls: mediators

Tutorial Attendance → Understanding → Exam Score

Understanding is a mediator.

If you control for understanding, you remove part of the effect of tutorials.

Bad controls: colliders

Tutorial Attendance → Teacher Attention ← Student Motivation
Student Motivation → Exam Score

Teacher attention is a collider.

Controlling for teacher attention can create bias.

Bad controls: descendants of outcome

Tutorial Attendance → Exam Score → Scholarship

Scholarship happens after the outcome.

Controlling for scholarship is generally wrong if estimating the effect on exam score.

18. Example 1: Coffee and exam score

Question:

Does coffee improve exam scores?

Possible graph:

Study Time → Coffee
Study Time → Exam Score

Sleep → Coffee
Sleep → Exam Score

Prior Ability → Exam Score
Prior Ability → Coffee

Coffee → Alertness
Alertness → Exam Score

Coffee → Exam Score

Visual:

```
graph LR
    StudyTime[Study Time] --> Coffee[Coffee]
    StudyTime --> ExamScore[Exam Score]
    Coffee --> Alertness[Alertness]
    Alertness --> ExamScore
    Sleep[Sleep] --> Coffee
    Sleep --> ExamScore
    PriorAbility[Prior Ability] --> ExamScore
    PriorAbility --> Coffee
    Coffee --> ExamScore
```

Prior Ability $\rightarrow$ Coffee
 $\rightarrow$ Exam Score

For total effect of coffee on exam score:

Good controls:

Study Time
Sleep
Prior Ability

Do not control for:

Alertness

because alertness is a mediator.

19. Example 2: Tutorials and exam performance

Question:

Does attending tutorials improve final exam score?

Possible graph:

Prior Ability $\rightarrow$ Tutorial Attendance
Prior Ability $\rightarrow$ Exam Score

Motivation $\rightarrow$ Tutorial Attendance
Motivation $\rightarrow$ Exam Score

Difficulty Struggling $\rightarrow$ Tutorial Attendance
Difficulty Struggling $\rightarrow$ Exam Score

Tutorial Attendance $\rightarrow$ Understanding $\rightarrow$ Exam Score
Tutorial Attendance $\rightarrow$ Exam Score

Good controls:

Prior Ability
Motivation

Difficulty Struggling

Bad control for total effect:

Understanding

because it is a mediator.

Important nuance:

Difficulty Struggling may create negative selection.

Students who are struggling may attend tutorials more. They may also score lower unless helped.

So the naive comparison may underestimate the tutorial effect.

Naive result:

Tutorial students score lower.

Possible causal reality:

Tutorials help, but struggling students selected into tutorials.

This is common in education and medicine.

20. Example 3: Medicine and recovery

Question:

Does a medicine improve recovery?

Possible graph:

Severity → Medicine

Severity → Recovery

Age → Severity

Age → Recovery

Medicine → Side Effects

Side Effects → Recovery

Medicine → Recovery

Good controls:

Severity

Age

Bad control for total effect:

Side Effects

because side effects are caused by medicine and may mediate part of the effect on recovery.

Naive comparison may say:

Medicine users recover worse.

But that may be because sicker patients were more likely to receive medicine.

This is called **confounding by indication**.

21. Example 4: Job training and income

Question:

Does job training increase income?

Possible graph:

Education → Job Training

Education → Income

Prior Income → Job Training

Prior Income → Future Income

Motivation → Job Training

Motivation → Income

Job Training → Skills → Future Income

Job Training → Future Income

Good controls:

Education

Prior Income

Motivation

Bad control for total effect:

Skills after training

because skills are part of the mechanism.

But motivation is tricky because it may be unobserved or poorly measured. If we do not measure it, our causal estimate may still be biased.

22. Instruments

An **instrumental variable** affects the treatment but affects the outcome only through treatment.

Basic graph:

Instrument → Treatment → Outcome

The instrument should not directly affect the outcome.

Example:

Distance to college → College attendance → Income

Distance to college may affect whether someone attends college.

If distance affects income only through college attendance, it can be used as an instrument.

But this assumption is strong.

If distance to college is also related to local job markets, family background, or urban/rural status, then it may directly affect income or be confounded.

Then it is not a valid instrument.

For now, know the role:

Instrument helps create treatment variation when treatment is not random.

We will study IV later.

23. Effect modifiers

An **effect modifier** changes the size of the treatment effect.

Example:

Coffee effect differs by sleep level.

Sleep may be an effect modifier:

Sleep level	Effect of coffee
Sleep deprived	+8
Well rested	+1
Anxious/overstimulated	-5

Effect modification is not the same as confounding.

A variable can be:

a confounder
an effect modifier
both
neither

In modeling, effect modification often appears as an interaction:

$$\text{ExamScore} = \beta_0 + \beta_1 \text{ Coffee} + \beta_2 \text{ Sleep} + \beta_3 \text{ Coffee} \times \text{Sleep} + \text{error}$$

If β_3 is nonzero, the effect of coffee depends on sleep.

24. Proxy variables

A **proxy** is an imperfect measurement of something you really care about.

Example:

```
True Motivation → Tutorial Attendance  
True Motivation → Exam Score
```

But true motivation is hard to measure.

So you use:

```
number of study app logins  
attendance record  
assignment submission punctuality
```

as proxies.

Problem:

Proxies may not fully remove confounding.

A weak proxy for motivation may reduce bias but not eliminate it.

25. How DAGs connect to DoWhy

DoWhy typically asks you for four things:

```
data  
treatment  
outcome  
causal graph
```

Example conceptually:

```
model = CausalModel(  
    data=df,  
    treatment="coffee",  
    outcome="score",  
    graph=graph  
)
```

The graph tells DoWhy what causal assumptions you are making.

Then DoWhy tries to:

1. Identify the causal estimand.
2. Estimate the effect.
3. Refute/test robustness using sensitivity-style checks.

But DoWhy does not magically know the true graph.

If your DAG is wrong, your causal result can still be wrong.

DoWhy automates workflow, not causal judgment.

26. Causal graph syntax

A causal graph can often be written in DOT format.

Example:

```
digraph {
    study_time -> coffee;
    study_time -> score;
    sleep -> coffee;
    sleep -> score;
    prior_ability -> coffee;
    prior_ability -> score;
    coffee -> alertness;
    alertness -> score;
    coffee -> score;
}
```

This encodes:

```
study_time affects coffee and score
sleep affects coffee and score
prior_ability affects coffee and score
coffee affects alertness
alertness affects score
coffee affects score
```

For total effect of coffee on score, DoWhy should identify that we adjust for:

```
study_time
sleep
prior_ability
```

not:

```
alertness
```

because alertness is a mediator.

27. Manual adjustment formula from DAG

Suppose the valid adjustment set is:

```
X = {study_time, sleep, prior_ability}
```

Then the ATE is identified as:

$$ATE = E_X[E[Y \mid T = 1, X] - E[Y \mid T = 0, X]]$$

In words:

```
For each type of student X:
  compare coffee vs no coffee
Average those comparisons over the population
```

That is what regression/matching/IPW try to estimate in different ways.

28. Mini DoWhy-style example without code

Suppose your dataset has:

coffee	score	study_time	sleep	prior_ability	alertness
1	85	6	5	80	9
0	78	6	5	80	6
1	90	8	6	85	9

coffee	score	study_time	sleep	prior_ability	alertness
0	82	8	6	85	7

Treatment:

coffee

Outcome:

score

Confounders:

study_time
sleep
prior_ability

Mediator:

alertness

Valid graph:

```
study_time -> coffee
study_time -> score
sleep -> coffee
sleep -> score
prior_ability -> coffee
prior_ability -> score
coffee -> alertness
alertness -> score
coffee -> score
```

If estimating total effect:

adjust for study_time, sleep, prior_ability
do not adjust for alertness

If estimating controlled direct effect:

you may control for alertness

But that answers a different question.

29. Common beginner mistakes

Mistake 1: Controlling for everything

Wrong because you may control for mediators or colliders.

Mistake 2: Ignoring time order

A confounder must usually occur before treatment.

If a variable happens after treatment, it is probably not a confounder.

Example:

Coffee → Alertness

Alertness after coffee is not a pre-treatment confounder.

Mistake 3: Thinking regression automatically gives causality

Regression gives adjusted associations.

It becomes causal only under causal assumptions.

Mistake 4: Using data alone to decide controls

You cannot decide controls only by checking correlations.

A variable can be weakly correlated but causally important.

A variable can be strongly correlated but inappropriate to control for.

The graph matters.

Mistake 5: Forgetting unobserved confounding

If an important confounder is not measured, adjustment may fail.

Example:

Motivation → Tutorial Attendance

Motivation → Exam Score

If motivation is missing from the data, your estimate may be biased.

30. Practical control variable checklist

When deciding whether to control for a variable, ask:

1. Did this variable exist before treatment?
2. Does it affect treatment?
3. Does it affect outcome?
4. Is it caused by treatment?
5. Is it a collider?
6. Is it a proxy for an unobserved confounder?
7. Which effect do I want: total effect or direct effect?

Rules of thumb:

Variable type	Control for total effect?
Pre-treatment confounder	Yes
Mediator	No
Collider	No
Instrument	Usually no, unless using IV method
Effect modifier	Maybe, especially for subgroup effects
Proxy confounder	Maybe, but be cautious
Descendant of treatment	Usually no

31. Chapter 3 summary

A DAG represents causal assumptions using arrows.

The central graph task is to identify which paths from treatment to outcome should be open or blocked.

The causal path should remain open:

Treatment → Outcome

Backdoor paths should be blocked:

Treatment ← Confounder → Outcome

Mediators lie on the causal pathway:

Treatment → Mediator → Outcome

Do not control for mediators when estimating total effects.

Colliders have two arrows pointing into them:

A → Collider ← B

Do not control for colliders, because conditioning on them can create fake associations.

The three major path structures are:

Fork: A ← C → B

Chain: A → C → B

Collider: A → C ← B

Adjustment means comparing treated and untreated units within levels of the control variables.

A valid adjustment set blocks all backdoor paths without blocking the causal effect of interest.

DoWhy uses DAGs to identify estimands, but the graph still comes from your causal assumptions.

32. Minimal vocabulary for Chapter 3

Term	Meaning
DAG	Directed acyclic graph representing causal assumptions

Term	Meaning
Node	Variable in the graph
Edge/arrow	Causal relationship
Parent	Direct cause of a variable
Child	Direct effect of a variable
Ancestor	Direct or indirect cause
Descendant	Direct or indirect effect
Confounder	Common cause of treatment and outcome
Backdoor path	Non-causal path from treatment to outcome starting with arrow into treatment
Adjustment set	Variables controlled for to block backdoor paths
Mediator	Variable on causal pathway from treatment to outcome
Collider	Common effect of two variables
D-separation	Graphical criterion for blocked/open paths
Instrument	Variable affecting outcome only through treatment
Effect modifier	Variable that changes treatment effect size
Proxy	Imperfect measurement of another variable

33. Practice questions

Q1

In this graph, what is the confounder?

Motivation → Tutorial Attendance
 Motivation → Exam Score
 Tutorial Attendance → Exam Score

Q2

For the total effect of tutorials on exam score, should you control for understanding?

Tutorial Attendance → Understanding → Exam Score

Q3

What is the collider?

Talent → Admission ← Wealth

Q4

Why is controlling for colliders dangerous?

Q5

What is the backdoor path here?

Sleep → Coffee
Sleep → Exam Score
Coffee → Exam Score

Q6

For the total effect of coffee on exam score, should you control for alertness?

Coffee → Alertness → Exam Score
Coffee → Exam Score

Q7

What is wrong with “just control for every variable”?

34. Answers

A1

The confounder is:

Motivation

because it affects both tutorial attendance and exam score.

A2

No, not if estimating the total effect.

Understanding is a mediator:

Tutorial Attendance → Understanding → Exam Score

Controlling for it blocks part of the tutorial effect.

A3

The collider is:

Admission

because two arrows point into it:

Talent → Admission ← Wealth

A4

Because conditioning on a collider can create a fake association between its causes.

Example:

Among admitted students, talent and wealth may become statistically related even if they were unrelated in the general population.

A5

The backdoor path is:

Coffee ← Sleep → Exam Score

It starts with an arrow into coffee.

A6

No, not for the total effect.

Alertness is a mediator. Controlling for it would block:

Coffee → Alertness → Exam Score

A7

Because some variables are bad controls.

Controlling for confounders can reduce bias.

Controlling for mediators can block the effect.

Controlling for colliders can create bias.

So causal control selection depends on the DAG, not just on having more variables.

5/27/2026, 12:55:35 PM



Chapter 4 — First Implementation with DoWhy

Now we move from theory to implementation.

The goal of this chapter:

Use DoWhy to estimate a causal effect from data using a causal graph.

We will use the same conceptual workflow:

- 1. Define treatment and outcome.
- 2. Draw the causal graph.
- 3. Identify the estimand.
- 4. Estimate the effect.
- 5. Refute/check robustness.

DoWhy's structure mirrors causal inference theory.

1. What DoWhy does

DoWhy is a Python library for causal inference.

Its workflow is built around four steps:

Model → Identify → Estimate → Refute

Meaning:

Step	Meaning
Model	Define treatment, outcome, data, and causal graph
Identify	Find a valid causal estimand from the graph
Estimate	Estimate the causal effect numerically
Refute	Test whether the estimate is fragile or suspicious

The important part:

DoWhy does not magically discover the true causal graph.

You provide the graph. DoWhy then uses that graph to decide how the effect can be identified.

2. Installation

In a Python environment:

```
pip install dowhy
```

Common useful packages:

```
pip install pandas numpy scikit-learn matplotlib
```

Then import:

```
import dowhy
from dowhy import CausalModel
import pandas as pd
import numpy as np
```

3. Example causal question

We will use a synthetic example:

Does drinking coffee improve exam scores?

Variables:

Variable	Meaning
coffee	Treatment, 1 if student drank coffee
score	Outcome, exam score
study_time	Confounder
sleep	Confounder

Variable	Meaning
prior_ability	Confounder
alertness	Mediator

Causal graph:

```
study_time → coffee
study_time → score

sleep → coffee
sleep → score

prior_ability → coffee
prior_ability → score

coffee → alertness
alertness → score

coffee → score
```

For the **total effect** of coffee on score, we should adjust for:

```
study_time
sleep
prior_ability
```

We should not adjust for:

```
alertness
```

because alertness is a mediator.

4. Generate synthetic data

We use synthetic data because we know the true structure.

```
import numpy as np
import pandas as pd
```

```

np.random.seed(42)

n = 1000

study_time = np.random.normal(5, 2, n)
sleep = np.random.normal(7, 1.5, n)
prior_ability = np.random.normal(70, 10, n)

# Probability of drinking coffee depends on confounders
logit_p = (
    -1
    + 0.3 * study_time
    - 0.4 * sleep
    + 0.02 * prior_ability
)

p_coffee = 1 / (1 + np.exp(-logit_p))
coffee = np.random.binomial(1, p_coffee)

# Mediator
alertness = (
    5
    + 2 * coffee
    + 0.3 * sleep
    + np.random.normal(0, 1, n)
)

# Outcome
score = (
    40
    + 2.5 * coffee
    + 4 * study_time
    + 2 * sleep
    + 0.5 * prior_ability
    + 1.5 * alertness
    + np.random.normal(0, 5, n)
)

df = pd.DataFrame({
    "coffee": coffee,

```

```
"score": score,  
"study_time": study_time,  
"sleep": sleep,  
"prior_ability": prior_ability,  
"alertness": alertness  
})  
  
df.head()
```

This creates a dataset where coffee affects score in two ways:

```
coffee → score  
coffee → alertness → score
```

Direct effect:

```
2.5
```

Indirect effect through alertness:

```
coffee → alertness = 2  
alertness → score = 1.5  
indirect effect = 2 × 1.5 = 3
```

So the true total effect is approximately:

```
2.5 + 3 = 5.5
```

Because of noise, estimates will not be exactly 5.5.

5. Naive comparison

Before causal adjustment, compare average scores:

```
df.groupby("coffee")["score"].mean()
```

Then compute:

```
naive_difference = (  
    df[df["coffee"] == 1]["score"].mean()  
)
```

```

- df[df["coffee"] == 0]["score"].mean()
)

print(naive_difference)

```

This gives the raw association:

```

average score among coffee drinkers
-
average score among non-coffee drinkers

```

But this is not necessarily causal because coffee drinkers may differ in study time, sleep, and prior ability.

6. Build the causal graph

DoWhy accepts a graph in DOT format.

```

graph = """
digraph {
    study_time -> coffee;
    study_time -> score;

    sleep -> coffee;
    sleep -> score;

    prior_ability -> coffee;
    prior_ability -> score;

    coffee -> alertness;
    alertness -> score;

    coffee -> score;
}
"""

```

This graph says:

```

study_time, sleep, prior_ability are confounders
alertness is a mediator

```

```
coffee is the treatment  
score is the outcome
```

For total effect, DoWhy should find a backdoor adjustment set that blocks confounding paths without controlling for the mediator.

7. Create the DoWhy causal model

```
from dowhy import CausalModel  
  
model = CausalModel(  
    data=df,  
    treatment="coffee",  
    outcome="score",  
    graph=graph  
)
```

Optional visualization:

```
model.view_model()
```

Depending on your environment, this may generate a graph image file.

In notebooks, graph visualization sometimes needs Graphviz installed.

On Windows, you may need:

```
pip install graphviz pygraphviz
```

and possibly the Graphviz system installer.

If graph visualization fails, the causal model can still work. Visualization is useful but not mandatory.

8. Identify the causal estimand

Now ask DoWhy:

```
identified_estimand = model.identify_effect()
```

```
print(identified_estimand)
```

This step answers:

Given the graph, can the causal effect be expressed using observed variables?

For this graph, DoWhy should identify a backdoor estimand.

Conceptually:

$$ATE = E_X[E[\text{score} \mid \text{coffee} = 1, X] - E[\text{score} \mid \text{coffee} = 0, X]]$$

where:

$X = \text{study_time}, \text{sleep}, \text{prior_ability}$

The key idea:

adjust for confounders
do not adjust for mediator

9. Estimate the effect using linear regression

Now estimate numerically:

```
estimate = model.estimate_effect(  
    identified_estimand,  
    method_name="backdoor.linear_regression"  
)  
  
print(estimate)  
print("Estimated causal effect:", estimate.value)
```

This estimates the ATE using linear regression adjustment.

The regression is approximately:

$$\text{score} \sim \text{coffee} + \text{study_time} + \text{sleep} + \text{prior_ability}$$

DoWhy chooses the adjustment set from the graph.

Expected result:

Estimated effect should be near 5.5

It will not be exact because the data contains random noise.

10. Why the effect is near 5.5

Recall the data-generating process:

```
coffee → score = 2.5
coffee → alertness = 2
alertness → score = 1.5
```

So:

```
direct effect = 2.5
indirect effect = 2 × 1.5 = 3
total effect = 5.5
```

Because we did **not** control for alertness, the estimated effect includes both:

```
coffee → score
coffee → alertness → score
```

That is the total effect.

11. What if we wrongly control for alertness?

If you include alertness as a confounder-like control, you block the mediator path.

Then you estimate something closer to the direct effect:

```
coffee → score
```

not the total effect.

To demonstrate, create a wrong graph where alertness is treated as if it should be adjusted for. One simple way is to estimate manually with regression:

```
import statsmodels.api as sm

X_wrong = df[["coffee", "study_time", "sleep", "prior_ability", "alertness"]]
X_wrong = sm.add_constant(X_wrong)

wrong_model = sm.OLS(df["score"], X_wrong).fit()

print(wrong_model.summary())
print("Coffee coefficient when controlling for alertness:",
      wrong_model.params["coffee"])
```

Expected result:

```
coffee coefficient closer to 2.5 than 5.5
```

Why?

Because controlling for alertness blocks:

```
coffee → alertness → score
```

So you removed the indirect effect.

This is the practical version of Chapter 3.

12. Estimate using matching

DoWhy can also estimate effects using matching.

```
estimate_matching = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_matching"
)

print(estimate_matching)
print("Matching estimate:", estimate_matching.value)
```

This tries to compare coffee drinkers with non-coffee drinkers who have similar confounder profiles.

Instead of fitting:

```
score ~ coffee + confounders
```

matching tries to create comparable treated and untreated groups.

13. Estimate using propensity score weighting

Another common method is inverse probability weighting.

```
estimate_ipw = model.estimate_effect(  
    identified_estimand,  
    method_name="backdoor.propensity_score_weighting"  
)  
  
print(estimate_ipw)  
print("IPW estimate:", estimate_ipw.value)
```

This estimates each student's probability of receiving treatment:

```
P(coffee = 1 | study_time, sleep, prior_ability)
```

Then it weights observations to create a pseudo-population where treatment is less confounded.

Intuition:

Student type	Problem	IPW idea
Very likely to drink coffee and did drink coffee	Overrepresented among treated	Give lower weight
Unlikely to drink coffee but did drink coffee	Rare treated case	Give higher weight
Very likely to avoid coffee and avoided coffee	Overrepresented among untreated	Give lower weight
Likely to drink coffee but avoided coffee	Rare untreated case	Give higher weight

IPW relies heavily on good overlap. If some groups almost always receive treatment, weights can explode.

14. Refutation tests

DoWhy includes refuters.

These are not proof that your estimate is correct.

They are stress tests.

Refuter 1: Random common cause

Add a random fake confounder and see whether the estimate changes drastically.

```
refute_random = model.refute_estimate(  
    identified_estimand,  
    estimate,  
    method_name="random_common_cause"  
)  
  
print(refute_random)
```

Expected behavior:

Adding irrelevant random noise should not strongly change the estimate.

If it changes a lot, your estimate may be unstable.

Refuter 2: Placebo treatment

Replace treatment with a fake random treatment.

```
refute_placebo = model.refute_estimate(  
    identified_estimand,  
    estimate,  
    method_name="placebo_treatment_refuter"  
)  
  
print(refute_placebo)
```

Expected behavior:

Fake treatment should have effect near 0.

If a fake treatment shows a strong effect, your design may be detecting patterns that are not truly causal.

Refuter 3: Data subset refuter

Estimate effect on random subsets of the data.

```
refute_subset = model.refute_estimate(
    identified_estimand,
    estimate,
    method_name="data_subset_refuter"
)

print(refute_subset)
```

Expected behavior:

Effect should remain reasonably similar across subsets.

If the estimate changes wildly, it may be unstable or dependent on a small part of the data.

15. Complete script

Here is the full runnable version.

```
import numpy as np
import pandas as pd
from dowhy import CausalModel

np.random.seed(42)

n = 1000

study_time = np.random.normal(5, 2, n)
sleep = np.random.normal(7, 1.5, n)
prior_ability = np.random.normal(70, 10, n)

logit_p = (
```

```

-1
+ 0.3 * study_time
- 0.4 * sleep
+ 0.02 * prior_ability
)

p_coffee = 1 / (1 + np.exp(-logit_p))
coffee = np.random.binomial(1, p_coffee)

alertness = (
    5
    + 2 * coffee
    + 0.3 * sleep
    + np.random.normal(0, 1, n)
)

score = (
    40
    + 2.5 * coffee
    + 4 * study_time
    + 2 * sleep
    + 0.5 * prior_ability
    + 1.5 * alertness
    + np.random.normal(0, 5, n)
)

df = pd.DataFrame({
    "coffee": coffee,
    "score": score,
    "study_time": study_time,
    "sleep": sleep,
    "prior_ability": prior_ability,
    "alertness": alertness
})

print("First rows:")
print(df.head())

naive_difference = (
    df[df["coffee"] == 1]["score"].mean()
    - df[df["coffee"] == 0]["score"].mean()
)

```

```

)

print("\nNaive difference:")
print(naive_difference)

graph = """
digraph {
    study_time -> coffee;
    study_time -> score;

    sleep -> coffee;
    sleep -> score;

    prior_ability -> coffee;
    prior_ability -> score;

    coffee -> alertness;
    alertness -> score;

    coffee -> score;
}
"""

model = CausalModel(
    data=df,
    treatment="coffee",
    outcome="score",
    graph=graph
)

identified_estimand = model.identify_effect()

print("\nIdentified estimand:")
print(identified_estimand)

estimate_lr = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.linear_regression"
)

print("\nLinear regression estimate:")

```

```

print(estimate_lr)
print("Estimated causal effect:", estimate_lr.value)

estimate_matching = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_matching"
)

print("\nPropensity score matching estimate:")
print(estimate_matching)
print("Estimated causal effect:", estimate_matching.value)

estimate_ipw = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_weighting"
)

print("\nPropensity score weighting estimate:")
print(estimate_ipw)
print("Estimated causal effect:", estimate_ipw.value)

refute_random = model.refute_estimate(
    identified_estimand,
    estimate_lr,
    method_name="random_common_cause"
)

print("\nRandom common cause refuter:")
print(refute_random)

refute_placebo = model.refute_estimate(
    identified_estimand,
    estimate_lr,
    method_name="placebo_treatment_refuter"
)

print("\nPlacebo treatment refuter:")
print(refute_placebo)

refute_subset = model.refute_estimate(
    identified_estimand,

```

```
estimate_lr,  
method_name="data_subset_refuter"  
)  
  
print("\nData subset refuter:")  
print(refute_subset)
```

16. What your output should look like conceptually

You should expect:

```
Naive difference: some biased association  
Linear regression estimate: near 5.5  
Matching estimate: roughly near 5.5  
IPW estimate: roughly near 5.5  
Placebo refuter: effect near 0
```

Exact numbers may differ.

The important interpretation:

```
Naive difference = association  
DoWhy estimate = causal estimate under the graph assumptions  
Refuters = robustness checks, not proof
```

17. What happens if the graph is wrong?

Suppose the real world has:

```
motivation → coffee  
motivation → score
```

but your data does not include motivation, and your graph ignores it.

Then your estimate may be biased.

Even if DoWhy prints a clean identified estimand.

Why?

Because DoWhy only knows the graph you gave it.

Wrong graph:

```
study_time → coffee
study_time → score
coffee → score
```

True graph:

```
motivation → coffee
motivation → score
study_time → coffee
study_time → score
coffee → score
```

If motivation is omitted, there is an open backdoor path:

```
coffee ← motivation → score
```

DoWhy cannot adjust for motivation if it is missing.

So DoWhy can tell you:

Given your graph, the effect is identifiable.

It cannot guarantee:

Your graph is true.

18. How to think while using DoWhy

Do not start with code.

Start with the causal question.

Use this checklist:

1. What is the treatment?
2. What is the outcome?
3. What population am I studying?
4. What is the time order?
5. What variables affect treatment?

6. What variables affect outcome?
7. Which variables are confounders?
8. Which variables are mediators?
9. Which variables are colliders?
10. Which effect do I want: total effect or direct effect?

Only after this should you write the graph.

19. Common DoWhy beginner mistakes

Mistake 1: Treating DoWhy as automatic causality

DoWhy does not discover causality by itself.

It formalizes your causal assumptions.

Mistake 2: Including mediators as confounders

Example:

```
coffee → alertness → score
```

If estimating total effect, do not adjust for alertness.

Mistake 3: Forgetting unobserved confounders

If a major confounder is missing, the estimate may be invalid.

Mistake 4: Thinking refuters prove causality

Refuters only check certain failure modes.

Passing refuters does not prove the DAG is correct.

Mistake 5: Ignoring overlap

If some students almost always drink coffee and others almost never do, comparison becomes weak.

Check propensity scores later.

20. Manual regression equivalent

For this simple linear example, DoWhy's linear regression adjustment is conceptually similar to:

```
import statsmodels.api as sm

X = df[["coffee", "study_time", "sleep", "prior_ability"]]
X = sm.add_constant(X)

ols_model = sm.OLS(df["score"], X).fit()

print(ols_model.summary())
print("Estimated coffee effect:", ols_model.params["coffee"])
```

This estimates:

$$\text{score} = \beta_0 + \beta_1 \text{ coffee} + \beta_2 \text{ study_time} + \beta_3 \text{ sleep} + \beta_4 \text{ prior_ability} + \text{error}$$

Under the DAG assumptions, β_1 is interpreted as the causal effect of coffee.

Without the DAG assumptions, β_1 is just an adjusted association.

21. Important distinction: DoWhy vs statsmodels

Tool	Main role
statsmodels	Fits statistical models
DoWhy	Structures causal workflow
scikit-learn	Predictive modeling
econml	Advanced heterogeneous treatment effect estimation
causalml	Uplift modeling and treatment effect estimation

DoWhy is useful because it forces you to separate:

causal model
identification

estimation
refutation

That structure is more important than the exact estimator.

22. Chapter 4 summary

DoWhy follows this workflow:

Model → Identify → Estimate → Refute

You provide:

data
treatment
outcome
causal graph

DoWhy identifies the estimand using the graph.

For backdoor adjustment, the estimand is usually:

$$ATE = E_X[E[Y \mid T = 1, X] - E[Y \mid T = 0, X]]$$

where X is a valid adjustment set.

Different estimators include:

linear regression
propensity score matching
propensity score weighting

Refuters include:

random common cause
placebo treatment
data subset

But:

DoWhy does not prove your graph is true.
Refuters do not prove causality.
The assumptions remain the core issue.

23. Minimal vocabulary for Chapter 4

Term	Meaning
CausalModel	DoWhy object containing data, treatment, outcome, and graph
Identified estimand	Causal quantity expressed using observed data
Estimate	Numerical causal effect estimate
Backdoor linear regression	Regression adjustment using graph-selected controls
Propensity score	Probability of receiving treatment given covariates
Matching	Comparing treated and untreated units with similar covariates
IPW	Reweighting data by inverse treatment probabilities
Refuter	Robustness check for a causal estimate
Placebo treatment	Fake treatment used to test whether effect disappears
Random common cause	Fake confounder added to test estimate stability

24. Practice questions

Q1

In DoWhy, what are the four main workflow steps?

Q2

Why should you not include `alertness` as a control when estimating the total effect of coffee on score?

coffee → alertness → score

Q3

What does `model.identify_effect()` do?

Q4

What does `model.estimate_effect()` do?

Q5

If the true graph has an unobserved confounder missing from your data, can DoWhy automatically fix that?

Q6

What is the difference between the naive difference and the DoWhy causal estimate?

Q7

What does a placebo treatment refuter check?

25. Answers

A1

Model → Identify → Estimate → Refute

A2

Because alertness is a mediator.

For the total effect, we want to include:

coffee → alertness → score

If we control for alertness, we block that pathway and estimate something closer to the direct effect.

A3

It uses the causal graph to determine whether the causal effect can be identified from observed data, and what adjustment strategy is valid.

A4

It numerically estimates the identified causal effect using a chosen method, such as linear regression, matching, or propensity score weighting.

A5

No.

If an important confounder is missing, DoWhy cannot automatically recover it. The estimate may be biased even if the code runs correctly.

A6

The naive difference is:

$$E[Y \mid T = 1] - E[Y \mid T = 0]$$

It is just an observed association.

The DoWhy causal estimate tries to estimate:

$$E[Y(1) - Y(0)]$$

using the causal graph and adjustment assumptions.

A7

It replaces the real treatment with a fake/random treatment.

If the fake treatment still produces a large effect, the original causal estimate may be suspicious or unstable.

5/28/2026, 11:16:42 AM



Chapter 5 - Before Starting the CRP Causal Inference Project

You are not done yet. You know enough to understand the idea, but before starting the actual internship-style project, you need a few practical concepts:

1. How to define a causal question properly
2. How to translate business variables into treatments/outcomes/confounders
3. How to handle binary outcomes like failure/success
4. How to distinguish prediction features from intervention variables
5. How to validate causal estimates responsibly
6. How to avoid overclaiming causality

For your uploaded CRP proposal, this chapter matters because the project is not merely “use DoWhy.” The actual goal is to build a causal inference engine that explains why a change is risky, estimates intervention impact, and supports what-if simulation for Change Risk Prediction. The proposal explicitly frames the engine as complementary to the existing ML risk model and LLM recommendations, adding causal explanation, quantified intervention impact, and what-if modelling. `Causal_Inference_Internship_Proposal.docx`

Rough estimate: after this, there are about **4-5 essential chapters left** before you are reasonably ready to start implementing a serious PoC.

1. The practical causal workflow

For a real project, the workflow is:

1. Define the decision problem.
2. Define the treatment.
3. Define the outcome.
4. Define the unit of analysis.
5. Define time ordering.
6. Identify confounders.
7. Build a DAG.
8. Choose an estimand.
9. Estimate effect.
10. Validate/refute.
11. Report assumptions and limitations.

In the CRP project, the unit is probably:

one Change Request

The outcome is probably:

change failure

The treatments/interventions could be things like:

test coverage increase
rollback plan present
deployment window changed
approval count changed
CAB involvement
assignment-group readiness

The proposal's required data fields match this framing: change metadata, risk/testing information, approval workflow, deployment context, outcome, and historical assignment-group metrics. `Causal_Inference_Internship_Proposal.docx`

2. Unit of analysis

The **unit of analysis** is the entity for which you observe treatment and outcome.

For this project:

Unit = individual Change Request

Example row:

change_id	test_coverage	rollback_plan	deployment_window	failure
CHG001	45	0	weekend	1
CHG002	80	1	weekday	0

Each row is one change request.

This matters because causal assumptions are made at the row level.

You are asking:

For this type of change request, what would have happened if test coverage had been higher?

Not:

Does a team with good engineering culture have fewer failures?

Those are different causal questions.

3. Outcome definition

The outcome must be precise.

For CRP, likely outcome:

```
Y = 1 if change failed
Y = 0 if change succeeded
```

Possible definitions of failure:

```
rollback occurred
incident linked after deployment
change marked unsuccessful
SLA-impacting disruption occurred
manual failure label
```

You need one clear operational definition.

Bad:

```
failure = risky change
```

Because that mixes prediction with outcome.

Better:

```
failure = incident linked within 24/48/72 hours after change deployment
```

or:

```
failure = rollback OR incident linkage OR unsuccessful closure code
```

The proposal lists “change result (success/failure)” and “incident linkage” as required outcome fields, which suggests the outcome should be explicitly tied to those records.

Causal_Inference_Internship_Proposal.docx

4. Treatment definition

A **treatment** is the thing whose effect you want to estimate.

For CRP, examples:

```
T = rollback plan present  
T = deployment during high-risk window  
T = test coverage above threshold  
T = CAB involvement  
T = approval count above threshold
```

The treatment must be actionable.

Good treatment:

```
rollback_plan = 1 vs 0
```

because a team can add a rollback plan.

Good treatment:

```
test_coverage >= 80% vs test_coverage < 80%
```

because test coverage can be changed.

Risky treatment:

```
assignment_group_failure_rate
```

because a team cannot instantly intervene on its historical failure rate. It is more likely a confounder or risk marker.

Bad treatment:

```
ML risk score = High
```

because this is a prediction, not a real-world intervention.

You usually do not want:

```
do(ML risk score = Low)
```

You want:

```
do(test coverage = 80%)  
do(rollback plan = present)  
do(deployment window = low-risk)
```

This distinction is central to your proposal because it explicitly separates predictive indicators from controllable interventions. `Causal_Inference_Internship_Proposal.docx`

5. Binary vs continuous treatment

Treatments can be binary, categorical, or continuous.

Binary treatment

Example:

```
rollback_plan = 1  
rollback_plan = 0
```

Causal question:

Does having a rollback plan reduce failure probability?

Estimand:

$$ATE = E[Y(1) - Y(0)]$$

Continuous treatment

Example:

```
test_coverage = 45%, 60%, 80%, 90%
```

Causal question:

What happens if test coverage increases from 45% to 80%?

This is not just binary ATE.

You may need a dose-response style question:

```
E[Y(test_coverage = 80)] - E[Y(test_coverage = 45)]
```

For a first PoC, it is often easier to binarize:

```
high_test_coverage = 1 if test_coverage >= 80  
high_test_coverage = 0 otherwise
```

Then estimate:

```
effect of high test coverage on failure probability
```

This is simpler and more interpretable.

6. Outcome is binary, so effect means probability change

Since failure is probably binary:

```
Y = 1 means failure  
Y = 0 means success
```

The causal effect is a change in probability.

Example:

```
E[Y | do(test_coverage_high = 1)] = 0.08  
E[Y | do(test_coverage_high = 0)] = 0.14
```

Then:

$$\text{ATE} = 0.08 - 0.14 = -0.06$$

Interpretation:

High test coverage reduces failure probability by 6 percentage points.

Important: this is **percentage points**, not percent.

Difference:

14% → 8%
absolute reduction = 6 percentage points
relative reduction = 6 / 14 = 42.9%

For business reporting, be explicit.

Bad:

Risk reduced by 6%.

Better:

Failure probability reduced by 6 percentage points, from 14% to 8%.

7. Controllable variable vs predictive variable

This is one of the most important project-specific distinctions.

A predictive feature helps forecast failure.

A causal intervention variable is something you can change to reduce failure.

Example:

Variable	Predictive?	Causal/actionable?	Likely role
Past assignment-group failure rate	Yes	Not directly	Confounder/risk marker
Test coverage	Yes	Yes	Treatment/intervention
Rollback plan flag	Yes	Yes	Treatment/intervention

Variable	Predictive?	Causal/actionable?	Likely role
Change priority	Yes	Usually not	Confounder/context
Deployment window	Yes	Yes, sometimes	Treatment/intervention
ML risk score	Yes	No	Prediction output, not treatment
Incident linkage	Outcome	No	Outcome

The causal engine should prioritize variables that are:

```
controllable
pre-treatment
measured reliably
not merely labels or predictions
```

This aligns with the proposal's goal of producing "controllable intervention candidates" with quantified effect estimates and statistical confidence.

`Causal_Inference_Internship_Proposal.docx`

8. Time ordering

Causal inference depends heavily on time.

For each variable, ask:

```
Was this known before the change was deployed?
Was it caused by the change?
Was it recorded after the failure?
```

Example timeline:

```
Change created
↓
Planning metadata recorded
↓
Approvals happen
↓
Testing completed
↓
```

```
Deployment scheduled
↓
Deployment executed
↓
Incident/rollback/failure observed
```

Good pre-treatment variables:

```
change type
priority
category
assignment group
planned deployment window
test coverage before deployment
rollback plan before deployment
approver count before approval completion
```

Bad controls:

```
post-deployment incident count
actual rollback after failure
post-change severity
LLM recommendation after risk scoring
operator response after alert
```

Why?

Because variables after treatment/outcome may be mediators, consequences, or colliders.

For example:

```
High-risk change → extra approval → failure
```

If approval is assigned because the change was already risky, approval count may be a confounded variable.

But:

```
approval count → better review → lower failure
```

could also be causal.

So time ordering and domain logic matter.

9. Example CRP DAG

A first rough DAG might look like this:

```
change_complexity → test_coverage
change_complexity → rollback_plan
change_complexity → approval_count
change_complexity → failure

priority → approval_count
priority → deployment_window
priority → failure

assignment_group_history → test_coverage
assignment_group_history → rollback_plan
assignment_group_history → failure

test_coverage → failure
rollback_plan → failure
deployment_window → failure
approval_count → failure
CAB_involvement → failure
```

DOT format:

```
digraph {
    change_complexity -> test_coverage;
    change_complexity -> rollback_plan;
    change_complexity -> approval_count;
    change_complexity -> failure;

    priority -> approval_count;
    priority -> deployment_window;
    priority -> failure;

    assignment_group_history -> test_coverage;
    assignment_group_history -> rollback_plan;
    assignment_group_history -> failure;
```

```
test_coverage -> failure;
rollback_plan -> failure;
deployment_window -> failure;
approval_count -> failure;
CAB_involvement -> failure;
}
```

This is not automatically correct. It is a starting hypothesis.

The proposal specifically says the DAG should be domain-driven, reviewed by experts, and refined based on statistical dependencies and feedback.

Causal_Inference_Internship_Proposal.docx

10. Choosing an estimand

Before coding, decide what effect you want.

Example 1: rollback plan

Question:

Does having a rollback plan reduce failure probability?

Treatment:

$T = \text{rollback_plan}$

Outcome:

$Y = \text{failure}$

Estimand:

$ATE = E[Y(1) - Y(0)]$

Interpretation:

Average change in failure probability if all changes had rollback plans versus if none had rollback plans.

Example 2: high test coverage

Question:

Does high test coverage reduce failure probability?

Treatment:

```
T = 1 if test_coverage >= 80%  
T = 0 otherwise
```

Outcome:

```
Y = failure
```

Estimand:

```
ATE = E[Y(1) - Y(0)]
```

Interpretation:

Average effect of high test coverage on failure probability.

Example 3: deployment window

Question:

What is the effect of deploying during off-hours?

Treatment:

```
T = off_hours_deployment
```

Outcome:

```
Y = failure
```

This may be heavily confounded because high-risk changes may be deliberately scheduled during special windows.

You need to control for:

```
priority
change complexity
service criticality
assignment group
release type
CAB involvement
```

11. The major CRP confounding problem

In CRP, many “good process controls” are assigned to risky changes.

Example:

```
change_complexity → rollback_plan
change_complexity → failure
```

Complex changes are more likely to require rollback plans.

Complex changes are also more likely to fail.

Naive comparison may show:

```
rollback plan associated with higher failure
```

But causal reality may be:

```
rollback plans reduce failure or reduce incident severity,
but are used more often for high-risk changes
```

Same issue for:

```
CAB involvement
approver count
extra testing
deployment freezes
manual review
```

This is called **confounding by indication**.

It appears in medicine too:

sicker patients receive stronger medicine
sicker patients have worse outcomes

Naively, medicine looks harmful.

In CRP:

riskier changes receive stricter controls
riskier changes fail more often

Naively, stricter controls may look harmful.

This is probably one of the central technical challenges of the project.

12. What should be adjusted for?

For each treatment, define its adjustment set separately.

There is no single universal adjustment set for every treatment.

Treatment: rollback plan

Possible confounders:

change complexity
priority
change type
assignment group
historical group failure rate
deployment environment
service criticality
change frequency

Do not control for:

actual rollback after deployment
incident after deployment
post-change actions

Treatment: test coverage

Possible confounders:

change complexity
team maturity
assignment group history
change type
codebase size
release urgency
priority

Do not control for:

bugs found after testing
post-deployment incident
ML risk score if computed using test coverage and other descendants

Treatment: approval count

Possible confounders:

priority
complexity
CAB requirement rules
change type
environment
assignment group

Potential issue:

Approval count may be both a treatment and a proxy for complexity. So causal claims need caution.

13. Positivity in CRP

Positivity means each type of change must have both treatment possibilities.

Example problem:

All emergency changes have low test coverage.
No emergency changes have high test coverage.

Then you cannot estimate:

effect of high test coverage among emergency changes

because there are no comparable emergency changes with high test coverage.

Another example:

All production database changes require CAB approval.

Then you cannot estimate the effect of CAB approval for production database changes using ordinary adjustment.

There is no untreated comparison group.

In data terms, check:

$$P(T = 1 \mid X)$$

If some values are near 0 or 1, overlap is weak.

This is especially important for propensity score methods.

14. SUTVA in CRP

SUTVA can fail in operational systems.

Recall SUTVA means:

1. No interference between units.
2. No hidden versions of treatment.

Interference problem

One change request can affect another.

Example:

Change A causes instability.
Change B is deployed shortly after.
Change B fails partly because of Change A.

Then outcomes are not independent.

Possible mitigation:

```
include recent deployment load
include same-service recent changes
include change collision indicators
limit analysis to independent windows
cluster by service/team
```

Hidden treatment versions

Example:

```
rollback_plan = 1
```

But rollback plans differ in quality.

One may be a real tested rollback strategy.

Another may be a checkbox with vague text.

So the treatment is not well-defined.

Better features:

```
rollback_plan_present
rollback_plan_tested
rollback_plan_approved_before_deployment
rollback_plan_has_owner
rollback_plan_quality_score
```

Causal inference improves when treatment definitions are operationally precise.

15. Identification vs estimation in the project

For the CRP engine:

Identification asks:

Under this DAG, can we estimate the effect of test coverage on failure?

Example:

```
test_coverage ← change_complexity → failure
```

If `change_complexity` is measured, you may adjust for it.

If not, effect may not be identifiable by backdoor adjustment.

Estimation asks:

What method do we use to compute the effect?

Possible methods:

```
logistic regression
linear probability model
propensity score matching
propensity score weighting
doubly robust estimation
causal forests
```

Do not jump to complex estimators first.

Start with:

1. simple interpretable regression
2. propensity score diagnostics
3. bootstrap confidence intervals
4. DoWhy refuters

Then move to advanced methods.

16. Binary outcome models

Because failure is binary, you have two common choices.

Option 1: Linear Probability Model

Use linear regression even though outcome is 0/1.

```
failure =  $\beta_0$  +  $\beta_1$  treatment +  $\beta_2$  confounders + error
```

Advantage:

```
 $\beta_1$  is easy to interpret as percentage-point change
```

Example:

$$\beta_1 = -0.06$$

means:

treatment reduces failure probability by 6 percentage points

Disadvantage:

can predict probabilities below 0 or above 1

But for a first causal PoC, it is often acceptable and interpretable.

Option 2: Logistic Regression

Model:

$$\text{logit}(P(\text{failure} = 1)) = \beta_0 + \beta_1 \text{ treatment} + \beta_2 \text{ confounders}$$

Advantage:

predicted probabilities stay between 0 and 1

Disadvantage:

coefficients are log-odds, less intuitive

For business reporting, convert to predicted probabilities:

$$P(\text{failure} \mid \text{do}(T = 1)) - P(\text{failure} \mid \text{do}(T = 0))$$

Do not report only odds ratios unless the audience understands them.

17. Confidence intervals

A causal estimate without uncertainty is weak.

Instead of:

test coverage reduces risk by 22%

report:

high test coverage is estimated to reduce failure probability by 6.2 percentage points

95% CI: 3.1 to 9.4 percentage points

The proposal explicitly expects quantified recommendations with confidence intervals, such as "95% CI: 18-26%." [Causal_Inference_Internship_Proposal.docx](#)

For implementation, you can use:

bootstrap confidence intervals
robust standard errors
cross-validation / holdout checks

For a PoC, bootstrap is intuitive:

1. resample rows with replacement
2. re-estimate causal effect
3. repeat 500-1000 times
4. take 2.5th and 97.5th percentiles

18. Refutation and robustness

DoWhy refuters are useful, but they are not proof.

The proposal lists placebo treatment, random common cause, data subset tests, sensitivity analysis, and holdout validation as required validation mechanisms.

[Causal_Inference_Internship_Proposal.docx](#)

You should understand what each test does.

Placebo treatment

Replace real treatment with fake random treatment.

Expected:

effect ≈ 0

If not, your pipeline may be detecting spurious structure.

Random common cause

Add a random fake confounder.

Expected:

estimate should not change much

If it changes drastically, the estimate is unstable.

Data subset refuter

Estimate effect on random subsets.

Expected:

effect should remain similar

If estimates vary wildly, sample size, overlap, or model dependence may be poor.

Sensitivity analysis

Ask:

How strong would an unmeasured confounder need to be to erase this effect?

This is especially important because CRP likely has unobserved confounders such as:

team maturity
actual code complexity
service architecture quality
developer experience
release pressure
quality of testing

19. Missing data

Missingness is not just a data cleaning issue. It can bias causal estimates.

Example:

```
test_coverage missing
```

Why is it missing?

Possibilities:

```
older systems did not record it
low-maturity teams skipped reporting
emergency changes bypassed test documentation
specific assignment groups use different tools
```

If missingness is related to failure risk, simple median imputation can be biased.

The proposal has a missing-data strategy: median imputation under 10%, model-based imputation for 10-30%, and exclusion above 30%, with graceful degradation when key causal variables are unavailable. `Causal_Inference_Internship_Proposal.docx`

That is a reasonable starting policy, but for causal inference you should also report:

```
missingness rate by treatment
missingness rate by outcome
missingness rate by assignment group
missingness rate by time period
```

Example:

```
test_coverage missing for 8% of successful changes
test_coverage missing for 35% of failed changes
```

That is a serious warning.

20. Baseline comparison

The proposal wants comparison against LLM-only recommendations and tracking whether changes following causal guidance had lower failure rates.

`Causal_Inference_Internship_Proposal.docx`

Be careful: this creates a new causal question.

Question:

Do causal recommendations reduce future failure rates compared to LLM-only recommendations?

Treatment:

T = received causal recommendation

Outcome:

future failure

But this cannot be cleanly answered from historical data unless there was already variation in recommendation type.

For a PoC, you can do:

offline validation
historical counterfactual simulation
expert review
stability/refutation tests

For true adoption impact, you need:

A/B test
staggered rollout
difference-in-differences
pre/post analysis with caution

That is a later-stage evaluation problem.

21. What not to claim

Do not claim:

We proved test coverage causes a 22% risk reduction.

Better:

Under the stated DAG assumptions and observed historical data, increasing test coverage from low to high is estimated to reduce failure probability by X

percentage points.

Do not claim:

Dowhy discovered the root cause.

Better:

Dowhy estimated the causal effect implied by a domain-specified DAG.

Do not claim:

The refutation tests prove the estimate is correct.

Better:

The estimate was stable under placebo, random common cause, and subset refutation checks, but remains dependent on the stated assumptions.

This phrasing matters in an internship project because stakeholders may treat numerical causal outputs as authoritative.

22. Minimum project-ready checklist

Before implementing the PoC, you should be able to answer these:

1. What is the unit of analysis?
2. What exactly is the outcome?
3. What is one treatment/intervention?
4. Is the treatment actionable?
5. Is the treatment measured before the outcome?
6. What are the likely confounders?
7. Which variables are mediators?
8. Which variables are colliders or post-treatment variables?
9. What is the DAG?
10. What is the estimand?
11. Is there overlap/positivity?
12. What estimator will you start with?
13. How will you compute confidence intervals?

14. What refuters will you run?
15. What limitations will you report?

For this CRP project, a good first PoC would not try to estimate everything.

Start with one treatment:

```
rollback_plan_present → failure
```

or:

```
high_test_coverage → failure
```

Then build a complete mini-pipeline around that.

23. Recommended first PoC design

Use one treatment first.

Treatment

```
high_test_coverage = 1 if test_coverage >= threshold
```

Outcome

```
failure = 1 if change failed or incident linked
```

Candidate confounders

```
change_type  
priority  
category  
assignment_group  
assignment_group_failure_rate_90d  
deployment_environment  
change_complexity  
deployment_window  
change_frequency
```

Avoid controlling for

```
post-deployment incident details
actual rollback after deployment
operator response after failure
LLM recommendation generated after risk score
ML risk score if it is downstream of treatment variables
```

Estimate

Start with:

```
linear probability model
logistic regression with marginal effect
propensity score weighting
```

Validate

Run:

```
placebo refuter
random common cause refuter
data subset refuter
bootstrap CI
overlap check
missingness report
```

Output

Example final output format:

```
Treatment: high test coverage  $\geq$  80%
Outcome: change failure within 48h

Estimated effect:
High test coverage is associated with an estimated 4.8 percentage-point
reduction in failure probability.

95% CI:
2.1 to 7.5 percentage points
```

Adjustment set:

change type, priority, assignment group history, deployment environment, change complexity

Assumptions:

No unmeasured confounding after adjustment.

Sufficient overlap across treatment groups.

Treatment measured before deployment.

Failure labels are reliable.

Limitations:

Team maturity and true code complexity may be imperfectly measured.

The estimate should be interpreted as causal only under the stated DAG assumptions.

That is the level of discipline you want.

24. Chapter 5 summary

Before starting the CRP causal inference engine, you need to be able to translate the business problem into a causal design.

The project is not:

run DoWhy on a dataset

It is:

define interventions

construct a defensible DAG

identify valid adjustment sets

estimate effects with uncertainty

stress-test assumptions

report limitations clearly

For CRP:

Unit = Change Request

Outcome = failure/success

Treatments = controllable operational decisions

Confounders = pre-treatment factors affecting both controls and failure

Bad controls = post-treatment variables, mediators, colliders, prediction outputs

The biggest project-specific risk is confounding by indication:

riskier changes receive stronger controls
riskier changes fail more often

So naive comparisons may make useful controls look harmful.

25. Minimal vocabulary for Chapter 5

Term	Meaning
Unit of analysis	Entity represented by each row
Binary outcome	Outcome taking values 0/1
Linear probability model	Linear regression with binary outcome
Logistic regression	Model for binary outcome probabilities/log-odds
Actionable treatment	Variable that can realistically be intervened on
Predictive feature	Variable useful for forecasting, not necessarily causal
Confounding by indication	High-risk units are more likely to receive treatment
Positivity violation	Some unit types never/always receive treatment
Bootstrap CI	Confidence interval from repeated resampling
Sensitivity analysis	Checks how robust estimate is to hidden confounding
Graceful degradation	System falls back when causal analysis is not valid/available

26. Practice questions

Q1

In the CRP project, what is the likely unit of analysis?

Q2

Why is `ML risk score` usually a bad treatment variable?

Q3

Why might rollback plans appear associated with higher failure rates, even if they help?

Q4

What is a good interpretation of this effect?

```
ATE = -0.06
```

where:

```
Y = failure  
T = high_test_coverage
```

Q5

Why is positivity important for estimating the effect of CAB involvement?

Q6

Why might `actual rollback occurred` be a bad control when estimating whether rollback planning reduces failure?

Q7

What should you say instead of “we proved test coverage reduces risk”?

27. Answers

A1

The likely unit of analysis is:

```
one Change Request
```

Each row should represent one historical change request.

A2

Because the ML risk score is a prediction output, not a real-world intervention.

You usually cannot meaningfully ask:

```
what happens if we force the ML risk score to Low?
```

You want to ask:

```
what happens if we increase test coverage?  
what happens if we add a rollback plan?  
what happens if we change deployment timing?
```

A3

Because of confounding by indication.

Riskier or more complex changes are more likely to require rollback plans, and those changes are also more likely to fail. So a naive comparison may make rollback plans look harmful even if they reduce risk.

A4

It means:

```
High test coverage reduces failure probability by 6 percentage points on  
average,  
under the causal assumptions.
```

For example, from 14% to 8%.

A5

Because if every high-risk or production change always has CAB involvement, there may be no comparable non-CAB changes in that subgroup.

Without both treated and untreated examples, the effect cannot be estimated reliably from the data.

A6

Because actual rollback happens after deployment and may be caused by failure.

It is not a pre-treatment confounder. Controlling for it can block causal pathways or introduce post-treatment bias.

A7

Say:

Under the stated DAG assumptions and observed historical data, high test coverage is estimated to reduce failure probability by X percentage points.

28. What remains before you are project-ready

You have about **4-5 chapters left**.

Recommended order:

Chapter 6: Propensity scores, overlap, matching, and weighting
Chapter 7: Binary outcomes, logistic regression, marginal effects, and confidence intervals
Chapter 8: Sensitivity analysis, refuters, and robustness reporting
Chapter 9: Building the actual DoWhy CRP PoC pipeline
Chapter 10: Turning causal estimates into what-if simulation and quantified recommendations

After Chapter 10, you would be ready to start a basic but defensible implementation.

5/28/2026, 12:55:41 PM



Chapter 6 - Propensity Scores, Overlap, Matching, and Weighting

This chapter is still essential before starting the CRP project.

You now know:

Chapter 1: causal vs predictive questions
Chapter 2: potential outcomes and treatment effects
Chapter 3: DAGs and adjustment
Chapter 4: DoWhy implementation
Chapter 5: translating CRP into causal design

Now we cover the most common practical estimation tools:

propensity scores
overlap checks
matching
inverse probability weighting
covariate balance

These are especially relevant for your CRP project because the proposal expects historical Change Request data, causal effect estimation, intervention ranking, confidence intervals, DoWhy refutation, and robustness checks. `Causal_Inference_Internship_Proposal.docx`

1. The problem propensity scores solve

Suppose we want to estimate:

`rollback_plan → failure`

Treatment:

$T = 1$ if rollback plan present
 $T = 0$ if rollback plan absent

Outcome:

$Y = 1$ if change failed
 $Y = 0$ if change succeeded

Naive comparison:

failure rate among changes with rollback plan
-
failure rate among changes without rollback plan

Problem:

changes with rollback plans are probably not comparable to changes without rollback plans

Why?

Because rollback plans are more likely for:

complex changes
high-priority changes
production changes
risky assignment groups
CAB-reviewed changes
large deployment windows

So the treated and untreated groups differ before treatment.

Propensity score methods try to make treated and untreated groups more comparable.

2. Propensity score definition

The **propensity score** is:

$$e(X) = P(T = 1 \mid X)$$

Meaning:

The probability that a unit receives treatment, given its observed covariates.

Where:

T = treatment
X = pre-treatment covariates/confounders

Example:

$e(X) = P(\text{rollback_plan} = 1 \mid \text{complexity, priority, assignment_group_history, deployment_environment})$

If a change has:

high complexity
high priority
production environment
bad assignment group history

then it may have a high propensity for rollback planning.

Example:

$e(X) = 0.85$

Meaning:

Based on its observed characteristics, this change had an 85% probability of having a rollback plan.

3. Why propensity scores are useful

Instead of matching/adjusting on many covariates directly:

complexity
priority
category
assignment group
deployment window
environment
history

we compress them into one score:

$e(X)$

Then we compare treated and untreated units with similar propensity scores.

Example:

Change	Rollback plan	Propensity score	Failure
A	1	0.82	0
B	0	0.80	1
C	1	0.30	0
D	0	0.32	0

A and B are comparable because both had high probability of receiving rollback plans.

C and D are comparable because both had low probability.

So we compare:

treated high-propensity changes vs untreated high-propensity changes
treated low-propensity changes vs untreated low-propensity changes

4. Important warning

Propensity scores only adjust for observed covariates.

They do not solve unobserved confounding.

If this exists:

true_code_complexity $\rightarrow$ rollback_plan
true_code_complexity $\rightarrow$ failure

but true_code_complexity is not measured, propensity score methods cannot fix it.

They can only balance variables included in X .

So the core assumption remains:

$(Y(1), Y(0)) \perp\!\!\!\perp T \mid X$

Meaning:

After controlling for observed covariates X , treatment is as-if random.

This is a causal assumption, not something the algorithm proves.

5. Propensity score estimation

For binary treatment, we usually estimate propensity scores using logistic regression:

$$P(T = 1 \mid X)$$

Example:

```
rollback_plan ~ complexity + priority + assignment_group_history + environment
```

In Python:

```
from sklearn.linear_model import LogisticRegression

X = df[[
    "change_complexity",
    "priority_encoded",
    "assignment_group_failure_rate_90d",
    "environment_encoded"
]]

T = df["rollback_plan"]

ps_model = LogisticRegression(max_iter=1000)
ps_model.fit(X, T)

df["propensity_score"] = ps_model.predict_proba(X)[:, 1]
```

Now every change has an estimated probability of receiving the treatment.

6. Overlap / common support

Before estimating effects, check overlap.

Overlap means:

treated and untreated groups exist at similar propensity score ranges

Good overlap:

treated scores: 0.15 to 0.85
untreated scores: 0.10 to 0.80

Bad overlap:

treated scores: 0.70 to 0.99
untreated scores: 0.01 to 0.30

If treated and untreated groups barely overlap, you are comparing fundamentally different types of changes.

That means the causal effect estimate depends on extrapolation.

7. CRP overlap examples

Example 1: CAB involvement

Suppose:

all production database changes require CAB involvement

Then for production database changes:

$$P(\text{CAB} = 1 \mid \text{production database change}) = 1$$

There are no comparable non-CAB changes.

So you cannot estimate:

effect of CAB involvement among production database changes

with ordinary adjustment.

Example 2: test coverage

Suppose emergency hotfixes almost always have low test coverage.

Then:

```
P(high_test_coverage = 1 | emergency hotfix) ≈ 0
```

You cannot reliably estimate the effect of high test coverage for emergency hotfixes unless there are enough emergency hotfixes with high test coverage.

This matters because the proposal expects what-if simulation, but some what-if questions may be unsupported by historical data. `Causal_Inference_Internship_Proposal.docx`

8. Visual overlap check

A simple diagnostic:

```
import matplotlib.pyplot as plt

treated = df[df["rollback_plan"] == 1]["propensity_score"]
untreated = df[df["rollback_plan"] == 0]["propensity_score"]

plt.hist(treated, alpha=0.5, label="Rollback plan")
plt.hist(untreated, alpha=0.5, label="No rollback plan")
plt.xlabel("Propensity score")
plt.ylabel("Count")
plt.legend()
plt.show()
```

Interpretation:

Large overlap → comparison is more credible
Little overlap → weak causal support
No overlap → do not estimate for that region

For a serious report, include this diagnostic.

9. Trimming

If overlap is poor at the extremes, you can trim observations.

Example:

```
remove rows where propensity_score < 0.05
remove rows where propensity_score > 0.95
```

Python:

```
df_trimmed = df[
    (df["propensity_score"] > 0.05) &
    (df["propensity_score"] < 0.95)
].copy()
```

This changes the estimand.

You are no longer estimating the effect for all changes.

You are estimating the effect for changes where treatment assignment had some uncertainty.

Better wording:

```
Estimated effect among changes with sufficient overlap
```

not:

```
Estimated effect for all changes
```

10. Matching

Matching compares treated units to similar untreated units.

With propensity score matching:

```
For each treated unit, find untreated unit(s) with similar propensity score.
```

Example:

Change	T	Propensity	Match
A	1	0.72	B
B	0	0.70	A

Change	T	Propensity	Match
C	1	0.41	D
D	0	0.43	C

Then estimate:

average difference in outcomes between matched pairs

If:

A failure = 0
B failure = 1

then:

pair difference = 0 - 1 = -1

Meaning the treated change had lower failure than its matched untreated comparison.

11. Matching intuition for CRP

Treatment:

rollback_plan_present

For a treated change:

priority = high
complexity = high
environment = production
assignment_group_failure_rate = 12%

Find an untreated change with similar:

priority
complexity
environment
assignment-group history

Then compare failure outcomes.

This is more reasonable than comparing all rollback-plan changes to all non-rollback-plan changes.

12. Matching in DoWhy

DoWhy can do propensity score matching:

```
estimate_matching = model.estimate_effect(  
    identified_estimand,  
    method_name="backdoor.propensity_score_matching"  
)  
  
print(estimate_matching)  
print("Estimated effect:", estimate_matching.value)
```

This assumes your causal graph identifies a backdoor adjustment set.

DoWhy estimates the effect using the graph-selected confounders.

13. Matching limitations

Matching has several problems.

Problem 1: bad matches

If the nearest untreated unit is still very different, the match is weak.

Example:

```
treated propensity = 0.92  
nearest untreated propensity = 0.61
```

That is not a good match.

Use a caliper.

A **caliper** is a maximum allowed distance.

Example:

```
only match if propensity score difference ≤ 0.05
```

Problem 2: discarded data

Matching may discard many units.

This can reduce sample size and precision.

Problem 3: still only observed confounders

Matching does not fix unmeasured variables.

Problem 4: high-dimensional sparse data

If many categorical variables exist, exact matching becomes hard.

Example:

```
assignment_group
environment
change_type
priority
category
deployment_window
```

There may be few close matches.

14. Inverse Probability Weighting

Inverse Probability Weighting, or IPW, uses propensity scores differently.

Instead of matching units, it weights them.

The goal is to create a pseudo-population where treatment is independent of covariates.

For ATE:

```
weight = 1 / e(X)      if T = 1
weight = 1 / (1-e(X))  if T = 0
```

Where:

```
e(X) = P(T = 1 | X)
```

15. IPW intuition

Suppose a treated unit had low probability of being treated:

$$T = 1$$
$$e(X) = 0.10$$

Weight:

$$1 / 0.10 = 10$$

This unit is rare and informative, so it gets high weight.

Suppose another treated unit had high probability of being treated:

$$T = 1$$
$$e(X) = 0.90$$

Weight:

$$1 / 0.90 = 1.11$$

This unit is common, so it gets lower weight.

For untreated units:

$$T = 0$$
$$e(X) = 0.90$$

Weight:

$$1 / (1 - 0.90) = 10$$

This is an unusual untreated unit, so it gets high weight.

16. Why IPW can explode

If propensity scores are close to 0 or 1, weights become huge.

Example:

$$e(X) = 0.01$$
$$\text{treated weight} = 1 / 0.01 = 100$$

or:

```
e(X) = 0.99  
untreated weight = 1 / 0.01 = 100
```

Huge weights make estimates unstable.

This is why overlap diagnostics are mandatory.

In CRP, this can happen when policy rules almost determine treatment.

Example:

```
all emergency changes require special approval  
all production changes require CAB approval  
all low-risk changes skip CAB approval
```

Then IPW can become unstable.

17. Stabilized weights

A more stable version uses stabilized weights.

For ATE:

```
if T = 1:  
    weight = P(T = 1) / e(X)  
  
if T = 0:  
    weight = P(T = 0) / (1 - e(X))
```

These reduce variance while preserving the balancing idea.

Example:

```
P(T = 1) = 0.40  
e(X) = 0.10  
stabilized treated weight = 0.40 / 0.10 = 4
```

instead of:

```
unstabilized weight = 10
```

18. IPW in DoWhy

```
estimate_ipw = model.estimate_effect(  
    identified_estimand,  
    method_name="backdoor.propensity_score_weighting"  
)  
  
print(estimate_ipw)  
print("Estimated effect:", estimate_ipw.value)
```

This is often a useful comparison against regression and matching.

If all methods give similar results, that increases confidence.

If they disagree heavily, investigate:

```
poor overlap  
bad model specification  
outliers  
nonlinear effects  
unmeasured confounding  
wrong DAG  
wrong treatment definition
```

19. Covariate balance

The goal of matching/weighting is not just to estimate a number.

It is to balance covariates.

Before matching/weighting, treated and untreated groups may differ.

Example:

Covariate	Treated mean	Untreated mean	Difference
Complexity	8.1	4.3	3.8
Priority	3.5	2.1	1.4
Group failure rate	0.14	0.05	0.09

After matching/weighting, differences should shrink.

Example:

Covariate	Treated mean	Untreated mean	Difference
Complexity	6.9	6.8	0.1
Priority	2.9	3.0	-0.1
Group failure rate	0.09	0.10	-0.01

This tells you the comparison is more credible.

20. Standardized Mean Difference

A common balance metric is **Standardized Mean Difference**, or SMD.

For a covariate X :

$$\text{SMD} = (\text{mean_treated} - \text{mean_control}) / \text{pooled_standard_deviation}$$

Interpretation:

SMD near 0 = balanced
large SMD = imbalance

Common rule of thumb:

$$|\text{SMD}| < 0.1 \text{ is acceptable balance}$$

This is not a law, but it is a useful diagnostic.

21. SMD Python function

```
import numpy as np

def standardized_mean_difference(df, treatment_col, covariate_col,
weight_col=None):
    treated = df[df[treatment_col] == 1]
    control = df[df[treatment_col] == 0]
```

```

if weight_col is None:
    mean_t = treated[covariate_col].mean()
    mean_c = control[covariate_col].mean()
    var_t = treated[covariate_col].var()
    var_c = control[covariate_col].var()
else:
    mean_t = np.average(treated[covariate_col],
weights=treated[weight_col])
    mean_c = np.average(control[covariate_col],
weights=control[weight_col])

    var_t = np.average(
        (treated[covariate_col] - mean_t) ** 2,
        weights=treated[weight_col]
    )
    var_c = np.average(
        (control[covariate_col] - mean_c) ** 2,
        weights=control[weight_col]
    )

pooled_sd = np.sqrt((var_t + var_c) / 2)

if pooled_sd == 0:
    return 0

return (mean_t - mean_c) / pooled_sd

```

Usage:

```

covariates = [
    "change_complexity",
    "priority_encoded",
    "assignment_group_failure_rate_90d",
    "environment_encoded"
]

for c in covariates:
    smd = standardized_mean_difference(df, "rollback_plan", c)
    print(c, smd)

```

22. Weighted balance check

After computing IPW weights:

```
df["ipw_weight"] = np.where(
    df["rollback_plan"] == 1,
    1 / df["propensity_score"],
    1 / (1 - df["propensity_score"])
)
```

Check weighted SMD:

```
for c in covariates:
    smd = standardized_mean_difference(
        df,
        treatment_col="rollback_plan",
        covariate_col=c,
        weight_col="ipw_weight"
    )
    print(c, smd)
```

Expected:

weighted SMDs should be closer to 0 than unweighted SMDs

If not, your propensity score model is not balancing covariates well.

23. Regression adjustment vs propensity scores

Regression adjustment models:

$$E[Y \mid T, X]$$

Propensity score methods model:

$$P[T = 1 \mid X]$$

So:

Method	What it models
Regression adjustment	Outcome
Propensity score matching	Treatment assignment
IPW	Treatment assignment
Doubly robust methods	Both outcome and treatment

Regression asks:

Given treatment and covariates, what is the expected outcome?

Propensity asks:

Given covariates, how likely was treatment?

Both aim to adjust for confounding.

24. Doubly robust estimation

A **doubly robust** estimator uses both:

1. outcome model
2. treatment model / propensity model

The useful property:

If either the outcome model or the treatment model is correctly specified, the estimator can still be consistent.

Not magic. But useful.

Examples:

Augmented Inverse Probability Weighting
Double Machine Learning
EconML estimators

For your first PoC, do not start here.

Start with:

```
linear/logistic regression  
propensity score weighting  
matching  
balance diagnostics
```

Then use doubly robust methods later.

25. Propensity methods for multiple treatments

So far we assumed binary treatment.

But CRP may involve categorical treatments:

```
deployment_window = weekday/daytime  
deployment_window = weekday/night  
deployment_window = weekend  
deployment_window = freeze period
```

For a first PoC, simplify to binary.

Example:

```
off_hours_deployment = 1  
off_hours_deployment = 0
```

or:

```
high_test_coverage = 1 if test_coverage >= 80  
high_test_coverage = 0 otherwise
```

Do not begin with multi-treatment causal inference unless necessary.

26. Propensity methods for continuous treatment

For continuous treatment:

```
test_coverage = 0 to 100
```

ordinary binary propensity score does not apply directly.

Options:

1. binarize treatment
2. use generalized propensity scores
3. use dose-response models
4. use causal forests / EconML later

For the CRP PoC, binarization is likely best.

Example thresholds:

```
test_coverage >= 80
test_coverage >= 70
test_coverage improved by at least 20 points
```

But be careful: arbitrary thresholds can distort conclusions.

Use domain-relevant thresholds if possible.

27. How to report propensity results

A good causal report should include:

```
Treatment definition
Outcome definition
Adjustment variables
Propensity score model
Overlap plot
Covariate balance before/after
Estimator used
Estimated effect
Confidence interval
Refutation results
Limitations
```

For CRP, that means not just:

```
rollback plan reduces risk by 8%
```

but:

Treatment: rollback_plan_present
Outcome: failure within 48h
Adjustment set: priority, complexity, environment, assignment_group_failure_rate_90d
Overlap: acceptable after trimming 3.2% of rows
Balance: all weighted SMDs < 0.1
Estimate: -4.6 percentage points
95% CI: -7.8 to -1.4 percentage points
Limitations: true code complexity and team maturity are imperfectly observed

28. Complete synthetic example

This example shows the workflow without DoWhy.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression, LinearRegression

np.random.seed(42)
n = 2000

complexity = np.random.normal(0, 1, n)
priority = np.random.normal(0, 1, n)
group_history = np.random.normal(0, 1, n)

# Treatment assignment: risky changes more likely to have rollback plans
logit_t = -0.2 + 1.0 * complexity + 0.7 * priority + 0.8 * group_history
p_t = 1 / (1 + np.exp(-logit_t))
rollback_plan = np.random.binomial(1, p_t)

# True causal effect: rollback plan reduces failure probability
logit_y = (
    -1
    - 0.8 * rollback_plan
    + 1.0 * complexity
    + 0.7 * priority
    + 0.8 * group_history
)

p_y = 1 / (1 + np.exp(-logit_y))
```

```

failure = np.random.binomial(1, p_y)

df = pd.DataFrame({
    "rollback_plan": rollback_plan,
    "failure": failure,
    "complexity": complexity,
    "priority": priority,
    "group_history": group_history
})

# Naive difference
naive = (
    df[df["rollback_plan"] == 1]["failure"].mean()
    - df[df["rollback_plan"] == 0]["failure"].mean()
)

print("Naive difference:", naive)

# Estimate propensity score
X = df[["complexity", "priority", "group_history"]]
T = df["rollback_plan"]

ps_model = LogisticRegression(max_iter=1000)
ps_model.fit(X, T)

df["propensity_score"] = ps_model.predict_proba(X)[:, 1]

# IPW weights
df["ipw_weight"] = np.where(
    df["rollback_plan"] == 1,
    1 / df["propensity_score"],
    1 / (1 - df["propensity_score"])
)

# Weighted outcome means
treated = df[df["rollback_plan"] == 1]
control = df[df["rollback_plan"] == 0]

weighted_treated_failure = np.average(
    treated["failure"],
    weights=treated["ipw_weight"]
)

```

```

)

weighted_control_failure = np.average(
    control["failure"],
    weights=control["ipw_weight"]
)

ipw_ate = weighted_treated_failure - weighted_control_failure

print("IPW ATE:", ipw_ate)

```

Expected pattern:

Naive estimate may be misleading because rollback plans are assigned to riskier changes.
IPW estimate should move closer to the true protective effect.

29. Why naive estimates can reverse direction

This is important for CRP.

Suppose rollback plans truly reduce failure.

But rollback plans are mostly used on complex changes.

Then observed data may show:

failure rate with rollback plan = 18%
failure rate without rollback plan = 10%
naive difference = +8 percentage points

Naive interpretation:

rollback plans increase failure

But after adjustment:

comparable high-risk changes with rollback plan = 18%
comparable high-risk changes without rollback plan = 25%
adjusted difference = -7 percentage points

Causal interpretation:

rollback plans reduce failure among comparable changes

This is why causal inference is useful.

The raw association can point in the wrong direction.

30. Chapter 6 summary

Propensity scores estimate:

$P(T = 1 \mid X)$

They help compare treated and untreated units with similar observed covariates.

Main uses:

matching
weighting
overlap diagnostics
covariate balance checking

Key assumptions:

no unmeasured confounding
positivity / overlap
correct treatment timing
well-defined treatment

Matching compares treated and untreated units with similar propensity scores.

IPW reweights units to create a pseudo-population where treatment is less confounded.

Always check:

propensity score overlap
covariate balance before/after adjustment
extreme weights
sample loss after trimming

For CRP, propensity methods are useful because many process controls are assigned non-randomly to riskier changes.

31. Minimal vocabulary for Chapter 6

Term	Meaning
Propensity score	Probability of receiving treatment given covariates
Common support	Region where treated and untreated units both exist
Overlap	Treated and untreated groups have comparable propensity scores
Matching	Pairing treated and untreated units with similar covariates/scores
Caliper	Maximum allowed matching distance
IPW	Inverse Probability Weighting
Stabilized weight	Lower-variance version of IPW weight
Extreme weight	Very large weight caused by propensity near 0 or 1
Covariate balance	Similarity of covariate distributions after adjustment
SMD	Standardized Mean Difference
Trimming	Removing observations with poor overlap
Doubly robust estimator	Uses both treatment and outcome models

32. Practice questions

Q1

What is the propensity score?

Q2

Why are propensity scores useful?

Q3

In CRP, why might rollback plans be more common among changes that fail?

Q4

What does poor overlap mean?

Q5

What happens to IPW weights when propensity scores are close to 0 or 1?

Q6

What is covariate balance?

Q7

Why does matching not solve unobserved confounding?

Q8

For the first CRP PoC, why is binary treatment easier than continuous treatment?

33. Answers

A1

The propensity score is:

$$e(X) = P(T = 1 \mid X)$$

It is the probability of receiving treatment given observed covariates.

A2

They help compare treated and untreated units that had similar probabilities of receiving treatment. This makes the comparison less confounded by observed covariates.

A3

Because rollback plans are likely required for riskier, more complex, or higher-priority changes. Those changes are also more likely to fail.

So rollback plans can appear positively associated with failure even if they causally reduce failure risk.

A4

Poor overlap means treated and untreated units do not exist in the same covariate/propensity score regions.

Example:

```
all high-complexity changes have rollback plans
no high-complexity changes lack rollback plans
```

Then there is no valid comparison group for high-complexity treated changes.

A5

Weights become very large.

Example:

```
e(X) = 0.01
treated weight = 1 / 0.01 = 100
```

Large weights make estimates unstable.

A6

Covariate balance means treated and untreated groups have similar distributions of pre-treatment covariates after matching or weighting.

Example:

```
complexity, priority, environment, assignment-group history
```

should be similar across treated and untreated groups.

A7

Because matching only balances observed variables.

If an important hidden variable affects both treatment and outcome, matching cannot account for it.

A8

Because binary treatments allow simpler estimands like:

$$E[Y(1) - Y(0)]$$

Continuous treatments require dose-response modelling or generalized propensity scores, which are more advanced.

For a first PoC, use treatments such as:

```
high_test_coverage = 1 if coverage >= 80%  
rollback_plan_present = 1/0
```

34. What remains

About **3-4 chapters left** before you are ready to implement the CRP PoC.

Recommended next:

```
Chapter 7: Binary outcomes, logistic regression, marginal effects, and  
confidence intervals  
Chapter 8: Sensitivity analysis, refuters, and robustness reporting  
Chapter 9: Building the actual DoWhy CRP PoC pipeline  
Chapter 10: What-if simulation and quantified recommendations
```

5/28/2026, 2:08:28 PM



Chapter 7 - Binary Outcomes, Logistic Regression, Marginal Effects, and Confidence Intervals

This chapter is necessary before implementing the CRP PoC because your outcome is almost certainly binary:

```
failure = 1  
success = 0
```

The proposal's required outcome fields include "change result (success / failure)" and "incident linkage," so most CRP causal estimates will be effects on the probability of failure.

Causal_Inference_Internship_Proposal.docx

The key issue:

When the outcome is binary, your causal effect should usually be reported as a change in probability, not just as a regression coefficient.

1. Binary outcome setup

For CRP:

```
Y = 1 if the change failed  
Y = 0 if the change succeeded
```

Treatment examples:

```
T = 1 if rollback plan present  
T = 0 if rollback plan absent
```

or:

```
T = 1 if test coverage >= 80%  
T = 0 otherwise
```

The causal question becomes:

Does the treatment reduce the probability of failure?

The causal effect is:

$$E[Y(1) - Y(0)]$$

Since Y is binary, this is:

$$P(\text{failure if treated}) - P(\text{failure if untreated})$$

Example:

$$P(\text{failure} \mid \text{do}(T = 1)) = 0.08$$

$$P(\text{failure} \mid \text{do}(T = 0)) = 0.14$$

$$\text{ATE} = 0.08 - 0.14 = -0.06$$

Interpretation:

The treatment reduces failure probability by 6 percentage points.

2. Percentage points vs percent

This distinction matters.

Suppose failure risk goes from:

$$14\% \rightarrow 8\%$$

Then:

$$\text{absolute reduction} = 14\% - 8\% = 6 \text{ percentage points}$$

$$\text{relative reduction} = 6 / 14 = 42.9\%$$

So these are different:

failure risk reduced by 6 percentage points

failure risk reduced by 42.9 percent relative to baseline

For causal reporting, prefer:

The treatment reduced failure probability by 6 percentage points, from 14% to

8%.

Avoid vague phrasing like:

risk reduced by 6%

because it is ambiguous.

This is directly relevant to the proposal's example: "Increasing test coverage from 45% to 80% would reduce risk by 22%." That should be clarified as either a **22 percentage-point absolute reduction** or a **22% relative reduction**.

Causal_Inference_Internship_Proposal.docx

3. Linear Probability Model

The simplest model for a binary outcome is the **Linear Probability Model**, or LPM.

It is just linear regression where the outcome is 0/1.

$$\text{failure} = \beta_0 + \beta_1 \text{ treatment} + \beta_2 X + \text{error}$$

Example:

$$\begin{aligned} \text{failure} = & \beta_0 \\ & + \beta_1 \text{ rollback_plan} \\ & + \beta_2 \text{ complexity} \\ & + \beta_3 \text{ priority} \\ & + \beta_4 \text{ assignment_group_history} \\ & + \text{error} \end{aligned}$$

Here:

$$\beta_1 = \text{effect of rollback plan on failure probability}$$

If:

$$\beta_1 = -0.05$$

then:

Rollback planning is estimated to reduce failure probability by 5 percentage points, under the stated assumptions.

4. Why LPM is useful

The main advantage is interpretability.

Example:

$$\beta_1 = -0.04$$

means:

-4 percentage points

That is easy to explain to stakeholders.

For a first CRP PoC, LPM is often a good baseline because the project needs quantified recommendations and confidence intervals, not just black-box predictions. The proposal's intended output is "WHY causal + HOW MUCH quantified reduction," so interpretability is important. `Causal_Inference_Internship_Proposal.docx`

5. LPM limitations

LPM has weaknesses.

Problem 1: Predictions outside 0 and 1

A linear model can predict:

$$P(\text{failure}) = -0.08$$

or:

$$P(\text{failure}) = 1.12$$

These are impossible probabilities.

Problem 2: Nonlinear probability effects

The effect of treatment may not be constant.

Example:

Adding a rollback plan may reduce failure more for high-complexity changes than low-complexity changes.

Problem 3: Heteroskedasticity

Because the outcome is binary, the error variance is not constant.

This affects standard errors.

Use robust standard errors if using LPM.

6. LPM in Python

```
import statsmodels.api as sm

Y = df["failure"]

X = df[[
    "rollback_plan",
    "change_complexity",
    "priority_encoded",
    "assignment_group_failure_rate_90d",
    "environment_encoded"
]]

X = sm.add_constant(X)

lpm = sm.OLS(Y, X).fit(cov_type="HC3")

print(lpm.summary())
print("Estimated treatment effect:", lpm.params["rollback_plan"])
```

If output says:

```
rollback_plan coefficient = -0.047
```

interpret as:

Rollback planning is estimated to reduce failure probability by 4.7 percentage points.

7. Logistic regression

Logistic regression is often used for binary outcomes.

Instead of modeling probability directly, it models log-odds:

$$\text{logit}(P(Y = 1)) = \beta_0 + \beta_1 T + \beta_2 X$$

where:

$$\text{logit}(p) = \log(p / (1 - p))$$

Example:

$$\begin{aligned} \text{logit}(P(\text{failure} = 1)) = \\ & \beta_0 \\ & + \beta_1 \text{ rollback_plan} \\ & + \beta_2 \text{ complexity} \\ & + \beta_3 \text{ priority} \end{aligned}$$

Logistic regression keeps predicted probabilities between 0 and 1.

8. The problem with logistic coefficients

A logistic regression coefficient is not directly a probability change.

If:

$$\beta_1 = -0.7$$

that does not mean:

failure probability decreases by 70%

It means the log-odds decrease by 0.7.

Odds ratio:

$$\text{odds ratio} = \exp(\beta_1)$$

If:

```
 $\beta_1 = -0.7$   
 $\exp(-0.7) = 0.496$ 
```

Interpretation:

The odds of failure are about 50.4% lower, conditional on covariates.

But stakeholders usually care about probability:

```
failure probability goes from 14% to 9%
```

not odds.

So for the CRP project, convert logistic results into probability differences.

9. Logistic regression in Python

```
import statsmodels.api as sm  
  
Y = df["failure"]  
  
X = df[[  
    "rollback_plan",  
    "change_complexity",  
    "priority_encoded",  
    "assignment_group_failure_rate_90d",  
    "environment_encoded"  
]]  
  
X = sm.add_constant(X)  
  
logit_model = sm.Logit(Y, X).fit()  
  
print(logit_model.summary())  
print("Log-odds coefficient:", logit_model.params["rollback_plan"])  
print("Odds ratio:", np.exp(logit_model.params["rollback_plan"]))
```

This gives a logistic model, but not yet the best causal reporting number.

For causal interpretation, compute marginal effects.

10. Marginal effects

A **marginal effect** converts the logistic model into probability-scale effects.

For binary treatment, the most useful quantity is:

```
average predicted probability if everyone treated
-
average predicted probability if everyone untreated
```

That is:

```
mean(P(Y = 1 | T = 1, X))
-
mean(P(Y = 1 | T = 0, X))
```

This estimates:

```
E[Y(1) - Y(0)]
```

under the adjustment assumptions.

11. Manual marginal effect calculation

Suppose treatment is:

```
rollback_plan
```

Fit logistic regression:

```
import numpy as np
import statsmodels.api as sm

Y = df["failure"]

covariates = [
    "rollback_plan",
    "change_complexity",
    "priority_encoded",
    "assignment_group_failure_rate_90d",
```

```

    "environment_encoded"
]

X = sm.add_constant(df[covariates])

logit_model = sm.Logit(Y, X).fit()

```

Now create two counterfactual datasets.

Everyone treated

```

X_treated = X.copy()
X_treated["rollback_plan"] = 1

p_treated = logit_model.predict(X_treated)

```

Everyone untreated

```

X_untreated = X.copy()
X_untreated["rollback_plan"] = 0

p_untreated = logit_model.predict(X_untreated)

```

Average treatment effect

```

ate = (p_treated - p_untreated).mean()

print("ATE on probability scale:", ate)
print("ATE in percentage points:", ate * 100)

```

If:

```
ate = -0.046
```

report:

Rollback planning is estimated to reduce failure probability by 4.6 percentage points.

12. Difference between coefficient and marginal effect

For logistic regression:

```
coefficient ≠ probability effect
```

Example:

```
β1 = -0.7  
odds ratio = 0.50  
marginal effect = -0.046
```

These mean:

```
β1 = treatment lowers log-odds by 0.7  
odds ratio = treatment roughly halves the odds  
marginal effect = treatment lowers failure probability by 4.6 percentage points
```

For business use, the marginal effect is usually the most interpretable.

13. Average marginal effect vs effect at the mean

There are two common summaries.

Effect at the mean

Set covariates to their average values, then compare treated vs untreated.

```
P(Y=1 | T=1, X=mean)  
-  
P(Y=1 | T=0, X=mean)
```

Problem:

The “average change request” may not correspond to a real change.

Average marginal effect

Compute the effect for every row, then average.

```
for each change:  
  predict failure if treated  
  predict failure if untreated  
  average the differences
```

This is usually better.

Use average marginal effects.

14. Confidence intervals

A point estimate alone is not enough.

Bad output:

```
rollback plan reduces failure by 4.6 percentage points
```

Better output:

```
rollback plan reduces failure by 4.6 percentage points  
95% CI: 1.8 to 7.2 percentage points
```

A confidence interval communicates uncertainty.

The proposal expects confidence intervals in causal recommendations, so this is not optional for the final PoC. `Causal_Inference_Internship_Proposal.docx`

15. Bootstrap confidence interval

Bootstrap is one of the easiest methods.

Process:

1. Resample rows with replacement.
2. Refit the model.
3. Recompute the causal effect.
4. Repeat many times.
5. Use percentiles of effects as confidence interval.

Example:

```
import numpy as np  
import statsmodels.api as sm  
  
def estimate_logit_ate(data, treatment_col, outcome_col, covariate_cols):  
    Y = data[outcome_col]
```

```

X = sm.add_constant(data[[treatment_col] + covariate_cols])

model = sm.Logit(Y, X).fit(dis=False)

X_treated = X.copy()
X_treated[treatment_col] = 1

X_untreated = X.copy()
X_untreated[treatment_col] = 0

p_treated = model.predict(X_treated)
p_untreated = model.predict(X_untreated)

return (p_treated - p_untreated).mean()

treatment_col = "rollback_plan"
outcome_col = "failure"
covariate_cols = [
    "change_complexity",
    "priority_encoded",
    "assignment_group_failure_rate_90d",
    "environment_encoded"
]

B = 500
effects = []

for b in range(B):
    sample = df.sample(n=len(df), replace=True, random_state=b)

    try:
        effect = estimate_logit_ate(
            sample,
            treatment_col,
            outcome_col,
            covariate_cols
        )
        effects.append(effect)
    except Exception:
        continue

```

```

effects = np.array(effects)

lower = np.percentile(effects, 2.5)
upper = np.percentile(effects, 97.5)
point_estimate = estimate_logit_ate(
    df,
    treatment_col,
    outcome_col,
    covariate_cols
)

print("Point estimate:", point_estimate)
print("95% CI:", lower, upper)
print("Percentage-point estimate:", point_estimate * 100)
print("Percentage-point CI:", lower * 100, upper * 100)

```

16. Bootstrap interpretation

Suppose output is:

```

Point estimate: -0.046
95% CI: -0.073 to -0.018

```

Report:

Rollback planning is estimated to reduce failure probability by 4.6 percentage points, with a 95% bootstrap confidence interval from 1.8 to 7.3 percentage points.

Because the whole interval is below zero:

```
-7.3 pp to -1.8 pp
```

the estimate is directionally stable.

If output is:

```

Point estimate: -0.046
95% CI: -0.110 to 0.021

```

then report:

The point estimate suggests a 4.6 percentage-point reduction, but the confidence interval includes zero, so the evidence is statistically uncertain.

17. Standard error vs confidence interval

A standard error tells you uncertainty around the estimate.

A 95% confidence interval is often:

```
estimate ± 1.96 × standard error
```

But for causal workflows, bootstrap intervals are often easier because they can wrap around the whole estimation process:

```
propensity score estimation  
matching/weighting  
outcome model  
marginal effect calculation
```

For the CRP PoC, bootstrap CI is practical and defensible.

18. Class imbalance

Failure may be rare.

Example:

```
failure rate = 3%
```

This creates issues:

```
few positive examples  
unstable estimates  
large confidence intervals  
separation in logistic regression  
poor subgroup estimates
```

If failure is rare, you need to be careful.

Things to check:

- overall failure rate
- failure rate by treatment group
- number of failures in treated group
- number of failures in untreated group
- number of failures after trimming/matching

If you have:

- 20 total failures

do not estimate many separate treatment effects.

Start with one or two high-support interventions.

19. Separation in logistic regression

Separation happens when a variable perfectly predicts the outcome.

Example:

- all changes with rollback_plan = 1 succeeded

Then logistic regression may fail or produce huge coefficients.

Symptoms:

- model does not converge
- very large coefficient
- huge standard errors
- warning about perfect separation

Possible solutions:

- use LPM baseline
- use penalized logistic regression
- combine sparse categories
- increase sample size
- avoid overfitting with too many covariates

For a first PoC, if logistic regression struggles, use LPM with robust standard errors as a baseline.

20. Categorical variables

CRP data will contain categorical variables:

```
change_type
priority
category
environment
assignment_group
deployment_window
```

These need encoding.

For regression:

```
df_encoded = pd.get_dummies(
    df,
    columns=[
        "change_type",
        "priority",
        "environment",
        "deployment_window"
    ],
    drop_first=True
)
```

Be careful with high-cardinality variables like:

```
assignment_group
service_name
application_id
```

If there are hundreds of groups, one-hot encoding may overfit or create sparse categories.

Alternatives:

```
use assignment_group_failure_rate_90d
group rare categories into "Other"
```

```
use hierarchical/grouped features  
regularized models
```

For causal inference, avoid target encoding unless carefully time-safe.

Bad:

```
assignment_group_failure_rate computed using future failures
```

Good:

```
assignment_group_failure_rate computed only from previous 90 days before the  
change
```

The proposal specifically mentions “assignment group failure rate trailing 90 days,” which is good because it preserves time ordering. `Causal_Inference_Internship_Proposal.docx`

21. Time-safe feature engineering

This is critical.

For each change request, features must be computed using only information available before deployment.

Bad feature:

```
assignment group failure rate over the full dataset
```

Why bad?

Because it uses future failures.

Good feature:

```
assignment group failure rate in the 90 days before the scheduled start date
```

Bad feature:

```
incident count after deployment
```

Good feature:

```
incident count for same service in previous 30 days
```

Bad feature:

```
final approval delay after failure review
```

Good feature:

```
approval duration before deployment
```

This is both a machine learning leakage issue and a causal time-ordering issue.

22. Interaction effects

Treatment effects may differ across change types.

Example:

```
rollback plans may matter more for database changes than frontend copy changes
```

You can model this with an interaction:

```
failure ~ rollback_plan + database_change + rollback_plan × database_change +  
confounders
```

Python:

```
df["rollback_x_database"] = (  
    df["rollback_plan"] * df["database_change"]  
)  
  
x = df[[  
    "rollback_plan",  
    "database_change",  
    "rollback_x_database",  
    "change_complexity",  
    "priority_encoded"  
]]
```

Interpretation:

```
rollback_plan coefficient = effect for non-database changes
interaction coefficient = additional effect for database changes
```

For first PoC, do not add too many interactions. Start simple, then explore heterogeneity once the base pipeline is stable.

23. CATE for CRP

CATE means Conditional Average Treatment Effect.

Example:

```
effect of rollback plan among high-complexity changes
effect of rollback plan among low-complexity changes
```

Possible output:

Group	Estimated effect
Low complexity	-1.2 pp
Medium complexity	-3.8 pp
High complexity	-9.5 pp

This is useful because recommendations should be targeted.

But subgroup estimates need enough data.

Do not report:

```
rollback plan reduces failure by 12 pp for database changes
```

if there were only 8 database failures.

24. From model output to recommendation

Suppose logistic marginal effects give:

```
Treatment: high_test_coverage
ATE: -0.052
```

95% CI: -0.081 to -0.020

Good recommendation:

For comparable historical changes, high test coverage was associated with an estimated 5.2 percentage-point reduction in failure probability, under the stated DAG assumptions. The 95% bootstrap CI was 2.0 to 8.1 percentage points.

Bad recommendation:

Increase test coverage because it will reduce risk by 5.2%.

Why bad?

Because:

causal assumptions are hidden
percentage points unclear
confidence interval missing
historical-data limitation missing

25. Multiple treatments problem

The CRP engine may estimate effects for many possible interventions:

test coverage
rollback plan
deployment window
CAB involvement
approval count
change freeze
review duration
assignment group

If you test many treatments, some will appear significant by chance.

For a first PoC:

predefine 2-3 interventions
avoid fishing

report all tested interventions
do not cherry-pick only significant ones

Later, if ranking interventions, include:

effect size
confidence interval
sample support
overlap quality
actionability
business feasibility

Not just the largest numerical effect.

26. Recommended estimator stack for CRP PoC

For each treatment, run multiple estimators.

Baseline

Linear Probability Model with robust standard errors

Main interpretable estimate

Logistic regression with average marginal effect

Causal adjustment comparison

Propensity score weighting

Robustness

Bootstrap CI
DoWhy refuters
Overlap checks
Covariate balance

If all broadly agree, confidence increases.

If they disagree, do not force a conclusion. Investigate.

27. Example reporting table

Treatment	Effect on failure	95% CI	Overlap	Refuters	Recommendation
High test coverage	-5.2 pp	-8.1 to -2.0 pp	Good	Passed	Recommend
Rollback plan	-3.4 pp	-7.5 to +0.8 pp	Moderate	Mixed	Use caution
Off-hours deployment	+2.1 pp	-1.2 to +5.8 pp	Poor	Failed subset	Do not claim

This style is much better than a single ranked list.

28. Minimal Python function for logistic ATE

```
import numpy as np
import statsmodels.api as sm

def logistic_ate(
    df,
    treatment_col,
    outcome_col,
    covariate_cols
):
    data = df[[treatment_col, outcome_col] + covariate_cols].dropna()

    Y = data[outcome_col]
    X = sm.add_constant(data[[treatment_col] + covariate_cols])

    model = sm.Logit(Y, X).fit(dis=False)

    X1 = X.copy()
    X1[treatment_col] = 1

    X0 = X.copy()
    X0[treatment_col] = 0
```

```

p1 = model.predict(X1)
p0 = model.predict(X0)

ate = np.mean(p1 - p0)

return {
    "ate": ate,
    "ate_percentage_points": ate * 100,
    "model": model,
    "n": len(data)
}

```

Usage:

```

result = logistic_ate(
    df=df,
    treatment_col="high_test_coverage",
    outcome_col="failure",
    covariate_cols=[
        "change_complexity",
        "priority_encoded",
        "assignment_group_failure_rate_90d",
        "environment_encoded"
    ]
)

print(result["ate_percentage_points"])

```

29. Minimal Python function for bootstrap CI

```

def bootstrap_logistic_ate(
    df,
    treatment_col,
    outcome_col,
    covariate_cols,
    n_bootstrap=500,
    seed=42
):
    rng = np.random.default_rng(seed)

```

```

effects = []

clean_df = df[[treatment_col, outcome_col] + covariate_cols].dropna()

for _ in range(n_bootstrap):
    sample_indices = rng.choice(
        clean_df.index,
        size=len(clean_df),
        replace=True
    )

    sample = clean_df.loc[sample_indices]

    try:
        result = logistic_ate(
            df=sample,
            treatment_col=treatment_col,
            outcome_col=outcome_col,
            covariate_cols=covariate_cols
        )

        effects.append(result["ate"])

    except Exception:
        continue

effects = np.array(effects)

return {
    "mean_bootstrap_ate": effects.mean(),
    "lower_95": np.percentile(effects, 2.5),
    "upper_95": np.percentile(effects, 97.5),
    "n_successful_bootstraps": len(effects)
}

```

Usage:

```

ci = bootstrap_logistic_ate(
    df=df,
    treatment_col="high_test_coverage",

```

```

outcome_col="failure",
covariate_cols=[
    "change_complexity",
    "priority_encoded",
    "assignment_group_failure_rate_90d",
    "environment_encoded"
]
)

print("95% CI in percentage points:")
print(ci["lower_95"] * 100, ci["upper_95"] * 100)

```

30. Chapter 7 summary

For CRP, the outcome is likely binary:

```

failure = 1
success = 0

```

So causal effects should usually be reported as changes in failure probability.

The simplest model is the Linear Probability Model:

```

failure ~ treatment + confounders

```

Its coefficient is easy to interpret as percentage-point change.

Logistic regression is better behaved for probabilities, but its coefficients are log-odds, not probability effects.

For business-facing causal recommendations, convert logistic results into average marginal effects:

```

mean predicted failure if everyone treated
-
mean predicted failure if everyone untreated

```

Always include uncertainty:

```

point estimate + confidence interval

```

For the CRP PoC, bootstrap confidence intervals are practical.

Be careful about:

- class imbalance
- rare failures
- categorical variables
- time-safe feature engineering
- high-cardinality assignment groups
- multiple testing
- overclaiming

31. Minimal vocabulary for Chapter 7

Term	Meaning
Binary outcome	Outcome with values 0/1
Linear Probability Model	Linear regression with binary outcome
Logistic regression	Binary outcome model using log-odds
Logit	$\log(p / (1-p))$
Odds	$p / (1-p)$
Odds ratio	Multiplicative change in odds
Marginal effect	Effect on predicted probability
Average marginal effect	Average probability-scale effect across rows
Percentage point	Absolute probability difference
Relative reduction	Percentage reduction relative to baseline
Bootstrap	Resampling method for uncertainty
Class imbalance	One outcome class is much rarer
Separation	Variable perfectly predicts binary outcome
Time leakage	Using future information in features

32. Practice questions

Q1

If:

$$P(\text{failure} \mid \text{do}(T=1)) = 0.09$$

$$P(\text{failure} \mid \text{do}(T=0)) = 0.15$$

what is the treatment effect?

Q2

Why is “risk reduced by 6%” ambiguous?

Q3

In a Linear Probability Model, what does this mean?

$$\beta_{\text{treatment}} = -0.04$$

Q4

Why should you not directly interpret a logistic regression coefficient as a probability change?

Q5

What is an average marginal effect?

Q6

Why might logistic regression fail when failures are very rare?

Q7

Why must assignment-group failure rate be computed using only past data?

33. Answers

A1

$$0.09 - 0.15 = -0.06$$

The treatment reduces failure probability by 6 percentage points.

A2

Because it could mean:

6 percentage points absolute reduction

or:

6% relative reduction

Those are different.

Example:

20% → 14%

is a 6 percentage-point reduction but a 30% relative reduction.

A3

It means the treatment is estimated to reduce the probability of failure by 4 percentage points, assuming the causal adjustment assumptions are valid.

A4

Because logistic regression coefficients are on the log-odds scale.

A coefficient of `-0.7` means log-odds decrease by 0.7, not that probability decreases by 70%.

A5

An average marginal effect is the average difference between predicted outcomes under treatment and no treatment.

For binary treatment:

$\text{mean}(P(Y=1 \mid T=1, X)) - \text{mean}(P(Y=1 \mid T=0, X))$

It reports the effect on the probability scale.

A6

Because rare failures can cause unstable estimates, very large coefficients, huge standard errors, or separation where some variables perfectly predict success/failure.

A7

Because using future failures leaks information from after the change.

For causal inference, confounders must be measured before treatment and outcome. A trailing 90-day failure rate is valid only if it uses data before each change request.

34. What remains

About **2-3 chapters left** before implementation:

Chapter 8: Sensitivity analysis, refuters, and robustness reporting

Chapter 9: Building the actual DoWhy CRP PoC pipeline

Chapter 10: What-if simulation and quantified recommendations

Chapter 8 is the last major theory/practice chapter before building the project pipeline.

5/29/2026, 10:34:08 AM



Chapter 8 - Sensitivity Analysis, Refuters, and Robustness Reporting

This chapter is the final major theory/practice chapter before building the CRP PoC pipeline.

You now know how to:

```
define treatment/outcome
construct a DAG
identify confounders
use DoWhy
estimate effects with regression/matching/IPW
handle binary outcomes
report probability-scale effects
```

Now you need to learn how to answer:

How much should we trust this causal estimate?

This is critical because the CRP proposal expects validation mechanisms such as placebo treatment tests, random common cause tests, data subset tests, sensitivity analysis, holdout validation, and confidence intervals. `Causal_Inference_Internship_Proposal.docx`

1. Why robustness matters

A causal estimate is not automatically credible just because the code runs.

Suppose your engine outputs:

```
High test coverage reduces failure probability by 5.2 percentage points.
95% CI: 2.0 to 8.1 percentage points.
```

That sounds useful.

But you still need to ask:

1. Does the estimate depend on one weird subset of data?
2. Would a fake treatment also show an effect?
3. Would adding irrelevant noise change the result?
4. Is there enough overlap between treated and untreated changes?

5. Could an unmeasured confounder explain the result?
6. Does the result hold under different estimators?
7. Does the result generalize to held-out data?

This chapter is about those checks.

2. Refuters vs sensitivity analysis

These terms are related but not identical.

Refuters

Refuters are practical stress tests.

They ask:

Does the estimated effect behave the way it should under artificial perturbations?

Examples:

placebo treatment
random common cause
data subset refuter
bootstrap refuter
dummy outcome refuter

DoWhy includes several built-in refuters.

Sensitivity analysis

Sensitivity analysis asks:

How badly would the causal assumptions need to fail for the conclusion to disappear?

Example:

How strong would an unmeasured confounder need to be to explain away the estimated effect?

Refuters test stability.

Sensitivity analysis tests vulnerability to assumption violations.

3. Important warning

Passing refuters does **not** prove causality.

Bad interpretation:

```
The placebo test passed, so the effect is causal.
```

Correct interpretation:

```
The estimate was stable under this placebo test, but causal validity still depends on the DAG and no-unmeasured-confounding assumption.
```

In causal inference, robustness checks increase credibility. They do not replace assumptions.

4. DoWhy refuter workflow

After estimating an effect:

```
identified_estimand = model.identify_effect()

estimate = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.linear_regression"
)
```

You run refuters:

```
refute = model.refute_estimate(
    identified_estimand,
    estimate,
    method_name="placebo_treatment_refuter"
)

print(refute)
```

General workflow:

1. Estimate causal effect.
2. Run several refuters.

3. Compare original estimate vs refuted estimate.
4. Check whether conclusion changes.
5. Report results clearly.

5. Placebo treatment refuter

The placebo treatment refuter replaces the real treatment with a fake/random treatment.

Original treatment:

```
high_test_coverage
```

Fake treatment:

```
randomly generated variable unrelated to failure
```

Expected result:

```
placebo effect  $\approx$  0
```

If the fake treatment produces a large effect, your estimation pipeline may be detecting spurious patterns.

6. Placebo example in DoWhy

```
placebo_refuter = model.refute_estimate(  
    identified_estimand,  
    estimate,  
    method_name="placebo_treatment_refuter"  
)  
  
print(placebo_refuter)
```

Possible good result:

```
Original estimate: -0.052  
Placebo estimate: 0.003  
p-value: high / not significant
```

Interpretation:

The fake treatment did not reproduce the observed effect, which supports-but does not prove-the robustness of the original estimate.

Possible bad result:

Original estimate: -0.052
Placebo estimate: -0.041

Interpretation:

A random placebo treatment produced a similar effect. This suggests the pipeline may be unstable, confounded, or overfitting.

7. Random common cause refuter

The random common cause refuter adds a random variable as a fake confounder.

Original adjustment set:

priority
change_complexity
assignment_group_failure_rate_90d
environment

Add fake variable:

random_noise

Expected result:

estimate should not change much

If adding random noise drastically changes the estimate, the model is unstable.

8. Random common cause example

```
random_common_cause_refuter = model.refute_estimate(  
    identified_estimand,  
    estimate,  
    method_name="random_common_cause"  
)  
  
print(random_common_cause_refuter)
```

Good result:

```
Original estimate: -0.052  
New estimate: -0.050
```

Bad result:

```
Original estimate: -0.052  
New estimate: -0.011
```

Interpretation of bad result:

The estimate is highly sensitive to irrelevant covariates. This may indicate weak signal, small sample size, poor overlap, or unstable model specification.

9. Data subset refuter

The data subset refuter repeatedly estimates the effect on random subsets of the data.

Expected result:

```
effect should remain broadly similar
```

Example:

```
subset_refuter = model.refute_estimate(  
    identified_estimand,  
    estimate,  
    method_name="data_subset_refuter"  
)
```

```
print(subset_refuter)
```

Good pattern:

```
Full data estimate: -0.052
Subset estimates: around -0.045 to -0.060
```

Bad pattern:

```
Subset 1: -0.080
Subset 2: +0.010
Subset 3: -0.020
Subset 4: +0.060
```

Interpretation:

The estimate is not stable across data subsets. Do not present it as a strong causal finding.

10. Bootstrap refuter / bootstrap CI

Bootstrap checks sampling uncertainty.

Process:

1. Resample data with replacement.
2. Re-estimate effect.
3. Repeat many times.
4. Inspect distribution of estimates.

If most bootstrap estimates are near the original estimate, stability is better.

If bootstrap estimates swing across zero or change direction often, the finding is uncertain.

11. Bootstrap interpretation

Suppose treatment:

```
high_test_coverage
```

Outcome:

failure

Bootstrap results:

Point estimate: -5.2 pp

95% CI: -8.1 to -2.0 pp

This is reasonably stable.

But:

Point estimate: -5.2 pp

95% CI: -12.5 to +3.8 pp

means:

The point estimate suggests reduced failure probability, but uncertainty is too high to make a strong claim.

12. Dummy outcome refuter

A dummy outcome refuter uses an outcome that treatment should not affect.

Example:

Treatment:

rollback_plan_present

Real outcome:

failure_after_deployment

Dummy outcome:

change_category

or:

pre_existing_assignment_group_failure_rate

Treatment should not causally affect variables that existed before treatment.

If your treatment appears to affect a pre-treatment variable, that is suspicious.

Example bad result:

```
rollback_plan appears to affect historical assignment-group failure rate
```

That is impossible causally, because historical group failure rate happened before rollback planning.

This suggests confounding or leakage.

13. Manual placebo/dummy outcome check

You can do this manually even without DoWhy.

```
fake_outcome = "assignment_group_failure_rate_90d"

result = logistic_ate(
    df=df,
    treatment_col="rollback_plan",
    outcome_col=fake_outcome,
    covariate_cols=[
        "change_complexity",
        "priority_encoded",
        "environment_encoded"
    ]
)
```

But note:

If the dummy outcome is continuous, use linear regression rather than logistic regression.

Expected:

```
effect ≈ 0
```

If not, treatment groups differ on pre-treatment variables even after adjustment.

14. Sensitivity to unobserved confounding

This is the most important real-world issue.

Your DAG may adjust for observed confounders:

```
priority  
change complexity  
assignment group history  
deployment environment
```

But there may be unobserved confounders:

```
team maturity  
true code complexity  
developer experience  
release pressure  
quality of tests  
architectural fragility  
managerial urgency
```

If these affect both treatment and failure, your estimate may be biased.

Example:

```
team_maturity → high_test_coverage  
team_maturity → lower_failure
```

If team maturity is unmeasured, then high test coverage may look protective partly because mature teams both test more and fail less.

15. Sensitivity question

A sensitivity analysis asks:

```
How strong would an unmeasured confounder need to be to eliminate the estimated  
effect?
```

Suppose estimate:

```
high test coverage reduces failure by 5.2 pp
```

Sensitivity analysis might say:

An unmeasured confounder would need to increase the probability of high test coverage by 3x
and independently reduce failure probability by 8 pp
to explain away the result.

Interpretation:

If such a confounder is plausible, the result is fragile.
If such a confounder is implausibly strong, the result is more credible.

16. DoWhy unobserved confounder refuter

DoWhy has a refuter for simulated unobserved confounding.

Example:

```
unobserved_confounder_refuter = model.refute_estimate(  
    identified_estimand,  
    estimate,  
    method_name="add_unobserved_common_cause",  
    confounders_effect_on_treatment="binary_flip",  
    confounders_effect_on_outcome="linear",  
    effect_strength_on_treatment=0.02,  
    effect_strength_on_outcome=0.02  
)  
  
print(unobserved_confounder_refuter)
```

Exact parameters depend on the DoWhy version and estimator. The concept is:

```
simulate hidden confounder  
vary its strength  
observe how estimate changes
```

17. Sensitivity grid

A useful practical approach:

Try multiple confounder strengths.
Record how the estimated effect changes.

Example table:

Hidden confounder strength	Estimated effect
None	-5.2 pp
Weak	-4.8 pp
Medium	-3.1 pp
Strong	-0.7 pp
Very strong	+1.5 pp

Interpretation:

The estimate remains negative under weak and medium simulated confounding, but disappears under strong confounding.

This is better than saying:

Passed sensitivity analysis.

Sensitivity is not pass/fail. It is a robustness profile.

18. Estimator triangulation

Do not rely on one estimator.

For one treatment, compare:

1. Naive difference
2. Linear Probability Model
3. Logistic regression marginal effect
4. Propensity score matching
5. Propensity score weighting

Example:

Method	Estimated effect
Naive difference	+3.0 pp
LPM adjusted	-4.7 pp
Logistic marginal effect	-5.1 pp
Propensity matching	-4.3 pp
IPW	-5.6 pp

This pattern is credible:

Naive result differs because of confounding.
Adjusted methods broadly agree.

Bad pattern:

Method	Estimated effect
LPM adjusted	-5.0 pp
Logistic marginal effect	+1.2 pp
Matching	-12.0 pp
IPW	+7.0 pp

Interpretation:

The finding is estimator-dependent. Investigate before reporting.

19. Overlap diagnostics are robustness checks

Overlap is not optional.

For each treatment, report:

```
minimum propensity score
maximum propensity score
treated/control overlap range
number of trimmed observations
weight distribution
```

Example:

```
Treatment: CAB involvement
```

```
Propensity scores:
```

```
treated: 0.42-0.99
```

```
untreated: 0.01-0.58
```

```
Overlap is weak.
```

```
Effect estimate should not be generalized to all changes.
```

For CRP, weak overlap is likely for process controls governed by policy rules.

Example:

```
production database changes may almost always require CAB approval
emergency hotfixes may almost always skip full test coverage
high-priority changes may almost always require extra approvals
```

20. Weight diagnostics for IPW

For IPW, inspect weights.

```
df["ipw_weight"].describe()
```

Look for:

```
very high max weight
large gap between 75th percentile and max
many weights above 10 or 20
```

Example problematic output:

```
mean: 2.1
75%: 2.8
max: 97.4
```

This means a few rows dominate the estimate.

Possible mitigations:

```
trim extreme propensity scores
use stabilized weights
cap weights
change estimand to overlap population
use matching instead
report limitation
```

21. Covariate balance robustness

After matching or weighting, check whether confounders are balanced.

Before weighting:

Covariate	SMD
complexity	0.72
priority	0.48
group history	0.65

After weighting:

Covariate	SMD
complexity	0.08
priority	0.05
group history	0.09

Good.

If after weighting:

Covariate	SMD
complexity	0.41
priority	0.33
group history	0.52

Bad.

Interpretation:

Weighting did not successfully balance treated and untreated groups. The effect estimate is not reliable.

22. Holdout validation

The proposal mentions holdout validation using an 80/20 train/test split.

Causal_Inference_Internship_Proposal.docx

For causal inference, holdout validation is not the same as predictive validation.

In prediction, you ask:

Does the model predict Y well on unseen data?

In causal inference, you ask:

Are estimated effects stable when learned on one subset and evaluated/re-estimated on another?

Useful checks:

1. Estimate effect on train split.
2. Estimate effect on test split.
3. Compare direction and magnitude.
4. Check overlap/balance separately.

Example:

Split	Estimated effect
Train	-5.4 pp
Test	-4.8 pp

Good.

Split	Estimated effect
Train	-5.4 pp
Test	+1.7 pp

Bad or uncertain.

23. Temporal validation

For CRP, temporal validation is often better than random split.

Random split:

random 80% train, random 20% test

Temporal split:

train on older changes
test on newer changes

Temporal split is more realistic because production systems change over time.

Example:

Train: January-September changes
Test: October-December changes

This checks whether the effect is stable over future-like data.

For operational systems, prefer temporal validation when possible.

24. Why temporal validation matters

Random splits can leak operational regimes.

Suppose the organization changed deployment policy in July.

Random split mixes pre-policy and post-policy data into both train and test.

Temporal split reveals whether your causal conclusions hold after process changes.

Possible issue:

test coverage had strong effect before CI/CD upgrade
test coverage effect shrank after CI/CD upgrade

This is not necessarily failure. It means the causal effect is context-dependent.

25. Robustness reporting table

A good robustness table:

Check	Result	Interpretation
Placebo treatment	effect ≈ 0	Passed
Random common cause	estimate changed by 0.3 pp	Stable
Data subset	estimates ranged -4.1 to -5.8 pp	Stable
Bootstrap CI	-8.1 to -2.0 pp	Excludes zero
Overlap	4% rows trimmed	Acceptable
Balance	all SMD < 0.1	Good
IPW weights	max weight 8.7	Acceptable
Temporal validation	same direction	Stable
Sensitivity	strong confounder needed to erase effect	Moderate robustness

This is the kind of table your PoC report should include.

26. Red flags

Do not trust the effect strongly if:

1. Confidence interval crosses zero widely.
2. Placebo treatment shows strong effect.
3. Data subset estimates change direction.
4. Propensity overlap is poor.
5. IPW weights are extreme.
6. Covariate balance remains poor.
7. Different estimators disagree strongly.
8. Treatment is not clearly pre-outcome.
9. Treatment is not actionable.
10. Important confounders are unobserved.
11. Sample has very few failures.
12. Missingness differs heavily by treatment/outcome.

Any one red flag does not always kill the analysis, but it should weaken the claim.

27. Green flags

A causal estimate is more credible when:

1. DAG is domain-reviewed.
2. Treatment is actionable and pre-outcome.
3. Outcome is clearly defined.
4. Confounders are measured before treatment.
5. Treated and untreated groups overlap.
6. Matching/weighting improves balance.
7. Multiple estimators agree.
8. Bootstrap CI is reasonably narrow.
9. Placebo treatment effect is near zero.
10. Data subset estimates are stable.
11. Temporal validation gives same direction.
12. Sensitivity analysis suggests hidden confounding would need to be strong.

Still not proof. But much more defensible.

28. How to report causal claims

Use calibrated language.

Weak evidence

The estimated effect was directionally negative, but the confidence interval included zero and subset refuters were unstable. This should be treated as inconclusive.

Moderate evidence

Under the stated DAG assumptions, high test coverage was associated with a 4.8 percentage-point reduction in failure probability. The estimate was stable across subset and placebo refuters, but sensitivity analysis indicates possible vulnerability to unmeasured team maturity.

Stronger evidence

Under the domain-reviewed DAG and observed historical data, high test coverage was estimated to reduce failure probability by 5.2 percentage points. The estimate was stable across LPM, logistic marginal effects, matching, and IPW; bootstrap CI excluded zero; overlap and balance diagnostics were acceptable; and sensitivity analysis suggested that only a strong unmeasured confounder would eliminate the effect.

This is the correct level of caution.

29. What the CRP engine should output internally

For each intervention, store:

```
treatment_name
outcome_name
estimand_type
adjustment_set
estimator
point_estimate
confidence_interval
sample_size
treated_count
control_count
failure_count
overlap_status
balance_status
```

```
refuter_results
sensitivity_summary
limitations
recommendation_status
```

Example:

```
{
  "treatment_name": "high_test_coverage",
  "outcome_name": "failure_48h",
  "estimand_type": "ATE",
  "adjustment_set": [
    "priority",
    "change_complexity",
    "assignment_group_failure_rate_90d",
    "environment"
  ],
  "estimator": "logistic_marginal_effect",
  "point_estimate_pp": -5.2,
  "ci_95_pp": [-8.1, -2.0],
  "sample_size": 12400,
  "treated_count": 5100,
  "control_count": 7300,
  "overlap_status": "good",
  "balance_status": "all_smd_below_0.1",
  "recommendation_status": "recommend_with_moderate_confidence"
}
```

This structure will make the final what-if/recommendation layer easier.

30. Recommendation status logic

Do not recommend purely based on effect size.

Use a decision rule.

Example:

```
Recommend if:
1. effect reduces failure
2. CI excludes or mostly excludes zero
3. overlap is acceptable
```

4. balance is acceptable
5. refuters pass
6. treatment is actionable

Possible statuses:

```
recommend
recommend_with_caution
inconclusive
do_not_recommend
insufficient_support
not_estimable_due_to_overlap
not_estimable_due_to_missingness
```

This matters because the proposal expects the causal engine to produce quantified recommendations and what-if simulation, but the system should degrade gracefully when causal analysis is not valid. `Causal_Inference_Internship_Proposal.docx`

31. Example output wording

Strong recommendation

High test coverage is recommended for comparable changes.

Estimated effect:

-5.2 percentage points on failure probability

95% CI:

-8.1 to -2.0 percentage points

Interpretation:

Under the stated DAG assumptions, increasing test coverage above the defined threshold is estimated to reduce failure probability by 5.2 percentage points.

Robustness:

The estimate was stable across logistic marginal effects, IPW, and subset refutation. Placebo treatment effect was near zero. Overlap and covariate balance were acceptable.

Cautious recommendation

Rollback planning shows a potentially protective effect, but evidence is uncertain.

Estimated effect:

-3.4 percentage points

95% CI:

-7.5 to +0.8 percentage points

Interpretation:

The point estimate suggests reduced failure probability, but the confidence interval includes zero. Recommendation should be treated as tentative.

Robustness:

Subset estimates were stable, but sensitivity analysis indicates vulnerability to unmeasured change complexity.

Not estimable

CAB involvement effect was not estimated for production database changes.

Reason:

Insufficient overlap. Nearly all comparable production database changes had CAB involvement, leaving no reliable untreated comparison group.

Recommendation:

Do not generate a causal what-if estimate for this subgroup. Fall back to policy guidance or LLM-only recommendation with a causal-analysis-unavailable note.

32. Practical robustness function skeleton

A basic Python design:

```
def run_robustness_suite(  
    model,  
    identified_estimand,  
    estimate,  
    treatment_name
```

```

):
    results = {}

    try:
        results["placebo"] = model.refute_estimate(
            identified_estimand,
            estimate,
            method_name="placebo_treatment_refuter"
        )
    except Exception as e:
        results["placebo"] = f"failed: {e}"

    try:
        results["random_common_cause"] = model.refute_estimate(
            identified_estimand,
            estimate,
            method_name="random_common_cause"
        )
    except Exception as e:
        results["random_common_cause"] = f"failed: {e}"

    try:
        results["data_subset"] = model.refute_estimate(
            identified_estimand,
            estimate,
            method_name="data_subset_refuter"
        )
    except Exception as e:
        results["data_subset"] = f"failed: {e}"

    return results

```

This should not be the final production design, but it is good enough for a PoC.

33. Robustness scoring idea

For the engine, you can assign a simple robustness grade.

Example:

A = strong support
B = moderate support
C = weak/inconclusive
D = not reliable
N/A = not estimable

Possible rule:

Condition	Grade
Good overlap, good balance, CI excludes zero, refuters stable, estimators agree	A
Mostly stable, minor uncertainty	B
CI includes zero or estimator disagreement	C
Poor overlap or failed placebo/subset checks	D
No valid comparison group	N/A

Use this internally to decide recommendation status.

But be careful: do not pretend this grade is mathematically absolute. It is a reporting heuristic.

34. Chapter 8 summary

Robustness analysis answers:

How much should we trust this causal estimate?

DoWhy refuters help test stability:

placebo treatment
random common cause
data subset

Sensitivity analysis asks:

Could unmeasured confounding explain away the effect?

For CRP, robustness is essential because process controls are assigned non-randomly, failures may be rare, and important confounders such as team maturity or true code complexity may be imperfectly measured.

A good causal recommendation should include:

point estimate
confidence interval
adjustment set
overlap diagnostics
balance diagnostics
refuter results
sensitivity summary
limitations

The correct output is not just:

Do X to reduce risk.

It is:

Under these assumptions, for comparable historical changes, doing X is estimated to reduce failure probability by Y percentage points, with this uncertainty and these robustness diagnostics.

35. Minimal vocabulary for Chapter 8

Term	Meaning
Refuter	Stress test for causal estimate
Placebo treatment	Fake treatment expected to have no effect
Random common cause	Random fake confounder added to test stability
Data subset refuter	Re-estimates effect on subsets of data
Sensitivity analysis	Tests vulnerability to assumption violations

Term	Meaning
Unobserved confounding	Hidden variable affects treatment and outcome
Estimator triangulation	Comparing results across multiple estimators
Overlap diagnostics	Checks whether treated/control groups are comparable
Weight diagnostics	Checks whether IPW is dominated by extreme weights
Covariate balance	Similarity of treated/control covariates after adjustment
Temporal validation	Testing stability on future-like data
Robustness grade	Heuristic confidence score for reporting

36. Practice questions

Q1

What does a placebo treatment refuter check?

Q2

If a fake random treatment produces a large effect, what does that suggest?

Q3

What does the random common cause refuter test?

Q4

Why does passing refuters not prove causality?

Q5

What is the main purpose of sensitivity analysis?

Q6

Why is temporal validation often better than random train/test split for CRP?

Q7

What does poor overlap imply?

Q8

How should the CRP engine respond when an effect is not estimable due to poor overlap?

37. Answers

A1

It replaces the real treatment with a fake/random treatment and checks whether the estimated effect disappears.

Expected result:

```
placebo effect  $\approx 0$ 
```

A2

It suggests the pipeline may be unstable, overfitting, confounded, or detecting spurious structure rather than a real causal effect.

A3

It adds an irrelevant random variable as a fake confounder and checks whether the estimate changes drastically.

If the estimate changes a lot, the model is unstable.

A4

Because refuters only test specific failure modes. They do not prove that the DAG is correct or that all confounders were measured.

A5

Sensitivity analysis checks how vulnerable the estimate is to violations of assumptions, especially unmeasured confounding.

It asks:

```
How strong would a hidden confounder need to be to erase the effect?
```

A6

Because CRP is an operational system that changes over time.

A temporal split tests whether effects estimated on older changes remain stable on newer changes. This is closer to real deployment than random splitting.

A7

Poor overlap means treated and untreated units are not comparable in some regions.

There may be no valid historical evidence for estimating the treatment effect for those types of changes.

A8

It should not produce a causal what-if estimate.

It should return something like:

```
not_estimable_due_to_overlap
```

and fall back to non-causal guidance or LLM-only recommendation with a clear note that causal analysis was unavailable.

38. What remains

About **2 chapters left** before implementation readiness:

```
Chapter 9: Building the actual DoWhy CRP PoC pipeline
```

```
Chapter 10: What-if simulation and quantified recommendations
```

Chapter 9 should be code-heavy and project-specific: dataset schema, preprocessing, DAG construction, DoWhy model creation, estimators, diagnostics, and output format.

5/29/2026, 10:43:56 AM



Chapter 9 - Building the Actual DoWhy CRP PoC Pipeline

This chapter turns everything into an implementable PoC.

The project proposal expects four connected pieces: data exploration and feature engineering, DAG construction, effect estimation/validation, and CRP integration design. It also specifies fields such as change metadata, test coverage, rollback plan, approval workflow, deployment context, outcome, and assignment-group history.

`Causal_Inference_Internship_Proposal.docx`

The goal of this chapter:

Build a minimal causal inference pipeline that estimates whether one operational intervention reduces change failure probability.

Do **not** start by estimating every intervention. Start with one treatment and build a clean pipeline around it.

1. Recommended first PoC question

Use this as your first causal question:

Does high test coverage reduce change failure probability?

Define:

Unit = one Change Request

Treatment:

high_test_coverage = 1 if test_coverage >= 80

high_test_coverage = 0 otherwise

Outcome:

failure = 1 if change failed or incident linked

failure = 0 otherwise

This is a good first treatment because:

1. It is actionable.
2. It is pre-deployment.
3. It is interpretable.
4. It appears directly in the proposal's example outputs.
5. It can support what-if simulation later.

The proposal explicitly lists test coverage %, rollback plan flag, deployment window, approver count, CAB involvement, change result, incident linkage, assignment-group failure rate, and change frequency as required causal-analysis fields.

Causal_Inference_Internship_Proposal.docx

2. Target pipeline architecture

The PoC pipeline should look like this:

```
raw_change_data
  ↓
data validation
  ↓
feature engineering
  ↓
treatment/outcome construction
  ↓
DAG definition
  ↓
DoWhy CausalModel
  ↓
identify estimand
  ↓
estimate effect
  ↓
bootstrap confidence interval
  ↓
propensity overlap/balance checks
  ↓
DoWhy refuters
  ↓
structured result object
```

↓

report / recommendation layer

This matches the proposal's expectation that the engine provides causal impact analysis, what-if scenarios, and quantified recommendations rather than replacing the existing ML risk model and LLM layer. `Causal_Inference_Internship_Proposal.docx`

3. Expected input schema

Your input dataframe should roughly contain:

```
required_columns = [  
    "change_id",  
    "change_type",  
    "priority",  
    "category",  
    "scheduled_start",  
    "scheduled_end",  
    "assigned_risk_score",  
    "test_coverage",  
    "rollback_plan_flag",  
    "approver_count",  
    "approval_duration_hours",  
    "cab_involvement_flag",  
    "deployment_window",  
    "environment",  
    "assignment_group",  
    "change_result",  
    "incident_linked",  
    "assignment_group_failure_rate_90d",  
    "change_frequency_90d"  
]
```

Not every real dataset will already contain all of these. Some will need to be derived.

For example:

```
deployment_window = derived from scheduled_start  
approval_duration_hours = approval_completed_at - approval_requested_at  
assignment_group_failure_rate_90d = failures in that group during prior 90 days
```

change_frequency_90d = number of prior changes by that group/service in prior 90 days

Important:

Features like `assignment_group_failure_rate_90d` must be computed using only historical data before the change. Otherwise, you leak future information.

4. Project folder structure

Use a simple structure:

```
crp_causal_poc/  
|  
├─ data/  
|   ├─ raw/  
|   └─ processed/  
|  
├─ notebooks/  
|   ├─ 01_data_profile.ipynb  
|   ├─ 02_dag_and_estimands.ipynb  
|   └─ 03_effect_estimation.ipynb  
|  
├─ src/  
|   ├─ config.py  
|   ├─ data_validation.py  
|   ├─ feature_engineering.py  
|   ├─ dag.py  
|   ├─ estimators.py  
|   ├─ diagnostics.py  
|   ├─ refuters.py  
|   └─ reporting.py  
|  
├─ outputs/  
|   ├─ figures/  
|   ├─ reports/  
|   └─ results/  
|  
└─ main.py
```

For the internship PoC, this is enough. Avoid over-engineering.

5. Install packages

```
pip install pandas numpy scikit-learn statsmodels dowhy matplotlib networkx
```

Optional but useful:

```
pip install pygraphviz graphviz
```

If Graphviz causes installation issues, skip visualization at first. DoWhy can still run without pretty DAG rendering.

6. Load data

```
import pandas as pd

df = pd.read_csv("data/raw/change_requests.csv")

print(df.shape)
print(df.head())
print(df.columns)
```

Immediately check:

```
print(df.info())
print(df.isna().mean().sort_values(ascending=False))
```

You need to know:

1. How many rows?
2. How many failures?
3. How much missingness?
4. Which fields are categorical?
5. Which fields are timestamps?
6. Which variables exist before deployment?

7. Basic data validation

Create a validation function:

```
def validate_required_columns(df, required_columns):
    missing = [col for col in required_columns if col not in df.columns]

    if missing:
        raise ValueError(f"Missing required columns: {missing}")

    return True
```

Usage:

```
validate_required_columns(df, required_columns)
```

Then validate values:

```
def basic_data_quality_report(df):
    report = {
        "n_rows": len(df),
        "n_columns": df.shape[1],
        "missing_rate": df.isna().mean().sort_values(ascending=False),
        "duplicate_change_ids": df["change_id"].duplicated().sum()
        if "change_id" in df.columns else None
    }

    return report
```

8. Construct outcome

Define `failure`.

Example:

```
import numpy as np

def construct_failure_outcome(df):
    df = df.copy()

    df["failure"] = np.where(
        (df["change_result"].str.lower().isin(["failed", "unsuccessful",
```

```

"failure"]))) |
    (df["incident_linked"] == 1),
    1,
    0
)

return df

```

Check base rate:

```

df = construct_failure_outcome(df)

failure_rate = df["failure"].mean()
print("Failure rate:", failure_rate)
print("Failure count:", df["failure"].sum())

```

Interpretation:

```
failure_rate = 0.08
```

means:

8% of changes failed.

If failure count is very low, for example below 50-100 failures, the causal estimates will be unstable.

9. Construct treatment

For test coverage:

```

def construct_high_test_coverage(df, threshold=80):
    df = df.copy()

    df["high_test_coverage"] = np.where(
        df["test_coverage"] >= threshold,
        1,
        0
    )

```

```
return df
```

Usage:

```
df = construct_high_test_coverage(df, threshold=80)

print(df["high_test_coverage"].value_counts(dropna=False))
print(df.groupby("high_test_coverage")["failure"].mean())
```

This gives the naive failure rate difference.

Remember:

```
naive difference ≠ causal effect
```

10. Construct useful features

Deployment window

```
def add_deployment_time_features(df):
    df = df.copy()

    df["scheduled_start"] = pd.to_datetime(df["scheduled_start"])

    df["deployment_hour"] = df["scheduled_start"].dt.hour
    df["deployment_dayofweek"] = df["scheduled_start"].dt.dayofweek

    df["weekend_deployment"] = np.where(
        df["deployment_dayofweek"].isin([5, 6]),
        1,
        0
    )

    df["off_hours_deployment"] = np.where(
        (df["deployment_hour"] < 9) | (df["deployment_hour"] >= 18),
        1,
        0
    )
```

```
return df
```

Change duration

```
def add_change_duration(df):  
    df = df.copy()  
  
    df["scheduled_start"] = pd.to_datetime(df["scheduled_start"])  
    df["scheduled_end"] = pd.to_datetime(df["scheduled_end"])  
  
    duration = df["scheduled_end"] - df["scheduled_start"]  
    df["scheduled_duration_hours"] = duration.dt.total_seconds() / 3600  
  
    return df
```

Rollback plan

```
def normalize_rollback_flag(df):  
    df = df.copy()  
  
    df["rollback_plan_present"] = np.where(  
        df["rollback_plan_flag"].isin([1, True, "Yes", "yes", "Y"]),  
        1,  
        0  
    )  
  
    return df
```

11. Choose confounders for first model

For:

```
high_test_coverage → failure
```

candidate confounders:

```
confounders = [  
    "priority",
```

```
"change_type",  
"category",  
"environment",  
"assignment_group_failure_rate_90d",  
"change_frequency_90d",  
"scheduled_duration_hours",  
"weekend_deployment",  
"off_hours_deployment"  
]
```

Why these?

Because they may affect both test coverage and failure.

Examples:

```
complex/high-priority changes may receive more testing and also fail more often  
production changes may have stricter test requirements and higher failure  
impact  
assignment groups with bad history may test differently and fail differently
```

Avoid controlling for variables that are downstream of test coverage.

For example, if better test coverage causes approval confidence or changes risk score, those may be mediators or descendants, depending on timing.

12. Preprocess categorical variables

DoWhy and statsmodels need numeric inputs.

Use one-hot encoding:

```
def encode_categorical_variables(df, categorical_cols):  
    df = df.copy()  
  
    df_encoded = pd.get_dummies(  
        df,  
        columns=categorical_cols,  
        drop_first=True,  
        dummy_na=True  
    )
```

```
return df_encoded
```

Usage:

```
categorical_cols = [  
    "priority",  
    "change_type",  
    "category",  
    "environment"  
]  
  
df_model = encode_categorical_variables(df, categorical_cols)
```

Do not one-hot encode `assignment_group` immediately if it has hundreds of values. Prefer:

```
assignment_group_failure_rate_90d  
change_frequency_90d
```

as lower-dimensional history features.

13. Final modeling columns

```
treatment = "high_test_coverage"  
outcome = "failure"  
  
base_confounders = [  
    "assignment_group_failure_rate_90d",  
    "change_frequency_90d",  
    "scheduled_duration_hours",  
    "weekend_deployment",  
    "off_hours_deployment"  
]  
  
encoded_confounders = [  
    col for col in df_model.columns  
    if col.startswith("priority_")  
    or col.startswith("change_type_")  
    or col.startswith("category_")  
    or col.startswith("environment_")
```

```
]
```

```
adjustment_set = base_confounders + encoded_confounders
```

Create clean modeling dataframe:

```
model_cols = [treatment, outcome] + adjustment_set
```

```
analysis_df = df_model[model_cols].dropna().copy()
```

```
print(analysis_df.shape)
```

```
print(analysis_df[treatment].value_counts())
```

```
print(analysis_df[outcome].value_counts())
```

14. Data sufficiency checks

Before DoWhy, run:

```
def check_data_sufficiency(df, treatment, outcome):
    treated_count = (df[treatment] == 1).sum()
    control_count = (df[treatment] == 0).sum()

    treated_failures = ((df[treatment] == 1) & (df[outcome] == 1)).sum()
    control_failures = ((df[treatment] == 0) & (df[outcome] == 1)).sum()

    return {
        "n": len(df),
        "treated_count": treated_count,
        "control_count": control_count,
        "treated_failures": treated_failures,
        "control_failures": control_failures,
        "overall_failure_rate": df[outcome].mean()
    }
```

Usage:

```
check_data_sufficiency(analysis_df, treatment, outcome)
```

Red flags:

```
treated_count too small
control_count too small
treated_failures near zero
control_failures near zero
failure rate extremely low
```

If one side has almost no failures, logistic regression may be unstable.

15. Build the DAG

For the encoded dataframe, DoWhy can use a graph with variable names.

A simple graph:

```
def build_dot_graph(treatment, outcome, confounders):
    lines = ["digraph {"]

    for c in confounders:
        lines.append(f'    "{c}" -> "{treatment}";')
        lines.append(f'    "{c}" -> "{outcome}";')

    lines.append(f'    "{treatment}" -> "{outcome}";')
    lines.append("}")

    return "\n".join(lines)
```

Usage:

```
graph = build_dot_graph(
    treatment=treatment,
    outcome=outcome,
    confounders=adjustment_set
)

print(graph)
```

This creates:

```
confounder → treatment
confounder → outcome
```

```
treatment → outcome
```

This is a generic backdoor-adjustment DAG.

For the first PoC, this is acceptable, but document that it is a simplified DAG.

The proposal expects a domain-driven DAG that is refined with dependency checks and expert feedback, so this initial graph should be treated as a starting point, not the final causal truth. `Causal_Inference_Internship_Proposal.docx`

16. Create DoWhy model

```
from dowhy import CausalModel

model = CausalModel(
    data=analysis_df,
    treatment=treatment,
    outcome=outcome,
    graph=graph
)
```

Identify effect:

```
identified_estimand = model.identify_effect()

print(identified_estimand)
```

Expected:

```
Dowhy should identify a backdoor estimand.
```

Conceptually:

```
ATE = E_X[E[Y | T=1, X] - E[Y | T=0, X]]
```

17. Estimate effect with DoWhy

Start with linear regression:

```
estimate_lr = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.linear_regression"
)

print(estimate_lr)
print("Estimated effect:", estimate_lr.value)
```

Because `failure` is binary, linear regression is a Linear Probability Model.

If:

```
estimate.value = -0.047
```

interpret as:

High test coverage is estimated to reduce failure probability by 4.7 percentage points, under the DAG assumptions.

18. Estimate with propensity score weighting

```
estimate_ipw = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_weighting"
)

print(estimate_ipw)
print("IPW estimate:", estimate_ipw.value)
```

This gives a second estimate.

Useful pattern:

```
linear regression estimate: -0.047
IPW estimate: -0.052
```

Concerning pattern:

```
linear regression estimate: -0.047
IPW estimate: +0.018
```

If methods disagree, do not report a strong conclusion yet.

19. Estimate with propensity score matching

```
estimate_matching = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_matching"
)

print(estimate_matching)
print("Matching estimate:", estimate_matching.value)
```

Matching may fail or become unstable if overlap is poor.

That itself is useful information.

20. Manual logistic marginal effect

DoWhy's built-in estimators are useful, but for binary outcomes you should also compute a logistic average marginal effect.

```
import statsmodels.api as sm
import numpy as np

def logistic_ate(df, treatment_col, outcome_col, covariate_cols):
    data = df[[treatment_col, outcome_col] + covariate_cols].dropna().copy()

    Y = data[outcome_col]
    X = sm.add_constant(data[[treatment_col] + covariate_cols],
        has_constant="add")

    model = sm.Logit(Y, X).fit(dis=False)

    X1 = X.copy()
    X1[treatment_col] = 1

    X0 = X.copy()
```

```

X0[treatment_col] = 0

p1 = model.predict(X1)
p0 = model.predict(X0)

ate = np.mean(p1 - p0)

return {
    "ate": ate,
    "ate_pp": ate * 100,
    "model": model,
    "n": len(data),
    "p_treated_mean": p1.mean(),
    "p_untreated_mean": p0.mean()
}

```

Usage:

```

logit_result = logistic_ate(
    df=analysis_df,
    treatment_col=treatment,
    outcome_col=outcome,
    covariate_cols=adjustment_set
)

print(logit_result["ate_pp"])
print(logit_result["p_treated_mean"])
print(logit_result["p_untreated_mean"])

```

Example output:

```

p_treated_mean = 0.074
p_untreated_mean = 0.126
ATE = -5.2 percentage points

```

Report:

High test coverage is estimated to reduce failure probability from 12.6% to 7.4%, an absolute reduction of 5.2 percentage points.

21. Bootstrap confidence interval

```
def bootstrap_logistic_ate(
    df,
    treatment_col,
    outcome_col,
    covariate_cols,
    n_bootstrap=500,
    seed=42
):
    rng = np.random.default_rng(seed)
    data = df[[treatment_col, outcome_col] + covariate_cols].dropna().copy()

    effects = []

    for _ in range(n_bootstrap):
        sample_idx = rng.choice(data.index, size=len(data), replace=True)
        sample = data.loc[sample_idx]

        try:
            result = logistic_ate(
                df=sample,
                treatment_col=treatment_col,
                outcome_col=outcome_col,
                covariate_cols=covariate_cols
            )
            effects.append(result["ate"])
        except Exception:
            continue

    effects = np.array(effects)

    return {
        "point_estimate": logistic_ate(
            data,
            treatment_col,
            outcome_col,
            covariate_cols
        )["ate"],
        "lower_95": np.percentile(effects, 2.5),
        "upper_95": np.percentile(effects, 97.5),
    }
```

```
    "n_successful_bootstraps": len(effects)
}
```

Usage:

```
ci_result = bootstrap_logistic_ate(
    df=analysis_df,
    treatment_col=treatment,
    outcome_col=outcome,
    covariate_cols=adjustment_set,
    n_bootstrap=500
)

print(ci_result)
```

Convert to percentage points:

```
print("ATE pp:", ci_result["point_estimate"] * 100)
print("95% CI pp:", ci_result["lower_95"] * 100, ci_result["upper_95"] * 100)
```

22. Propensity score overlap

```
from sklearn.linear_model import LogisticRegression

def add_propensity_scores(df, treatment_col, covariate_cols):
    data = df[[treatment_col] + covariate_cols].dropna().copy()

    X = data[covariate_cols]
    T = data[treatment_col]

    ps_model = LogisticRegression(max_iter=1000)
    ps_model.fit(X, T)

    data["propensity_score"] = ps_model.predict_proba(X)[:, 1]

    return data, ps_model
```

Usage:

```
ps_df, ps_model = add_propensity_scores(
    analysis_df,
    treatment_col=treatment,
    covariate_cols=adjustment_set
)

print(ps_df.groupby(treatment)["propensity_score"].describe())
```

Interpretation:

If treated scores are mostly 0.8-1.0 and control scores are mostly 0.0-0.2, overlap is poor.

23. Plot overlap

```
import matplotlib.pyplot as plt

def plot_propensity_overlap(ps_df, treatment_col):
    treated = ps_df[ps_df[treatment_col] == 1]["propensity_score"]
    control = ps_df[ps_df[treatment_col] == 0]["propensity_score"]

    plt.figure()
    plt.hist(treated, alpha=0.5, label="Treated", bins=30)
    plt.hist(control, alpha=0.5, label="Control", bins=30)
    plt.xlabel("Propensity score")
    plt.ylabel("Count")
    plt.legend()
    plt.title("Propensity Score Overlap")
    plt.show()
```

Usage:

```
plot_propensity_overlap(ps_df, treatment)
```

For the PoC report, save it:

```
plt.savefig("outputs/figures/high_test_coverage_overlap.png", dpi=200)
```

24. Covariate balance with SMD

```
def standardized_mean_difference(df, treatment_col, covariate_col,
weight_col=None):
    treated = df[df[treatment_col] == 1]
    control = df[df[treatment_col] == 0]

    if weight_col is None:
        mean_t = treated[covariate_col].mean()
        mean_c = control[covariate_col].mean()

        var_t = treated[covariate_col].var()
        var_c = control[covariate_col].var()
    else:
        mean_t = np.average(treated[covariate_col],
weights=treated[weight_col])
        mean_c = np.average(control[covariate_col],
weights=control[weight_col])

        var_t = np.average(
            (treated[covariate_col] - mean_t) ** 2,
            weights=treated[weight_col]
        )
        var_c = np.average(
            (control[covariate_col] - mean_c) ** 2,
            weights=control[weight_col]
        )

    pooled_sd = np.sqrt((var_t + var_c) / 2)

    if pooled_sd == 0 or np.isnan(pooled_sd):
        return 0

    return (mean_t - mean_c) / pooled_sd
```

Compute unweighted balance:

```
balance_rows = []

for col in adjustment_set:
    smd = standardized_mean_difference(
```

```

        analysis_df,
        treatment_col=treatment,
        covariate_col=col
    )

    balance_rows.append({
        "covariate": col,
        "smd_unweighted": smd
    })

balance_df = pd.DataFrame(balance_rows)
print(balance_df.sort_values("smd_unweighted", key=abs,
ascending=False).head(20))

```

25. Add IPW weights and weighted balance

```

def add_ipw_weights(ps_df, treatment_col):
    df = ps_df.copy()

    eps = 1e-6
    ps = df["propensity_score"].clip(eps, 1 - eps)

    df["ipw_weight"] = np.where(
        df[treatment_col] == 1,
        1 / ps,
        1 / (1 - ps)
    )

    return df

```

Usage:

```

weighted_df = add_ipw_weights(ps_df, treatment)

print(weighted_df["ipw_weight"].describe())

```

Weighted SMD:

```

weighted_balance_rows = []

```

```

for col in adjustment_set:
    smd_weighted = standardized_mean_difference(
        weighted_df,
        treatment_col=treatment,
        covariate_col=col,
        weight_col="ipw_weight"
    )

    weighted_balance_rows.append({
        "covariate": col,
        "smd_weighted": smd_weighted
    })

weighted_balance_df = pd.DataFrame(weighted_balance_rows)

balance_report = balance_df.merge(
    weighted_balance_df,
    on="covariate",
    how="left"
)

print(balance_report.sort_values("smd_weighted", key=abs,
    ascending=False).head(20))

```

Good result:

```
most |SMD| values below 0.1 after weighting
```

Bad result:

```
many |SMD| values remain above 0.2
```

26. Run DoWhy refuters

```

def run_dowhy_refuters(model, identified_estimand, estimate):
    results = {}

    refuters = [
        "placebo_treatment_refuter",
        "random_common_cause",

```

```

        "data_subset_refuter"
    ]

    for refuter in refuters:
        try:
            results[refuter] = model.refute_estimate(
                identified_estimand,
                estimate,
                method_name=refuter
            )
        except Exception as e:
            results[refuter] = f"FAILED: {e}"

    return results

```

Usage:

```

refuter_results = run_dowhy_refuters(
    model,
    identified_estimand,
    estimate_lr
)

for name, result in refuter_results.items():
    print("\n", name)
    print(result)

```

The proposal explicitly expects placebo treatment, random common cause, data subset tests, sensitivity analysis, and holdout validation as validation mechanisms.

Causal_Inference_Internship_Proposal.docx

27. Create structured result object

Do not just print numbers. Build a result object.

```

def build_result_object(
    treatment,
    outcome,
    adjustment_set,
    sufficiency,
    dowhy_estimates,

```

```

logit_result,
ci_result,
overlap_summary,
balance_summary,
refuter_results
):
    return {
        "treatment": treatment,
        "outcome": outcome,
        "estimand": "ATE",
        "adjustment_set": adjustment_set,

        "data": sufficiency,

        "estimates": dowhy_estimates,

        "main_effect": {
            "method": "logistic_average_marginal_effect",
            "ate": logit_result["ate"],
            "ate_percentage_points": logit_result["ate_pp"],
            "p_failure_if_treated": logit_result["p_treated_mean"],
            "p_failure_if_untreated": logit_result["p_untreated_mean"],
            "ci_95": {
                "lower": ci_result["lower_95"],
                "upper": ci_result["upper_95"],
                "lower_percentage_points": ci_result["lower_95"] * 100,
                "upper_percentage_points": ci_result["upper_95"] * 100
            }
        },

        "diagnostics": {
            "overlap": overlap_summary,
            "balance": balance_summary,
            "refuters": {k: str(v) for k, v in refuter_results.items()}
        }
    }

```

28. Summarize overlap

```
def summarize_overlap(ps_df, treatment_col):
    treated = ps_df[ps_df[treatment_col] == 1]["propensity_score"]
    control = ps_df[ps_df[treatment_col] == 0]["propensity_score"]

    common_min = max(treated.min(), control.min())
    common_max = min(treated.max(), control.max())

    outside_common_support = (
        (ps_df["propensity_score"] < common_min) |
        (ps_df["propensity_score"] > common_max)
    ).mean()

    return {
        "treated_min": treated.min(),
        "treated_max": treated.max(),
        "control_min": control.min(),
        "control_max": control.max(),
        "common_support_min": common_min,
        "common_support_max": common_max,
        "share_outside_common_support": outside_common_support
    }
```

Usage:

```
overlap_summary = summarize_overlap(ps_df, treatment)
print(overlap_summary)
```

29. Summarize balance

```
def summarize_balance(balance_report):
    max_abs_unweighted = balance_report["smd_unweighted"].abs().max()
    max_abs_weighted = balance_report["smd_weighted"].abs().max()

    share_balanced_weighted = (
        balance_report["smd_weighted"].abs() < 0.1
    ).mean()

    return {
        "max_abs_smd_unweighted": max_abs_unweighted,
```

```

        "max_abs_smd_weighted": max_abs_weighted,
        "share_covariates_balanced_weighted_smd_lt_0_1":
share_balanced_weighted
    }

```

Usage:

```

balance_summary = summarize_balance(balance_report)
print(balance_summary)

```

30. Define recommendation status

```

def assign_recommendation_status(result):
    effect = result["main_effect"]["ate_percentage_points"]
    ci_lower = result["main_effect"]["ci_95"]["lower_percentage_points"]
    ci_upper = result["main_effect"]["ci_95"]["upper_percentage_points"]

    overlap_bad = (
        result["diagnostics"]["overlap"]["share_outside_common_support"] > 0.2
    )

    balance_bad = (
        result["diagnostics"]["balance"]
["share_covariates_balanced_weighted_smd_lt_0_1"] < 0.8
    )

    if overlap_bad:
        return "not_estimable_due_to_poor_overlap"

    if balance_bad:
        return "inconclusive_due_to_poor_balance"

    if effect < 0 and ci_upper < 0:
        return "recommend"

    if effect < 0 and ci_lower < 0 < ci_upper:
        return "recommend_with_caution"

    if ci_lower <= 0 <= ci_upper:
        return "inconclusive"

```

```

if effect > 0 and ci_lower > 0:
    return "do_not_recommend_potential_harm"

return "inconclusive"

```

Usage:

```
result["recommendation_status"] = assign_recommendation_status(result)
```

31. Generate human-readable summary

```

def generate_summary(result):
    treatment = result["treatment"]
    outcome = result["outcome"]

    effect = result["main_effect"]["ate_percentage_points"]
    lower = result["main_effect"]["ci_95"]["lower_percentage_points"]
    upper = result["main_effect"]["ci_95"]["upper_percentage_points"]

    p1 = result["main_effect"]["p_failure_if_treated"] * 100
    p0 = result["main_effect"]["p_failure_if_untreated"] * 100

    status = result["recommendation_status"]

    return f"""
Treatment: {treatment}
Outcome: {outcome}

Estimated effect:
{effect:.2f} percentage points

95% CI:
{lower:.2f} to {upper:.2f} percentage points

Predicted failure probability:
If treated: {p1:.2f}%
If untreated: {p0:.2f}%

Recommendation status:

```

```
{status}
```

Interpretation:

Under the stated DAG assumptions and observed historical data, {treatment} is estimated to change {outcome} probability by {effect:.2f} percentage points.

Adjustment set:

```
{"", ".join(result["adjustment_set"])]}"
"""
```

32. Save results

```
import json
from pathlib import Path

Path("outputs/results").mkdir(parents=True, exist_ok=True)

with open("outputs/results/high_test_coverage_result.json", "w") as f:
    json.dump(result, f, indent=2)
```

Save summary:

```
summary = generate_summary(result)

with open("outputs/reports/high_test_coverage_summary.txt", "w") as f:
    f.write(summary)
```

33. Full main pipeline skeleton

```
def run_crp_causal_poc(df):
    # 1. Outcome and treatment
    df = construct_failure_outcome(df)
    df = construct_high_test_coverage(df, threshold=80)

    # 2. Feature engineering
    df = add_deployment_time_features(df)
    df = add_change_duration(df)
    df = normalize_rollback_flag(df)
```

```

# 3. Encode categoricals
categorical_cols = [
    "priority",
    "change_type",
    "category",
    "environment"
]

df_model = encode_categorical_variables(df, categorical_cols)

# 4. Define columns
treatment = "high_test_coverage"
outcome = "failure"

base_confounders = [
    "assignment_group_failure_rate_90d",
    "change_frequency_90d",
    "scheduled_duration_hours",
    "weekend_deployment",
    "off_hours_deployment"
]

encoded_confounders = [
    col for col in df_model.columns
    if col.startswith("priority_")
    or col.startswith("change_type_")
    or col.startswith("category_")
    or col.startswith("environment_")
]

adjustment_set = base_confounders + encoded_confounders

model_cols = [treatment, outcome] + adjustment_set
analysis_df = df_model[model_cols].dropna().copy()

# 5. Data checks
sufficiency = check_data_sufficiency(
    analysis_df,
    treatment,
    outcome

```

```

)

# 6. DAG
graph = build_dot_graph(
    treatment=treatment,
    outcome=outcome,
    confounders=adjustment_set
)

# 7. Dowhy model
model = CausalModel(
    data=analysis_df,
    treatment=treatment,
    outcome=outcome,
    graph=graph
)

identified_estimand = model.identify_effect()

estimate_lr = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.linear_regression"
)

estimate_ipw = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_weighting"
)

estimate_matching = model.estimate_effect(
    identified_estimand,
    method_name="backdoor.propensity_score_matching"
)

dowhy_estimates = {
    "linear_regression": estimate_lr.value,
    "ipw": estimate_ipw.value,
    "matching": estimate_matching.value
}

# 8. Logistic marginal effect

```

```

logit_result = logistic_ate(
    analysis_df,
    treatment,
    outcome,
    adjustment_set
)

ci_result = bootstrap_logistic_ate(
    analysis_df,
    treatment,
    outcome,
    adjustment_set,
    n_bootstrap=500
)

# 9. Propensity diagnostics
ps_df, ps_model = add_propensity_scores(
    analysis_df,
    treatment,
    adjustment_set
)

overlap_summary = summarize_overlap(ps_df, treatment)

weighted_df = add_ipw_weights(ps_df, treatment)

balance_rows = []

for col in adjustment_set:
    balance_rows.append({
        "covariate": col,
        "smd_unweighted": standardized_mean_difference(
            analysis_df,
            treatment,
            col
        ),
        "smd_weighted": standardized_mean_difference(
            weighted_df,
            treatment,
            col,
            weight_col="ipw_weight"
    )

```

```

    )
    })

balance_report = pd.DataFrame(balance_rows)
balance_summary = summarize_balance(balance_report)

# 10. Refuters
refuter_results = run_dowhy_refuters(
    model,
    identified_estimand,
    estimate_lr
)

# 11. Result object
result = build_result_object(
    treatment=treatment,
    outcome=outcome,
    adjustment_set=adjustment_set,
    sufficiency=sufficiency,
    dowhy_estimates=dowhy_estimates,
    logit_result=logit_result,
    ci_result=ci_result,
    overlap_summary=overlap_summary,
    balance_summary=balance_summary,
    refuter_results=refuter_results
)

result["recommendation_status"] = assign_recommendation_status(result)
result["summary"] = generate_summary(result)

return result

```

Usage:

```

df = pd.read_csv("data/raw/change_requests.csv")

result = run_crp_causal_poc(df)

print(result["summary"])

```

34. What your PoC report should include

Your first PoC report should have these sections:

1. Causal question
2. Treatment definition
3. Outcome definition
4. Dataset description
5. Feature engineering
6. DAG and assumptions
7. Adjustment set
8. Naive comparison
9. Causal estimates from multiple estimators
10. Confidence interval
11. Overlap diagnostics
12. Covariate balance diagnostics
13. Refutation tests
14. Limitations
15. Recommendation status

This directly supports the proposal's deliverables: validated DAG, prioritized controllable interventions, quantified effect estimates, statistical confidence, what-if demonstration, and final documentation. `Causal_Inference_Internship_Proposal.docx`

35. Example final PoC output

Causal Question:

Does high test coverage reduce change failure probability?

Treatment:

`high_test_coverage = 1 if test_coverage >= 80%`

Outcome:

`failure = 1 if change failed or incident was linked within the defined window`

Naive Difference:

-2.1 percentage points

Main Causal Estimate:

-5.2 percentage points

95% Bootstrap CI:

-8.1 to -2.0 percentage points

Predicted Failure Probability:

If high test coverage: 7.4%

If low test coverage: 12.6%

Adjustment Set:

priority, change type, category, environment, assignment group failure rate
trailing 90 days,
change frequency trailing 90 days, scheduled duration, deployment timing

Diagnostics:

Overlap acceptable; 3.8% outside common support.

Weighted balance acceptable; 91% of covariates have $|SMD| < 0.1$.

Placebo refuter: stable.

Random common cause refuter: stable.

Data subset refuter: stable.

Recommendation:

Recommend.

Interpretation:

Under the stated DAG assumptions and observed historical data, high test coverage is estimated to reduce change failure probability by 5.2 percentage points for comparable historical changes.

36. Common implementation mistakes

Mistake 1: Using the ML risk score as a confounder without thinking

If `assigned_risk_score` is computed from treatment variables like test coverage, rollback plan, or approval patterns, controlling for it can block causal paths or introduce post-treatment bias.

Use it only if you know its timing and feature inputs.

Mistake 2: Treating all variables as pre-treatment

Variables recorded after deployment should not be used as confounders.

Examples to avoid:

```
actual rollback occurred  
post-deployment incident count  
post-change review notes  
resolution time
```

Mistake 3: Estimating too many treatments at once

Start with:

```
high_test_coverage
```

Then add:

```
rollback_plan_present
```

Then maybe:

```
off_hours_deployment  
CAB_involvement
```

Mistake 4: Ignoring overlap

If all comparable changes are treated, you cannot estimate the counterfactual from historical data.

Mistake 5: Reporting relative percent without baseline

Bad:

```
risk reduced by 22%
```

Good:

```
failure probability reduced from 12.6% to 7.4%, an absolute reduction of 5.2  
percentage points, equivalent to a 41.3% relative reduction.
```

37. Minimal vocabulary for Chapter 9

Term	Meaning
Pipeline	Ordered process from raw data to result
Schema	Expected dataframe columns and types
Feature engineering	Creating usable variables from raw fields
Treatment construction	Converting raw field into intervention variable
Outcome construction	Defining success/failure label
Adjustment set	Confounders controlled for in estimation
DoWhy CausalModel	Object containing data, treatment, outcome, graph
Estimand	Causal quantity identified from DAG
Estimator	Statistical method used to compute effect
Result object	Structured output containing estimates and diagnostics
Recommendation status	Decision label based on effect and robustness

38. Practice questions

Q1

Why should the first PoC estimate only one treatment?

Q2

Why is `high_test_coverage` a reasonable first treatment?

Q3

What is the likely outcome variable in the CRP project?

Q4

Why should `assignment_group_failure_rate_90d` be computed only from past data?

Q5

Why might controlling for `assigned_risk_score` be dangerous?

Q6

What should the result object contain besides the point estimate?

Q7

What should the system do if overlap is poor?

39. Answers

A1

Because the first goal is to validate the end-to-end causal workflow: treatment definition, outcome definition, DAG, estimation, diagnostics, refuters, and reporting. Estimating many treatments immediately makes debugging and interpretation harder.

A2

Because test coverage is actionable, pre-deployment, interpretable, and directly relevant to reducing deployment risk. It also appears in the proposal's example causal recommendation.

A3

The likely outcome is:

```
failure = 1 if the change failed or had incident linkage
failure = 0 otherwise
```

A4

Because using future failures leaks information from after the change. That would make the feature invalid for both prediction and causal inference.

A5

Because the risk score may be downstream of causal variables like test coverage, rollback readiness, approval pattern, or other process controls. Controlling for it may block part of the causal effect or introduce post-treatment bias.

A6

It should include:

```
treatment
outcome
adjustment set
point estimate
confidence interval
sample size
treated/control counts
overlap diagnostics
balance diagnostics
refuter results
limitations
recommendation status
```

A7

It should not produce a strong causal estimate. It should return something like:

```
not_estimable_due_to_poor_overlap
```

and fall back to non-causal guidance or LLM-only recommendation with a clear note.

40. What remains

Only **one core chapter** remains:

```
Chapter 10: What-if simulation and quantified recommendations
```

After Chapter 10, you will have the minimum conceptual and implementation foundation to start building the actual CRP causal inference PoC.

5/29/2026, 10:45:22 AM



Chapter 10 - What-If Simulation and Quantified Recommendations

This is the final core chapter before you can start the CRP causal inference PoC.

So far, you can estimate effects like:

```
high_test_coverage → failure
rollback_plan_present → failure
off_hours_deployment → failure
```

But the project's actual value is not just estimating effects. The proposal wants a Causal Inference Engine that can provide:

```
WHY the change is risky
HOW MUCH an intervention may reduce risk
WHAT-IF simulation before deployment
quantified recommendations
```

That aligns directly with the proposal's stated goal: augmenting CRP with causal explanation, quantified intervention impact, and what-if scenario modelling.

`Causal_Inference_Internship_Proposal.docx`

1. What “what-if” means in causal inference

A predictive what-if asks:

If I change this input in the ML model, how does the prediction change?

A causal what-if asks:

If we intervened in the real process and changed this factor, how would the outcome probability change?

Those are not the same.

Example:

Prediction:

Change `test_coverage` from 45% to 80% in the ML model input.

Causal inference:

Estimate failure probability under `do(test_coverage = 80%)` versus `do(test_coverage = 45%)`.

The causal version requires assumptions about confounding, treatment timing, and valid adjustment.

2. Prediction what-if vs causal what-if

Suppose the ML model says:

Current predicted failure risk = 18%

If test coverage is changed to 80%, predicted risk = 10%

This does **not** automatically mean:

Increasing test coverage causes an 8 percentage-point reduction.

Why?

Because the ML model may use associations.

Maybe teams with high test coverage are more mature. Maybe their lower failure rate comes partly from team maturity, not test coverage itself.

A causal what-if tries to isolate:

effect of changing test coverage itself

from:

differences between teams that usually have high vs low test coverage

3. Basic intervention notation

Causal interventions are written using `do(...)`.

Example:

```
P(failure | do(high_test_coverage = 1))
```

This means:

The probability of failure if we force high test coverage, rather than merely observe it.

Compare:

```
P(failure | high_test_coverage = 1)
```

This means:

The probability of failure among changes that happened to have high test coverage.

The difference matters.

Observed condition:

```
P(Y | T = 1)
```

Intervention:

```
P(Y | do(T = 1))
```

Causal inference is about estimating the second one.

4. What the CRP engine should simulate

For a candidate change request, the engine might simulate:

Current state:

test_coverage = 45%

rollback_plan_present = 0

deployment_window = off-hours

approver_count = 1

Candidate interventions:

1. Increase test coverage to ≥80%

2. Add rollback plan

3. Move deployment to normal business hours
4. Require CAB review
5. Increase approval depth

Output:

Baseline estimated failure probability: 18.0%

What-if interventions:

- high_test_coverage: estimated risk becomes 12.8%
- rollback_plan_present: estimated risk becomes 14.5%
- normal_hours_deployment: estimated risk becomes 16.2%

Best supported intervention:

increase test coverage to $\geq 80\%$

Estimated absolute reduction:

5.2 percentage points

95% CI:

2.0 to 8.1 percentage points

The proposal specifically wants outputs like quantified intervention impact and scenario modelling before deployment, so this is the point where causal estimation becomes product functionality. `Causal_Inference_Internship_Proposal.docx`

5. The simplest what-if simulation

Assume you already estimated:

Treatment: high_test_coverage

Outcome: failure

ATE: -5.2 percentage points

95% CI: -8.1 to -2.0 percentage points

For a change request with current ML risk:

baseline_failure_probability = 18%

Then a simple causal recommendation says:

If high test coverage is achieved, estimated failure probability becomes:

$$18\% - 5.2\% = 12.8\%$$

This is simple, but there is a caveat.

The ATE is an average effect across the studied population. It may not be the exact effect for this specific change.

Better wording:

For comparable historical changes, high test coverage was estimated to reduce failure probability by 5.2 percentage points on average.

6. Individualized what-if simulation

A better approach uses the fitted outcome model to compute this specific row's predicted counterfactual probabilities.

For logistic regression:

$$\begin{aligned} &P(\text{failure} \mid \text{do}(T=1), X=x_i) \\ &- \\ &P(\text{failure} \mid \text{do}(T=0), X=x_i) \end{aligned}$$

For one change request **i**:

x_i = its priority, type, category, environment, group history, deployment timing, etc.

Then estimate:

failure probability if treated
failure probability if untreated

This produces a row-level what-if.

7. Row-level logistic what-if function

Assume you trained a logistic model with:

```
failure ~ high_test_coverage + confounders
```

Function:

```
def simulate_binary_intervention_logit(
    fitted_model,
    row_df,
    treatment_col,
    treatment_value
):
    """
    Simulate predicted failure probability for one row
    under a specified treatment value.

    row_df must contain the same columns used in the fitted model,
    including the constant column if the model was trained with one.
    """
    row_cf = row_df.copy()
    row_cf[treatment_col] = treatment_value

    probability = fitted_model.predict(row_cf)[0]

    return probability
```

Usage:

```
# row_model is one encoded row with same columns as training X
p_if_high_coverage = simulate_binary_intervention_logit(
    fitted_model=logit_result["model"],
    row_df=row_model,
    treatment_col="high_test_coverage",
    treatment_value=1
)

p_if_low_coverage = simulate_binary_intervention_logit(
    fitted_model=logit_result["model"],
    row_df=row_model,
    treatment_col="high_test_coverage",
    treatment_value=0
)
```

```
individual_effect = p_if_high_coverage - p_if_low_coverage

print(p_if_high_coverage, p_if_low_coverage, individual_effect)
```

Interpretation:

For this change's observed covariates, setting high_test_coverage=1 changes predicted failure probability by individual_effect.

Do not overstate this as true ITE. It is a model-based conditional effect.

8. Why row-level estimates need caution

Individual-level causal estimates are less reliable than average effects.

Why?

1. The model may not capture all treatment heterogeneity.
2. The row may be outside the strong-overlap region.
3. The intervention may be unrealistic for that change.
4. Unmeasured confounding is worse at fine granularity.
5. Confidence intervals are harder for individual predictions.

So for the PoC, report row-level what-if as:

```
model-based scenario estimate
```

not:

```
guaranteed individual causal effect
```

Better wording:

For changes with similar observed characteristics, this intervention is estimated to reduce failure probability by approximately X percentage points.

9. Recommendation engine logic

The causal recommendation layer should not simply pick the biggest effect.

It should consider:

effect size
confidence interval
overlap quality
balance quality
refuter results
actionability
current value of treatment
business feasibility
possible side effects

Example:

Intervention	Effect	CI	Overlap	Actionable?	Status
High test coverage	-5.2 pp	-8.1 to -2.0	Good	Yes	Recommend
Rollback plan	-3.4 pp	-7.5 to +0.8	Good	Yes	Caution
CAB involvement	-6.8 pp	-14.0 to +2.0	Poor	Sometimes	Not estimable
Off-hours deployment	+2.1 pp	-1.2 to +5.8	Moderate	Yes	Do not recommend

The recommendation should favor interventions that are both effective and well-supported.

10. Recommendation status categories

Use a small controlled vocabulary.

recommend
recommend_with_caution
inconclusive
do_not_recommend
not_estimable_due_to_overlap
not_estimable_due_to_missingness

```
not_actionable
insufficient_sample_size
```

This prevents the system from hallucinating causal advice when evidence is weak.

Example:

```
CAB_involvement:
not_estimable_due_to_overlap
```

Reason:

```
Nearly all comparable high-priority production changes already had CAB
involvement, leaving no reliable untreated comparison group.
```

That is a better system behavior than forcing a fake estimate.

11. Structured intervention result

Each intervention should have a structured result.

```
{
  "intervention": "high_test_coverage",
  "current_value": 0,
  "proposed_value": 1,
  "outcome": "failure_48h",
  "baseline_probability": 0.18,
  "simulated_probability": 0.128,
  "absolute_risk_reduction_pp": 5.2,
  "relative_risk_reduction_percent": 28.9,
  "ci_95_pp": [2.0, 8.1],
  "recommendation_status": "recommend",
  "evidence_grade": "A",
  "assumptions": [
    "No unmeasured confounding after adjustment",
    "Sufficient overlap for comparable changes",
    "Treatment is measured before deployment",
    "Failure label is reliable"
  ],
  "limitations": [
    "Team maturity may be imperfectly measured",
```

```
    "Effect is estimated from historical data"
  ]
}
```

This format is suitable for a downstream API or UI.

12. Absolute vs relative reduction

Always compute both, but prioritize absolute reduction.

If:

```
baseline risk = 18.0%
simulated risk = 12.8%
```

Then:

```
absolute reduction = 18.0 - 12.8 = 5.2 percentage points
relative reduction = 5.2 / 18.0 = 28.9%
```

Python:

```
def compute_risk_reduction(baseline_prob, simulated_prob):
    absolute_reduction = baseline_prob - simulated_prob

    if baseline_prob > 0:
        relative_reduction = absolute_reduction / baseline_prob
    else:
        relative_reduction = None

    return {
        "absolute_reduction": absolute_reduction,
        "absolute_reduction_pp": absolute_reduction * 100,
        "relative_reduction": relative_reduction,
        "relative_reduction_percent": (
            relative_reduction * 100 if relative_reduction is not None else
None
        )
    }
```

Report:

failure probability reduced by 5.2 percentage points, from 18.0% to 12.8%

Optionally add:

equivalent to a 28.9% relative reduction

13. Ranking interventions

Ranking should use a robust score, not just effect size.

Example scoring:

```
score = benefit_score + evidence_score + actionability_score - risk_penalty
```

Possible components:

benefit_score:

larger failure reduction = higher

evidence_score:

narrow CI, good overlap, refuters passed = higher

actionability_score:

easy/controllable intervention = higher

risk_penalty:

poor overlap, wide CI, possible operational cost = lower

Simple implementation:

```
def rank_intervention(result):
    status = result["recommendation_status"]

    if status == "recommend":
        status_score = 3
    elif status == "recommend_with_caution":
        status_score = 2
    elif status == "inconclusive":
        status_score = 1
    else:
```

```

status_score = 0

effect_pp = result.get("absolute_risk_reduction_pp", 0)
ci_width = result.get("ci_width_pp", 999)

score = (
    status_score * 100
    + effect_pp * 5
    - ci_width
)

return score

```

This is a heuristic. Document it as such.

14. Example recommendation table

Rank	Intervention	Estimated new risk	Reduction	95% CI	Status
1	Increase test coverage to ≥80%	12.8%	-5.2 pp	-8.1 to -2.0 pp	Recommend
2	Add rollback plan	14.5%	-3.5 pp	-6.8 to -0.4 pp	Recommend
3	Move to normal-hours deployment	16.2%	-1.8 pp	-4.9 to +1.3 pp	Caution
-	Add CAB review	N/A	N/A	N/A	Not estimable: poor overlap

This is the kind of output the CRP layer can use.

15. Natural language recommendation

A good generated recommendation:

Recommendation: Increase test coverage to at least 80% before deployment.

Estimated impact:

For comparable historical changes, high test coverage was estimated to reduce failure probability by 5.2 percentage points, from 18.0% to 12.8%.

Uncertainty:

The 95% bootstrap confidence interval was 2.0 to 8.1 percentage points.

Evidence:

The estimate was stable across logistic marginal effects and IPW. Propensity overlap and weighted covariate balance were acceptable. Placebo and subset refuters did not materially change the conclusion.

Limitations:

This estimate depends on the stated DAG assumptions. Team maturity and true code complexity may be imperfectly measured.

Bad recommendation:

Increase test coverage. It will reduce risk by 28.9%.

Why bad:

- overclaims certainty
- uses relative percent only
- omits assumptions
- omits uncertainty
- omits support diagnostics

16. What-if simulation for multiple interventions

Suppose we have multiple fitted treatment models:

- high_test_coverage model
- rollback_plan model
- normal_hours_deployment model

For a given change, simulate each intervention separately.

```
def simulate_interventions(
    change_row,
    intervention_models
):
    results = []

    for intervention_name, model_bundle in intervention_models.items():
        result = simulate_single_intervention(
            change_row=change_row,
            model_bundle=model_bundle
        )
        results.append(result)

    results = sorted(
        results,
        key=lambda x: x.get("ranking_score", 0),
        reverse=True
    )

    return results
```

Each `model_bundle` should include:

```
fitted model
treatment column
covariate columns
ATE estimate
CI
diagnostics
recommendation status
```

17. Important: one model per treatment

Do not assume one causal model estimates every intervention cleanly.

Each treatment may need its own DAG and adjustment set.

Example:

Treatment 1: high test coverage

Confounders:

```
change complexity
priority
assignment group history
environment
change type
```

Treatment 2: CAB involvement

Confounders:

```
priority
environment
service criticality
change type
policy rules
change complexity
```

Treatment 3: off-hours deployment

Confounders:

```
priority
urgency
deployment environment
service criticality
release type
business calendar
```

Same outcome, different treatment, different adjustment logic.

The CRP engine should store a separate causal specification per intervention.

18. Intervention registry

Create a registry.

```
INTERVENTION_REGISTRY = {
  "high_test_coverage": {
    "treatment_col": "high_test_coverage",
```

```

    "outcome_col": "failure",
    "type": "binary",
    "actionable": True,
    "description": "Increase test coverage to at least 80%",
    "confounders": [
        "assignment_group_failure_rate_90d",
        "change_frequency_90d",
        "scheduled_duration_hours",
        "weekend_deployment",
        "off_hours_deployment"
    ],
    "minimum_sample_size": 500,
    "minimum_failures": 50
},
"rollback_plan_present": {
    "treatment_col": "rollback_plan_present",
    "outcome_col": "failure",
    "type": "binary",
    "actionable": True,
    "description": "Add a rollback plan before deployment",
    "confounders": [
        "assignment_group_failure_rate_90d",
        "change_frequency_90d",
        "scheduled_duration_hours",
        "priority_encoded",
        "environment_encoded"
    ],
    "minimum_sample_size": 500,
    "minimum_failures": 50
}
}

```

This makes the engine extensible.

19. What-if API design

A simple API input:

```

{
  "change_id": "CHG12345",
  "features": {

```

```

    "priority": "High",
    "change_type": "Standard",
    "category": "Database",
    "environment": "Production",
    "test_coverage": 45,
    "rollback_plan_present": 0,
    "deployment_window": "Off-hours",
    "assignment_group_failure_rate_90d": 0.12,
    "change_frequency_90d": 28
  }
}

```

Output:

```

{
  "change_id": "CHG12345",
  "baseline_failure_probability": 0.18,
  "recommendations": [
    {
      "intervention": "high_test_coverage",
      "description": "Increase test coverage to at least 80%",
      "simulated_failure_probability": 0.128,
      "absolute_reduction_pp": 5.2,
      "ci_95_pp": [2.0, 8.1],
      "status": "recommend"
    },
    {
      "intervention": "rollback_plan_present",
      "description": "Add a rollback plan before deployment",
      "simulated_failure_probability": 0.145,
      "absolute_reduction_pp": 3.5,
      "ci_95_pp": [0.4, 6.8],
      "status": "recommend"
    }
  ],
  "unavailable_interventions": [
    {
      "intervention": "cab_involvement",
      "reason": "not_estimable_due_to_poor_overlap"
    }
  ]
}

```

```
]
}
```

This is a practical integration target.

20. Combining with existing ML and LLM layers

The proposal says the causal engine should sit alongside the existing ML and LLM components rather than replace them. `Causal_Inference_Internship_Proposal.docx`

A clean architecture:

ML model:
predicts baseline risk

Causal engine:
estimates intervention effects

LLM:
turns structured evidence into readable recommendations

Example flow:

1. ML model says: High risk, 18% estimated failure probability.
2. Causal engine says: `high_test_coverage` reduces comparable failure risk by 5.2 pp.
3. LLM says: "Increase test coverage to at least 80%; historically this intervention..."

The LLM should not invent the causal number. It should only verbalize structured causal output.

21. Guardrails for the LLM layer

The LLM should receive structured evidence:

```
{
  "intervention": "high_test_coverage",
  "effect_pp": -5.2,
  "ci_95_pp": [-8.1, -2.0],
  "status": "recommend",
}
```

```
"assumptions": ["..."],  
"limitations": ["..."]  
}
```

And generate text constrained by rules:

1. Include effect size.
2. Include confidence interval.
3. Include "under stated DAG assumptions."
4. Say "estimated", not "proven."
5. Avoid causal claims for unavailable interventions.
6. Use percentage points for absolute risk changes.

Bad LLM output:

This will definitely prevent failure.

Good LLM output:

Under the stated DAG assumptions, this intervention is estimated to reduce failure probability by 5.2 percentage points for comparable historical changes.

22. Graceful degradation

Sometimes causal analysis should not run.

Reasons:

missing treatment variable
missing key confounders
too few treated examples
too few control examples
too few failures
poor overlap
failed refuters
wide confidence interval
non-actionable intervention

In those cases, output:

```
causal_analysis_available = false
```

with reason.

Example:

```
{  
  "intervention": "cab_involvement",  
  "causal_analysis_available": false,  
  "reason": "poor_overlap",  
  "message": "Nearly all comparable high-priority production changes already  
had CAB involvement, so historical data does not support a reliable no-CAB  
counterfactual."  
}
```

The proposal's missing-data section already expects graceful degradation when key causal variables are missing. `Causal_Inference_Internship_Proposal.docx`

23. What-if simulation should reject impossible interventions

Not every intervention is realistic.

Example:

```
Change is already deployed.
```

Then:

```
increase pre-deployment test coverage
```

may no longer be actionable.

Example:

```
Emergency security patch due immediately.
```

Then:

```
delay deployment by 3 days
```

may be operationally impossible.

Example:

```
Policy requires CAB for production database changes.
```

Then:

```
remove CAB involvement
```

may be invalid.

The engine needs actionability checks.

24. Actionability check function

```
def is_intervention_actionable(change_row, intervention_name):
    if change_row.get("status") in ["Deployed", "Closed", "Cancelled"]:
        return False, "change_already_finalized"

    if intervention_name == "high_test_coverage":
        if change_row.get("test_coverage", None) is None:
            return False, "test_coverage_missing"
        if change_row["test_coverage"] >= 80:
            return False, "already_satisfied"

    if intervention_name == "rollback_plan_present":
        if change_row.get("rollback_plan_present") == 1:
            return False, "already_satisfied"

    return True, "actionable"
```

This avoids recommending things that are already true or impossible.

25. Prioritizing controllable interventions

A useful hierarchy:

Strongly controllable

```
increase test coverage
add rollback plan
add approval/review step
change deployment window
perform additional validation
```

Partially controllable

```
CAB involvement
assignment group
deployment batching
release timing
```

Not directly controllable

```
historical group failure rate
past incident count
change category
service criticality
legacy system complexity
```

Do not recommend:

```
Reduce assignment_group_failure_rate_90d.
```

Instead recommend interventions that may improve future history:

```
Require rollback plan and higher test coverage for assignment groups with
elevated recent failure rates.
```

26. What-if simulation for continuous test coverage

The simple PoC binarizes:

```
high_test_coverage = test_coverage >= 80
```

But the proposal example says:

Increasing test coverage from 45% to 80%

That is continuous.

There are two options.

Option 1: Binary threshold

Treat the intervention as:

move from below 80 to at least 80

Output:

Raising test coverage above the 80% threshold is estimated to reduce failure probability by X percentage points.

This is easiest.

Option 2: Dose-response model

Estimate:

$P(\text{failure} \mid \text{do}(\text{test_coverage} = t))$

for different t .

Example:

Test coverage	Estimated failure probability
40%	16.0%
60%	13.5%
80%	10.8%
90%	10.2%

This is more advanced. Do not start here unless required.

For the first PoC, use threshold treatment.

27. Multi-intervention simulation

What if the user asks:

What if we both increase test coverage and add rollback plan?

Naive approach:

```
combined reduction = test coverage effect + rollback plan effect
```

Example:

```
5.2 pp + 3.5 pp = 8.7 pp
```

But this can be wrong because effects may interact.

Example:

```
rollback plan helps most when test coverage is low  
test coverage helps less when rollback planning is strong
```

For the PoC, avoid strong claims about combined interventions unless you explicitly model them.

Safer output:

```
Individually, high test coverage is estimated to reduce failure probability by  
5.2 pp, and rollback planning by 3.5 pp. The combined effect may not equal  
their sum because interventions can interact.
```

Later, model joint treatment:

```
T = high_test_coverage AND rollback_plan_present
```

or include interaction terms.

28. Recommendation explanation: WHY vs HOW MUCH

The proposal distinguishes existing LLM recommendations from causal-backed recommendations. The causal engine should provide both:

WHY = which factor appears causally relevant
HOW MUCH = estimated failure reduction

Example:

WHY:

Historical comparable changes with low test coverage had higher failure probability after adjusting for priority, change type, environment, assignment group history, and deployment timing.

HOW MUCH:

Raising test coverage above 80% is estimated to reduce failure probability by 5.2 percentage points, with a 95% CI of 2.0 to 8.1 percentage points.

This is stronger than:

Test coverage is correlated with success.

29. Final PoC deliverable structure

Your final PoC should contain:

1. Dataset profile
2. Causal question definitions
3. Treatment registry
4. Outcome definition
5. DAG assumptions
6. Estimation methodology
7. Effect estimates
8. Robustness diagnostics
9. What-if simulation demo
10. Quantified recommendation demo
11. Limitations
12. Production integration sketch

That maps directly to the proposal's PoC deliverables and integration deliverables: validated causal DAG, prioritized controllable intervention candidates, PoC report, causal inference prototype, what-if simulation, quantified recommendations, and final report.

Causal_Inference_Internship_Proposal.docx

30. Minimal end-to-end demo output

Your demo should show one example change request.

Input Change:

CHG12345

Priority: High

Type: Normal

Environment: Production

Test coverage: 45%

Rollback plan: No

Deployment window: Off-hours

Assignment group failure rate 90d: 12%

ML Baseline:

High risk

Estimated failure probability: 18.0%

Causal Recommendations:

1. Increase test coverage to $\geq 80\%$

Estimated new failure probability: 12.8%

Absolute reduction: 5.2 percentage points

95% CI: 2.0 to 8.1 percentage points

Status: Recommend

2. Add rollback plan

Estimated new failure probability: 14.5%

Absolute reduction: 3.5 percentage points

95% CI: 0.4 to 6.8 percentage points

Status: Recommend

Unavailable:

CAB involvement was not estimated because comparable non-CAB examples had poor overlap.

Caution:

All estimates are conditional on the stated DAG assumptions and historical-data support.

This is a credible internship PoC demonstration.

31. Minimum implementation checklist

Before coding the actual final version, verify:

1. You have a reliable failure label.
2. You have at least one actionable treatment.
3. Treatment is measured before outcome.
4. You have pre-treatment confounders.
5. You have a domain-reviewed DAG.
6. You can estimate ATE.
7. You can compute confidence intervals.
8. You can check overlap.
9. You can check balance.
10. You can run refuters.
11. You can return structured JSON.
12. You can generate a readable recommendation.
13. You can refuse/skip unsupported interventions.

If all are satisfied, you can start building.

32. What you are now ready to do

At this point, you know enough to begin the PoC.

You are ready to implement:

1. CRP data profiling notebook
2. feature engineering module
3. causal DAG construction
4. first DoWhy model
5. binary outcome effect estimation
6. bootstrap confidence intervals
7. overlap and balance diagnostics
8. refutation tests
9. structured result object
10. what-if recommendation output

You are **not** yet ready to build a production-grade causal inference platform, but you are ready to build a defensible internship-level proof of concept.

33. What to learn after starting implementation

Once implementation begins, learn these as needed:

1. EconML for heterogeneous treatment effects
2. Double Machine Learning
3. causal forests
4. generalized propensity scores for continuous treatments
5. difference-in-differences for policy changes
6. regression discontinuity for threshold rules
7. instrumental variables for quasi-random treatment assignment
8. sensitivity analysis methods such as E-values / Rosenbaum bounds
9. causal discovery limitations
10. production monitoring for causal estimates

Do not front-load these. Build the basic PoC first.

34. Further practice / learning links

Use these next:

Dowhy documentation:

<https://www.pywhy.org/dowhy/>

Dowhy GitHub examples:

https://github.com/py-why/dowhy/tree/main/docs/source/example_notebooks

Microsoft EconML:

<https://www.pywhy.org/EconML/>

Brady Neal - Introduction to Causal Inference:

<https://www.bradyneal.com/causal-inference-course>

The Effect by Nick Huntington-Klein:

<https://theeffectbook.net/>

Causal Inference for the Brave and True:

<https://matheusfacure.github.io/python-causality-handbook/>

Miguel Hernán and James Robins - Causal Inference: What If:

<https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/>

For this project, start with:

1. DoWhy docs
2. Causal Inference for the Brave and True
3. The Effect
4. EconML only after the basic PoC works

35. Final summary

The causal inference part is now complete enough to start implementation.

The minimum viable CRP Causal Inference Engine should:

1. take historical Change Request data
2. define failure outcome
3. define actionable intervention variables
4. construct a domain-driven DAG
5. identify a valid adjustment set
6. estimate intervention effects
7. compute uncertainty
8. run robustness checks
9. reject unsupported causal questions
10. output quantified what-if recommendations

The core principle:

Do not output causal recommendations unless the treatment is actionable, pre-outcome, supported by overlap, and robust enough to report.

For the PoC, begin with:

high_test_coverage → failure

Then add:

rollback_plan_present → failure

Only after those work cleanly, expand to deployment timing, approval workflow, and CAB involvement.

5/29/2026, 10:50:49 AM